

이음 프로젝트의

SW 설계 기술서(SDD)

2026. 05. 03

팀장 정승민
팀원 김수빈
김재현
최정원

이음

SW 설계 기술서(SDD)

목차

1. 개요

- 1.1 목적
- 1.2 시스템 개발 범위
- 1.3 구성 및 개요

2. 개발 환경

- 2.1 개발 및 설계 환경/도구
- 2.2 설계 고려사항
- 2.3 비기능 요구사항 구현 방안

3. 시스템 아키텍처 설계

- 3.1 아키텍처 개요
- 3.2 논리 아키텍처
- 3.3 물리 아키텍처
- 3.4 데이터 흐름 개요
- 3.5 설계 의도

4. 설계 클래스 다이어그램

5. 설계 클래스 명세

6. 공통 설계 패턴

- 6.1 동시성 제어 전략
- 6.2 이벤트 기반 아키텍처
- 6.3 CQRS 패턴 적용
- 6.4 캐싱 전략
- 6.5 멍등성 보장
- 6.6 삭제 정책
- 6.7 AOP 기반 횡단 관심사 분리
- 6.8 트랜잭션 전파 전략
- 6.9 외부 시스템 연동 패턴
- 6.10 보안 및 인가 패턴
- 6.11 페이징 및 정렬 표준
- 6.12 응답 표준화
- 6.13 로깅 전략

7. API 설계

8. 설계 시퀀스 다이어그램

9. 데이터베이스 설계

- 9.1 CORE ERD
- 9.2 도메인 ERD
- 9.3 테이블 정의

10. UI 설계

11. 설계 컴포넌트 다이어그램

- 11.1 컴포넌트 구성도
- 11.2 컴포넌트 의존 관계

12. 설계 디플로먼트 다이어그램

- 12.1 배포 구성도
- 12.2 배포 노드 명세
- 12.3 통신 프로토콜
- 12.4 프리티어 운영 전략
- 12.5 한계 및 향후 확장

13. 요구사항 추적표

1. 개요

1.1 목적

본 문서는 지역 기반 커뮤니티 및 거래 서비스인 "이음" 프로젝트의 소프트웨어 설계를 정의하기 위한 설계 기술서(SDD)이다.

요구사항 명세서(SRS)를 기반으로 시스템의 구조, 구성 요소, 데이터 흐름 및 인터페이스를 구체적으로 정의하여 개발 과정에서 일관된 기준을 제공하는 것을 목적으로 한다.

또한 본 문서는 시스템 설계에 대한 명확한 기준을 제시함으로써 개발 효율성을 향상시키고 유지보수 및 확장성을 고려한 구조 설계를 지원한다.

1.2 시스템 개발 범위

본 프로젝트는 지역 기반 커뮤니티 및 거래 서비스를 제공하는 플랫폼 "이음"의 시스템 설계를 목적으로 한다.

- 포함 범위
 - 회원 관리 및 인증 (회원가입, 로그인, JWT 기반 인증)
 - 활동 지역 설정 및 인증 기능
 - 동네 상점 관리 및 상품/주문/결제 기능
 - 중고 거래 게시글 및 거래 기능
 - 채팅 기능 (거래 및 커뮤니티 연동)
 - 커뮤니티 게시판 기능
 - 리뷰 및 평점 시스템
 - 관리자 기능 (회원 관리, 신고 처리, 상점 승인)
 - 알림 기능

- 제외 범위 (확장 기능)
 - 카풀 기능 (향후 확장 기능으로 분리)

본 설계 문서는 위 기능을 중심으로 시스템 구조 및 동작을 정의하며, 확장 기능은 향후 시스템 확장 시 반영할 수 있도록 고려한다.

1.3 구성 및 개요

본 문서는 요구사항 명세서(SRS)를 기반으로 시스템의 구조와 동작을 구체적으로 설계하기 위해 작성되었다.

- 개발 환경: 시스템 개발 및 설계에 사용되는 기술 스택과 설계 시 고려사항을 정의한다.
- 시스템 아키텍처 설계: 전체 시스템의 구조와 구성 요소 간 관계를 정의한다.
- 클래스 다이어그램: 도메인 객체 및 클래스 간 관계를 정의한다.
- 시퀀스 다이어그램: 주요 기능의 처리 흐름과 객체 간 상호작용을 정의한다.
- API 설계: 시스템 외부 및 내부 인터페이스를 정의한다.
- 클래스 명세: 각 클래스의 속성 및 책임을 상세히 정의한다.
- 데이터베이스 설계: 데이터 구조 및 테이블 관계를 정의한다.
- 컴포넌트/디플로이먼트 다이어그램: 시스템 구성 요소와 배포 구조를 정의한다.
- 요구사항 추적표: 요구사항과 설계 요소 간의 관계를 명확히 한다.
- 이를 통해 요구사항 → 설계 → 구현으로 이어지는 일관된 개발 흐름을 유지하고, 시스템의 안정성과 확장성을 확보하는 것을 목표로 한다.

2. 개발 환경

2.1 개발 및 설계 환경/도구

본 프로젝트의 개발 및 설계 환경은 다음과 같다.

- 개발 환경
 - Backend: Java, Spring Boot
 - Frontend App: React Native
- Frontend Web: React
 - Database: MySQL
 - Cache: Redis
 - Infra Environment: AWS EC2, Docker
 - Storage: AWS S3
- 개발 도구
 - IDE: IntelliJ IDEA, Visual Studio Code
 - 형상관리: Git, GitHub
 - 협업 도구: Notion
- 설계 도구
 - 다이어그램: PlantUML, Draw.io
 - UI 설계: Figma

2.2 설계 고려사항

본 시스템은 지역 기반 커뮤니티 및 거래 서비스를 안정적으로 제공하기 위해 다음과 같은 사항을 고려하여 설계하였다.

- 확장성 (Scalability)
 - 기능 단위로 도메인을 분리하여 구조를 설계
 - 향후 카풀 기능 등 확장 기능 추가를 고려한 구조 적용
- 유지보수성 (Maintainability)
 - Controller / Service / Repository 계층 분리
 - 역할 기반 설계를 통해 코드 변경 영향 최소화
- 성능 (Performance)
 - Redis를 활용한 캐싱 구조 적용
 - 조회 중심 기능에 대한 응답 속도 개선 고려
 - k6, Grafana 등의 모니터링 도구들을 활용한 성능 개선 고려
- 보안 (Security)
 - Spring Security 기반 인증 처리
 - JWT를 활용한 Stateless 인증 구조 적용
 - 비밀번호 암호화 (BCrypt)

2.3 비기능 요구사항 구현 방안

비기능 요구사항을 충족하기 위해 다음과 같은 구현 방안을 적용하였다.

- 인증 및 권한 관리
 - Access Token / Refresh Token 구조 적용
 - Refresh Token은 Redis에 저장하여 보안성 강화
 - 로그아웃 시 토큰 무효화 처리
- 데이터 일관성
 - 결제 처리 시 중복 요청 방지를 위한 Redis Lock 적용

- 트랜잭션 관리를 통한 데이터 무결성 유지
- 실시간 통신
 - WebSocket 기반 채팅 기능 구현
 - 사용자 간 실시간 메시지 전달 지원
- 외부 API 연동 안정성
 - 결제 API(PortOne) 연동 시 예외 처리 및 실패 대응 로직 구성
 - 외부 API 장애 발생 시 서비스 영향 최소화 고려
- 시스템 안정성
 - 예외 처리 로직 표준화
 - 주요 기능에 대한 로그 기록 및 모니터링 고려

3. 시스템 설계 아키텍처 설계

3.1 아키텍처 개요

본 시스템은 계층형 아키텍처(Layered Architecture)를 기반으로 설계되었으며, 각 계층은 역할에 따라 분리되어 유지보수성과 확장성을 고려하였다.

또한 외부 API 및 캐시 시스템을 활용하여 성능과 확장성을 확보하였다.

3.2 논리 아키텍처

- 표현 계층 (Presentation Layer)

표현 계층은 사용자 요청을 수신하고 응답을 반환하는 영역이다.

웹 및 모바일 클라이언트로부터 전달된 요청을 Controller가 수신하며, 요청 데이터 검증 및 인증 정보 확인 후 비즈니스 계층으로 전달한다.

- 주요 구성 요소

- React Native 기반 사용자 화면
- 관리자/사장 회원용 웹 화면
- Spring Boot Controller
- 인증 필터(JWT Filter)

- 비즈니스 계층 (Business Layer)

비즈니스 계층은 시스템의 핵심 비즈니스 로직을 처리하는 영역이다.

각 도메인 서비스는 사용자, 상점, 거래, 결제 등 주요 기능에 대한 로직을 수행한다.

주요 역할

- 회원 인증 및 권한 처리
- 지역 기반 서비스 로직 처리
- 상점 및 상품 관리
- 주문 및 결제 처리
- 거래 및 게시글 관리
- 리뷰 및 알림 처리

- 데이터 접근 계층 (Data Access Layer)

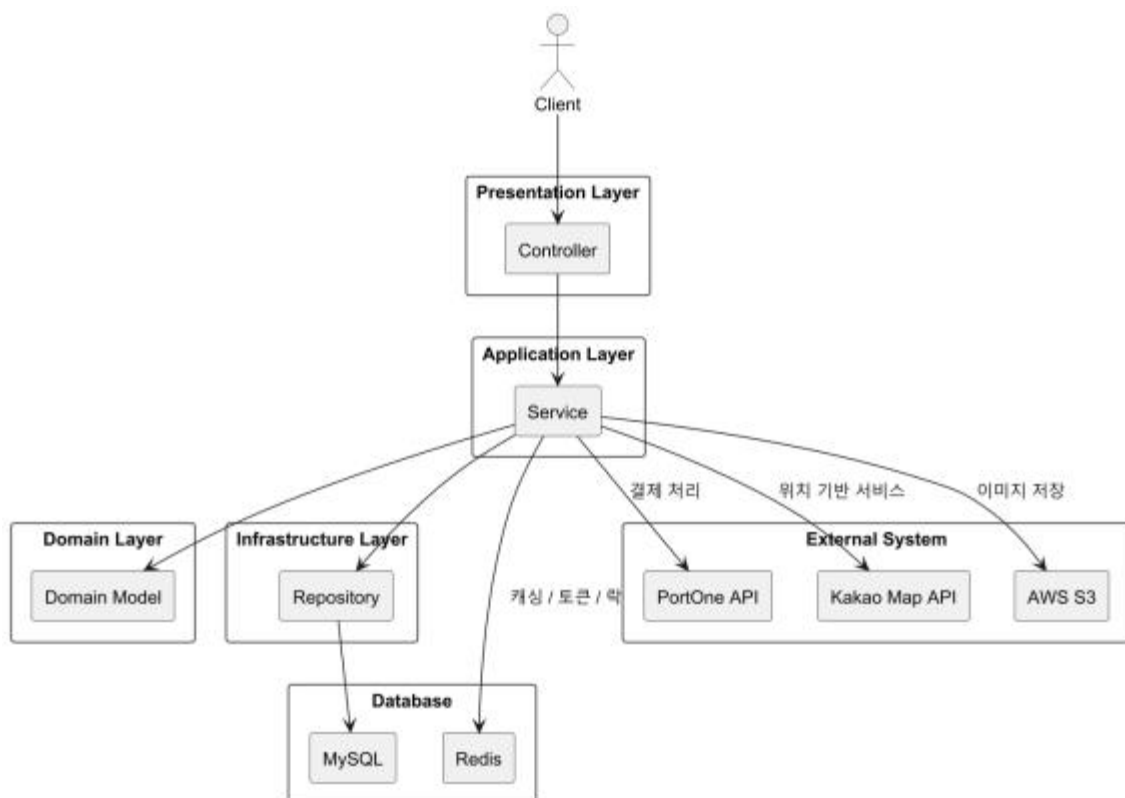
데이터 접근 계층은 데이터베이스와의 상호작용을 담당한다.

Repository를 통해 엔티티를 조회, 저장, 수정, 삭제하며, 비즈니스 계층과 데이터 저장 계층 간의 의존성을 분리한다.

- 데이터 저장 계층 (Data Storage Layer)
 - MySQL: 주요 영속 데이터 저장
 - Redis: 인증 데이터, 캐시, 분산 락 처리
 - AWS S3: 이미지 및 파일 데이터 저장
- 외부 연동 계층 (External Integration)

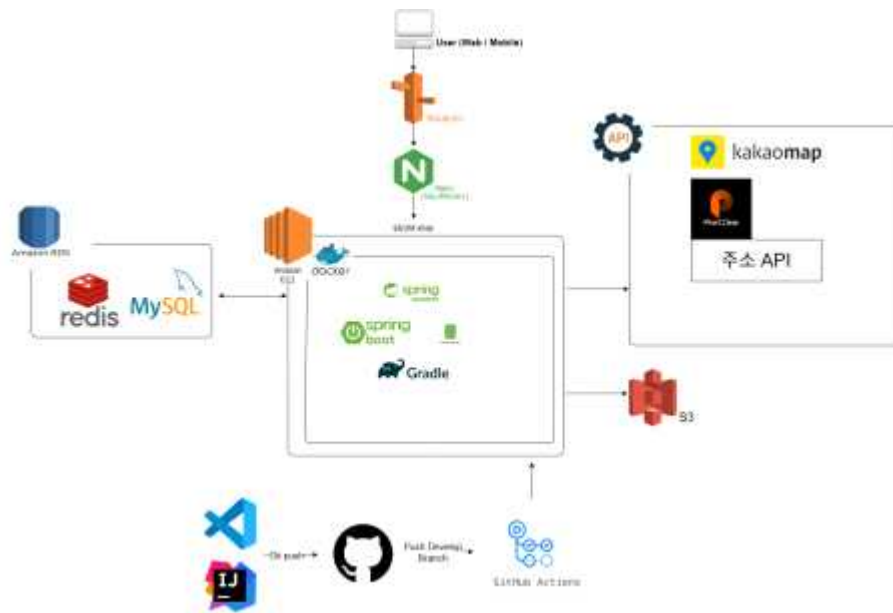
본 시스템은 다음과 같은 외부 서비스와 연동된다.

- PortOne API: 결제 처리
- Kakao Map API: 위치 기반 서비스 제공
- 사업자 인증 API: 사업자 등록 검증
- SMS 인증 API: 사용자 인증
- OAuth Provider: 소셜 로그인 지원



3.3 물리 아키텍처

본 시스템은 AWS EC2 환경에서 Docker 기반으로 배포된 Spring Boot 서버로 구성된다. 데이터 저장을 위해 MySQL을 사용하며, 성능 향상 및 토큰 관리, 동시성 제어를 위해 Redis를 활용하였다. 또한 외부 시스템 연동을 통해 결제, 지도, 지역 인증 기능을 제공하며, 이미지 파일 저장을 위해 AWS S3를 사용하였다.



3.4 데이터 흐름 개요

사용자의 요청은 Controller를 통해 Service로 전달되며, 비즈니스 로직 처리 후 Repository를 통해 DB에 접근한다.

필요 시 Redis 캐싱 및 외부 API 호출이 수행된다.

3.5 설계 의도

- 계층 분리를 통해 유지보수성과 확장성을 확보하였다.
- Redis를 활용하여 조회 성능을 개선 및 토큰 관리를하였다.
- 외부 API를 분리하여 시스템 결합도를 낮추었다.
- Domain 중심 설계를 통해 비즈니스 로직의 응집도를 높이고, 변경 영향 범위를 최소화하였다.
- 서버 중심 구조로 향후 MSA 확장이 가능하도록 설계하였다.

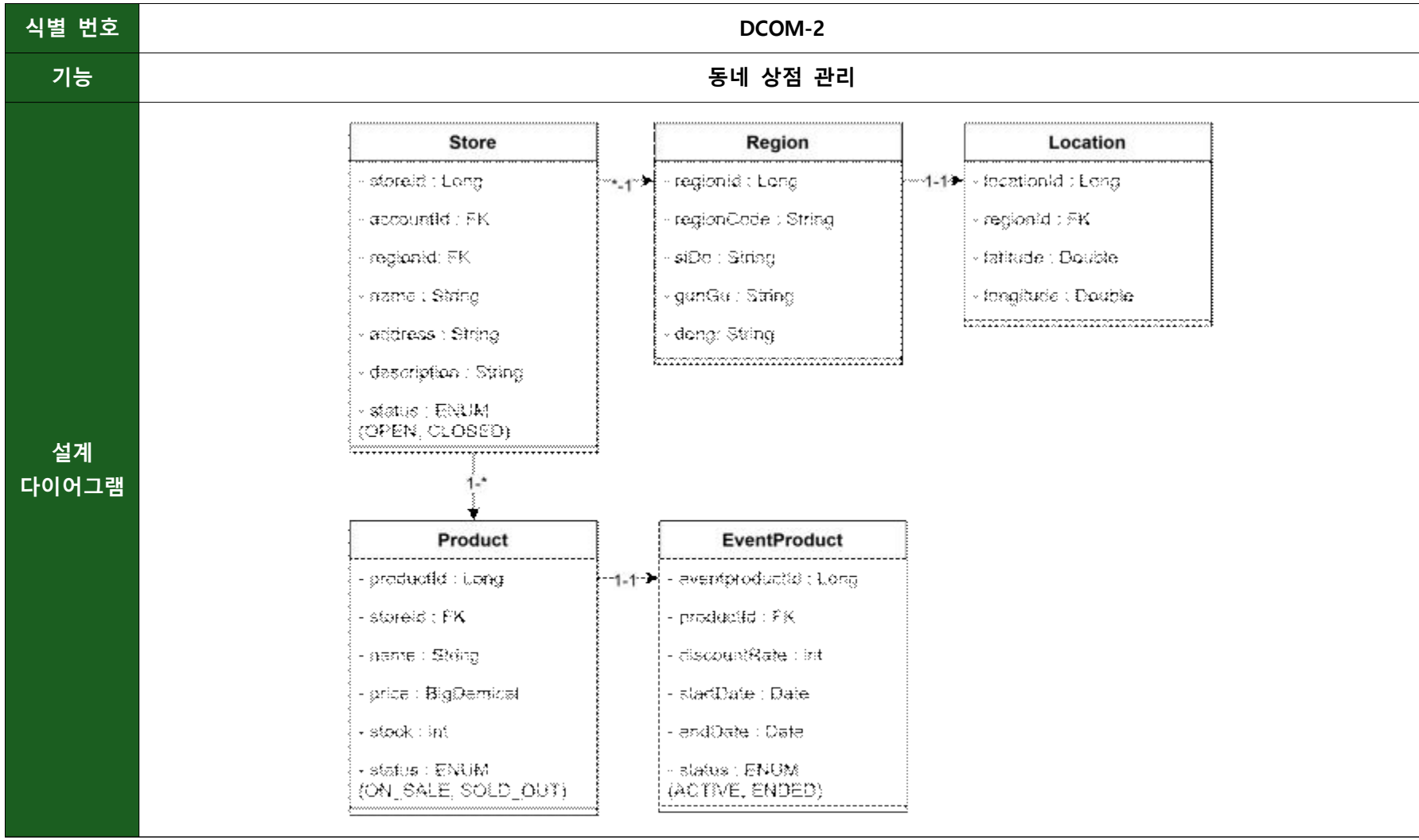
4. 설계 클래스 다이어그램

본 항목에서는 시스템을 구성하는 주요 엔티티와 이들 간의 연관 관계를 중심으로 설계 내용을 기술한다.

각 엔티티의 세부 동작 및 메서드 정의는 하위 명세서에서 별도로 기술한다.

공통 속성인 생성일자(createdAt)와 수정일자(modifiedAt)는 BaseEntity를 통해 관리하도록 설계하였다.

식별 번호	DCOM-1
기능	회원정보 관리
설계 다이아그램	<pre> classDiagram class OwnerInfo { -ownerInfoId : Long -accountId : FK -phone : String -businessNumber : String -approvalStatus : ENUM (PENDING, APPROVED, REJECTED) } class Account { -accountId : Long -email : String -password : String -name : String -nickname : String -password : ENUM (PASSWORD, NICKNAME, NULL) -passwordId : String -role : ENUM (ROLE_USER, ROLE_OWNER, ROLE_ADMIN) -status : ENUM (ACTIVE, SUSPENDED, WITHDRAWN) } class AccountRegion { -accountRegionId : Long -regionId : FK -accountId : FK -verified : boolean -isPrimary : boolean } class Region { -regionId : Long -regionCode : String -city : String -county : String -radius : int -country : String } class Location { -locationId : Long -regionId : FK -latitude : Double -longitude : Double } OwnerInfo "1" -- "1" Account Account "1" -- "1" AccountRegion AccountRegion "1" -- "1" Region Region "1" -- "1" Location </pre> The diagram illustrates the database schema for user management. It includes five main entities: OwnerInfo, Account, AccountRegion, Region, and Location. OwnerInfo is linked to Account (1:1), which is linked to AccountRegion (1:1). AccountRegion is linked to Region (1:1), which is linked to Location (1:1). Each entity has specific attributes, including IDs, foreign keys, and status codes.
설명	본 기능은 사용자의 회원 정보를 관리하기 위한 기능으로, 회원가입, 로그인, 회원 정보 조회 및 수정, 회원 탈퇴 기능을 포함한다. 사용자는 서비스 이용을 위해 계정을 생성하며, 계정 정보는 Account 엔티티를 통해 관리된다. 사장 회원의 경우 추가적인 사업자 정보를 OwnerInfo 엔티티를 통해 관리하며, 일반 사용자와 구분되는 권한을 가진다. 또한 사용자는 활동 지역을 최대 2개까지 등록할 수 있으며, 해당 정보는 AccountRegion 엔티티를 통해 관리된다. 각 활동 지역은 Region 엔티티와 연관되며, 사용자의 실제 위치 기반 인증 여부를 verified 속성으로 관리한다. 지역 기반 서비스 제공을 위해 Region에는 대응되는 위치 정보가 존재하며, 이는 Location 엔티티에 위도(latitude)와 경도(longitude) 형태로 저장된다. 이를 통해 사용자의 현재 위치와 등록된 지역 간의 거리 비교가 가능하도록 설계하였다.



설명

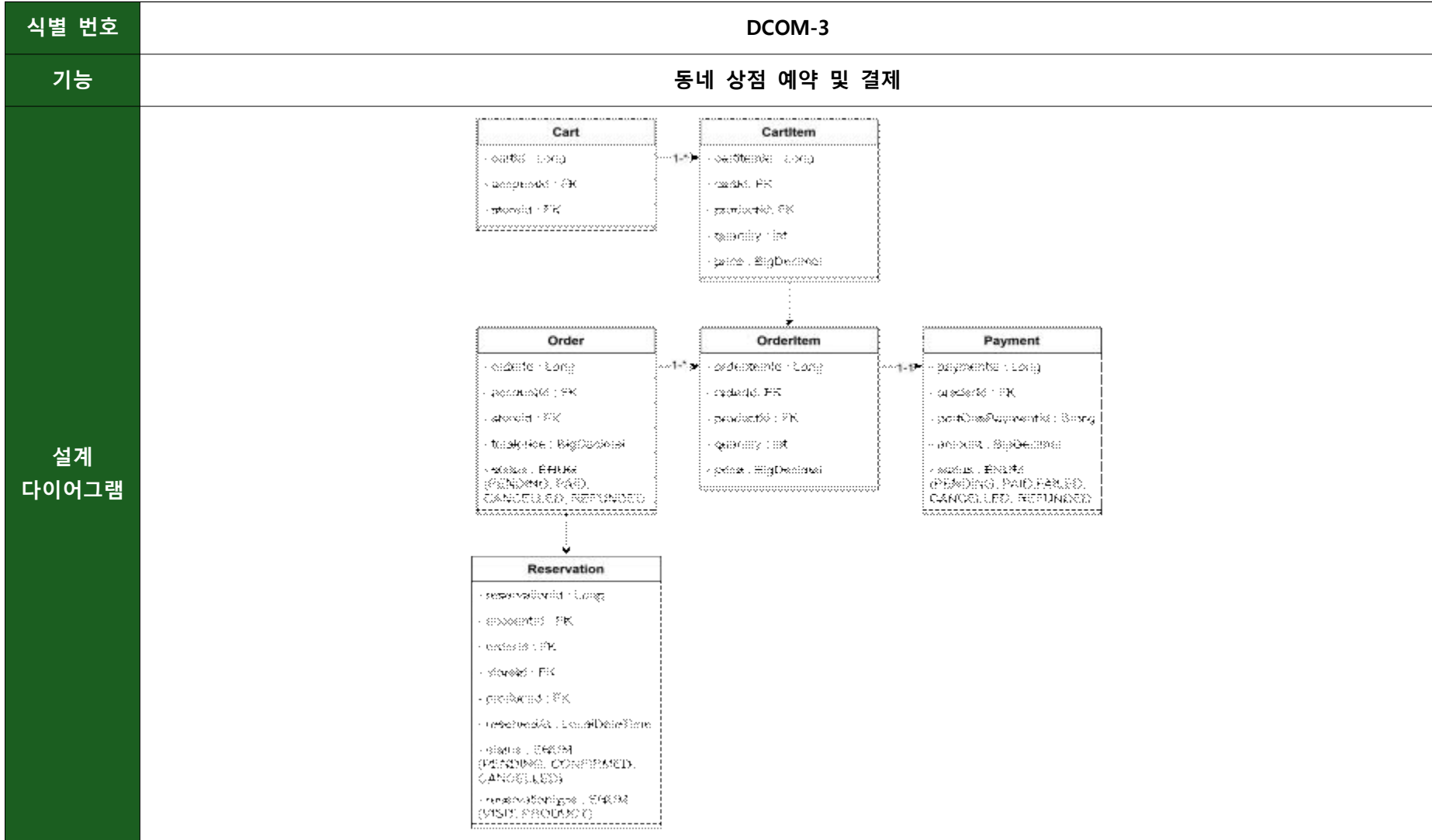
본 기능은 지역 기반 상점 운영을 위한 기능으로, 상점 등록 및 관리, 상품 등록 및 관리, 이벤트 상품 설정 기능을 포함한다.

사장 회원은 Account 엔티티를 기반으로 자신의 상점을 생성 및 관리할 수 있으며, 상점 정보는 Store 엔티티를 통해 관리된다. 각 상점은 특정 지역에 소속되며, Region과의 연관 관계를 통해 사용자에게 지역 기반으로 노출되도록 설계하였다.

상점에는 다수의 상품을 등록할 수 있으며, 상품 정보는 Product 엔티티를 통해 관리된다. 상품은 가격, 재고, 판매 상태 등의 정보를 포함하며, 상점 단위로 관리된다.

또한 특정 상품에 대해 할인 이벤트를 적용할 수 있으며, 해당 정보는 EventProduct 엔티티를 통해 관리된다. 이벤트는 시작일과 종료일을 기준으로 활성 상태를 판단하며, 할인 가격을 통해 사용자에게 혜택을 제공하도록 설계하였다.

이와 같은 구조를 통해 사장 회원은 상점과 상품을 효율적으로 관리할 수 있으며, 사용자는 자신의 지역에 맞는 상점과 상품 정보를 제공받을 수 있도록 하였다.



설명

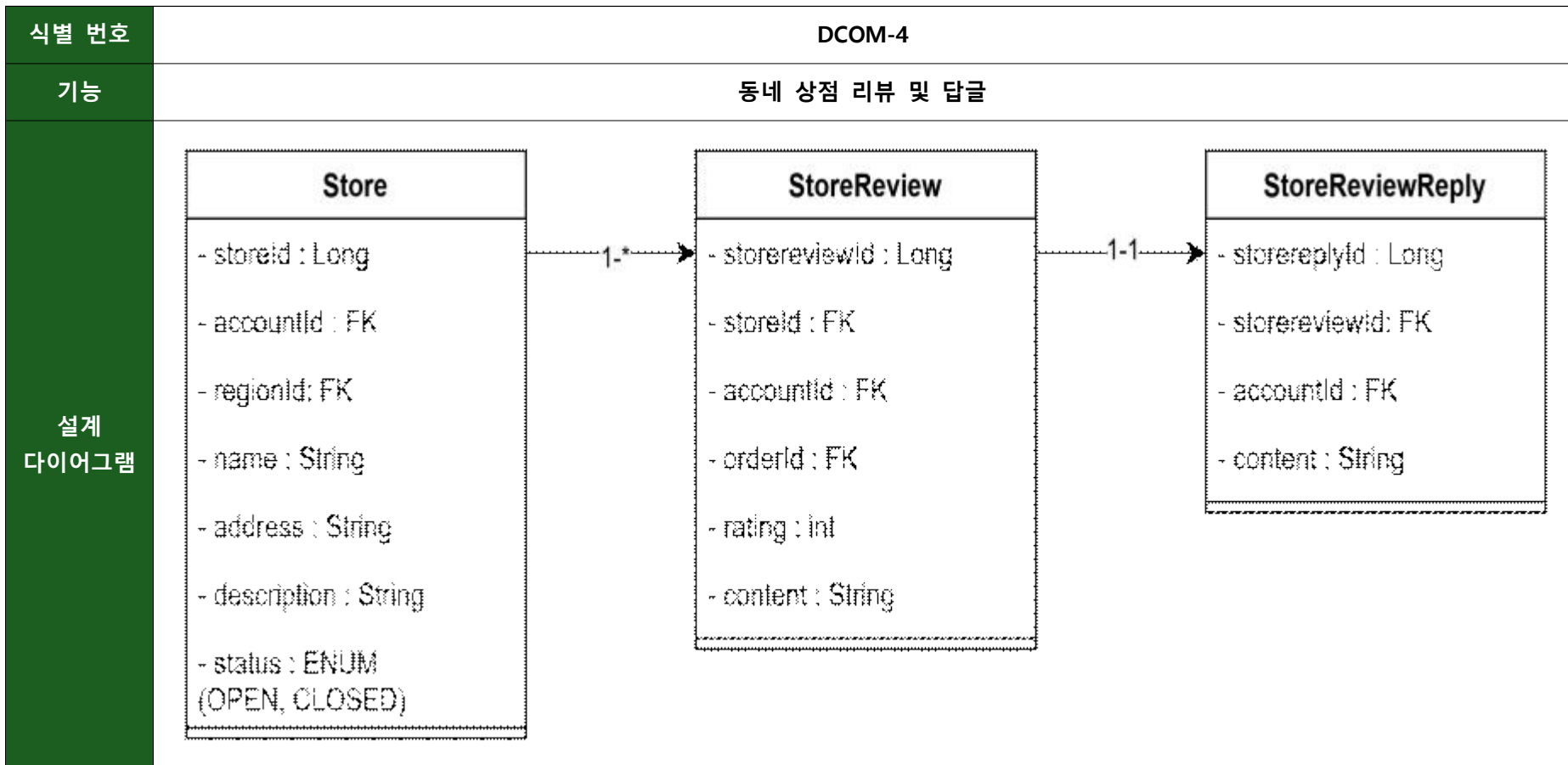
본 기능은 사용자의 상품 주문, 장바구니 관리, 결제 처리 및 예약 기능을 포함하는 기능으로, 상품 선택부터 결제 완료까지의 전반적인 흐름을 관리한다.

사용자는 상품을 장바구니에 담을 수 있으며, 장바구니는 Cart 엔티티를 통해 사용자별로 관리된다. 장바구니는 하나의 가게 단위로 유지되도록 설계하였으며, 다른 가게의 상품을 담을 경우 기존 장바구니 항목을 초기화하고 새로운 가게 기준으로 다시 구성되도록 하였다. 장바구니에 담긴 개별 상품 정보는 CartItem 엔티티를 통해 관리된다.

사용자가 결제를 진행하면 장바구니의 상품 정보는 Order와 OrderItem 엔티티로 변환되어 주문 정보로 저장된다. Order는 주문의 전체 정보를 관리하며, OrderItem은 주문에 포함된 개별 상품의 수량 및 가격 정보를 관리한다.

결제 처리는 Payment 엔티티를 통해 이루어지며, 외부 결제 시스템과 연동하여 결제 상태를 관리한다. 결제 성공 여부에 따라 주문 상태가 갱신되며, 이를 통해 주문의 진행 상태를 일관되게 관리할 수 있도록 설계하였다.

또한 예약 기능은 일반 주문과 구분되는 속성을 가지므로 Reservation 엔티티로 별도로 분리하여 관리하였다. 예약은 특정 주문과 연관되며, 예약 시간 및 예약 상태 정보를 포함한다. 이를 통해 일반 상품 결제와 방문 예약을 하나의 주문 흐름 안에서 유연하게 처리할 수 있도록 설계하였다.



설명	본 기능은 사용자가 상점 이용 후 리뷰를 작성하고, 상점 운영자가 이에 대해 답글을 작성할 수 있도록 하는 기능으로, 상점에 대한 평가 및 리뷰를 관리한다. 사용자는 주문을 통해 상품을 이용한 이후 해당 상점에 대해 리뷰를 작성할 수 있으며, 리뷰 정보는 StoreReview 엔티티를 통해 관리된다. 각 리뷰는 특정 상점(Store)과 사용자(Account), 그리고 주문(Order)과 연관되어 실제 이용 기반의 신뢰성 있는 평가가 가능하도록 설계하였다. 리뷰에는 평점(rating)과 내용(content)이 포함되며, 이를 통해 상점에 대한 사용자 경험을 정량적, 정성적으로 표현할 수 있도록 하였다. 상점 운영자는 작성된 리뷰에 대해 답글을 작성할 수 있으며, 해당 정보는 StoreReviewReply 엔티티를 통해 관리된다. 각 답글은 특정 리뷰와 연관되며, 이를 통해 사용자와 상점 간의 소통 및 피드백 반영이 가능하도록 설계하였다. 이와 같은 구조를 통해 상점에 대한 신뢰도 형성과 사용자 경험 개선을 지원하며, 사용자 간의 평가 정보를 기반으로 서비스 품질을 향상시킬 수 있도록 하였다.
------------------	--

식별 번호	DCOM-5
기능	중고 거래, 중고 거래 리뷰 및 답글
설계 다이어그램	<pre> classDiagram class UsedProduct { - usedproductId : Long - accountId : FK - regionId : FK - title : String - description : String - price : BigDecimal - category : ENUM - viewCount : int - status : ENUM (ON_SALE, RESERVED, SOLD) } class UsedReview { - userreviewId : Long - targetaccountId : FK - accountId : FK - rating : int - content : String } class UsedReviewReply { - usedreviewreplyId : Long - usedreviewId : FK - accountId : FK - content : String } UsedProduct "1" -- "*" UsedReview UsedReview "1" -- "1" UsedReviewReply </pre> The diagram illustrates the database schema for used products, reviews, and replies. It consists of three classes: UsedProduct, UsedReview, and UsedReviewReply. UsedProduct attributes: - usedproductId : Long - accountId : FK - regionId : FK - title : String - description : String - price : BigDecimal - category : ENUM - viewCount : int - status : ENUM (ON_SALE, RESERVED, SOLD) UsedReview attributes: - userreviewId : Long - targetaccountId : FK - accountId : FK - rating : int - content : String UsedReviewReply attributes: - usedreviewreplyId : Long - usedreviewId : FK - accountId : FK - content : String Relationships: UsedProduct (1) to UsedReview (*): A one-to-many relationship. UsedReview (1) to UsedReviewReply (1): A one-to-one relationship.

설명

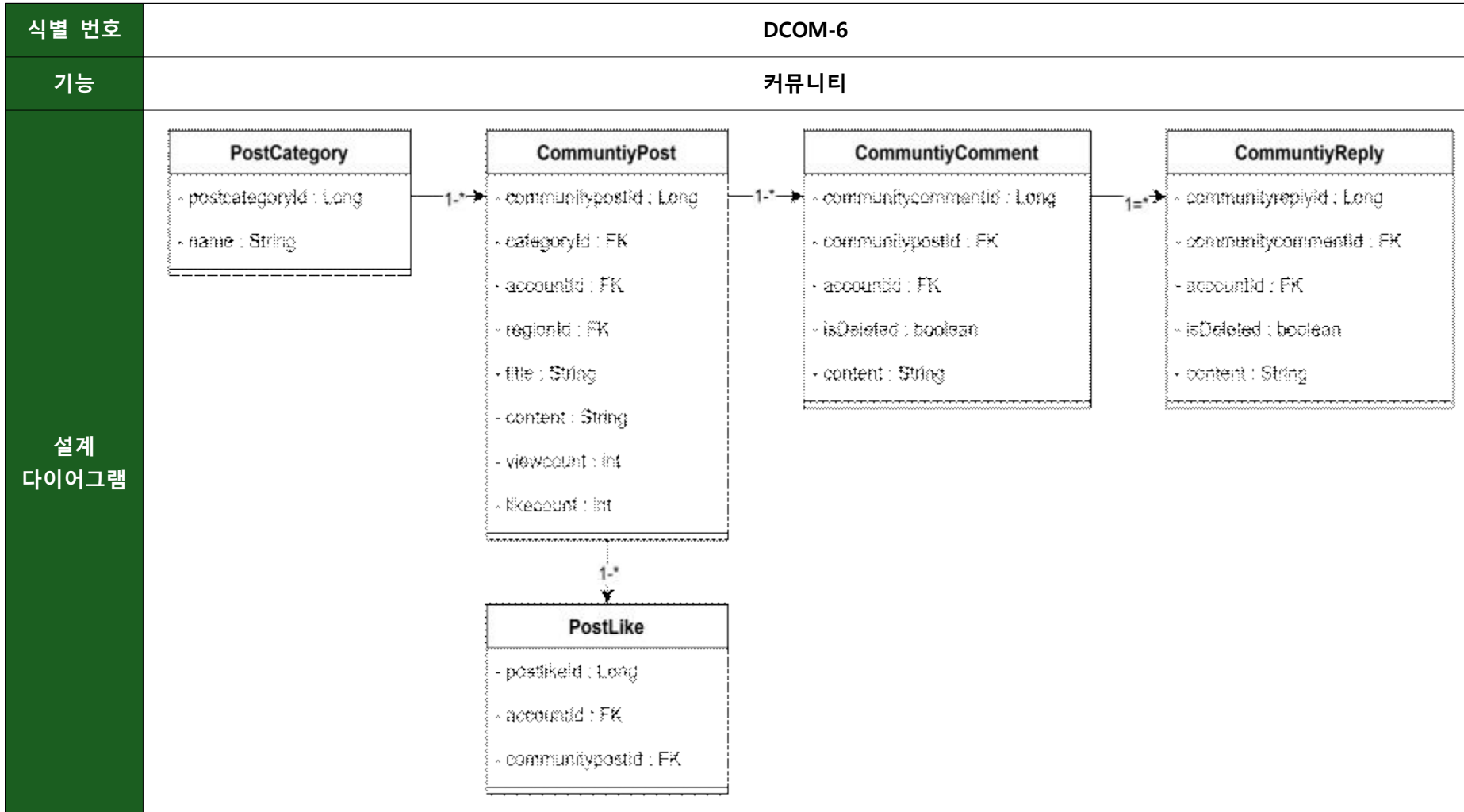
본 기능은 사용자 간 중고 상품 거래를 지원하기 위한 기능으로, 게시글 등록, 조회, 상태 관리 및 거래 이후 리뷰 작성 기능을 포함한다.

사용자는 중고 상품을 등록하여 판매할 수 있으며, 게시글 정보는 UsedProduct 엔티티를 통해 관리된다. 각 게시글은 작성자(Account) 및 지역 정보(Region)와 연관되어 지역 기반으로 노출되도록 설계하였다. 게시글에는 제목, 설명, 가격, 카테고리, 조회수 등의 정보가 포함되며, 거래 상태는 판매 중(ON_SALE), 예약 중(RESERVED), 거래 완료(SOLD)로 구분하여 관리한다.

사용자는 특정 게시글에 대해 거래를 완료한 이후 상대방에 대한 리뷰를 작성할 수 있으며, 해당 정보는 UsedReview 엔티티를 통해 관리된다. 리뷰는 대상 사용자(targetAccountId)와 작성자(Account) 간의 관계를 기반으로 생성되며, 평점(rating)과 내용(content)을 포함하여 거래 경험을 평가할 수 있도록 설계하였다.

또한 작성된 리뷰에 대해 답글을 작성할 수 있으며, 해당 정보는 UsedReviewReply 엔티티를 통해 관리된다. 이를 통해 사용자 간 상호 피드백 및 소통이 가능하도록 하였다.

이와 같은 구조를 통해 지역 기반 중고 거래를 지원하고, 거래 이후 리뷰 시스템을 통해 사용자 간 신뢰도를 형성할 수 있도록 설계하였다.



설명

본 기능은 사용자 간 정보 공유 및 소통을 위한 커뮤니티 게시판 기능으로, 게시글 작성, 댓글 및 대댓글 작성, 좋아요 기능을 포함한다.

사용자는 게시글을 작성할 수 있으며, 게시글 정보는 CommunityPost 엔티티를 통해 관리된다. 각 게시글은 작성자(Account), 카테고리(PostCategory), 그리고 지역 정보(Region)와 연관되어 지역 기반 커뮤니티 기능을 제공하도록 설계하였다. 게시글에는 제목, 내용, 조회수(viewCount), 좋아요 수(likeCount) 등의 정보가 포함된다.

게시글은 카테고리(PostCategory)를 기준으로 분류되며, 이를 통해 사용자들이 관심 있는 주제의 게시글을 쉽게 탐색할 수 있도록 하였다.

사용자는 게시글에 댓글을 작성할 수 있으며, 댓글 정보는 CommunityComment 엔티티를 통해 관리된다. 각 댓글은 특정 게시글과 작성자(Account)와 연관되며, 삭제 여부를 나타내는 isDeleted 속성을 통해 논리적 삭제가 가능하도록 설계하였다.

또한 댓글에 대한 대댓글 기능을 제공하며, 이는 CommunityCommentReply 엔티티를 통해 관리된다. 대댓글 역시 작성자(Account)와 연관되며, 댓글과 동일하게 논리적 삭제를 지원하여 데이터 무결성을 유지하도록 하였다.

사용자는 게시글에 대해 좋아요를 누를 수 있으며, 해당 정보는 PostLike 엔티티를 통해 관리된다. 이를 통해 사용자 간 게시글에 대한 관심도를 표현할 수 있도록 하였으며, 하나의 사용자는 동일 게시글에 대해 중복으로 좋아요를 누를 수 없도록 설계하였다.

이와 같은 구조를 통해 사용자 간 자유로운 소통과 정보 공유를 지원하며, 지역 기반 커뮤니티 활성화를 도모할 수 있도록 설계하였다.

식별 번호	DCOM-7
기능	채팅
설계 다이어그램	<pre> classDiagram class ChatRoom { ~roomId : Long ~refType : String ~refId : Long ~name : String ~isActive : boolean ~type : ENUM(PRIVATE, GROUP, GROUP_STREET) } class ChatParticipant { ~chatparticipantid : Long ~accountid : FK ~roomId : FK ~lastreadtime : LocalDateTime ~isRead : boolean ~status : ENUM(ACTIVE, LEFT) } class ChatMessage { ~chatmessageid : Long ~roomId : FK ~accountid : FK ~content : String ~imageUrl : String ~timestamp : LocalDateTime ~messageTYPE : ENUM(TEXT, IMAGE, SYSTEM) } ChatRoom "1" -- "*" ChatParticipant ChatRoom "1" -- "*" ChatMessage </pre> The diagram illustrates the relationships between three classes: ChatRoom, ChatParticipant, and ChatMessage. ChatRoom is associated with ChatParticipant (1 to many) and ChatMessage (1 to many). ChatParticipant has attributes for chatparticipantid, accountid, roomId, lastreadtime, isRead, and status. ChatMessage has attributes for chatmessageid, roomId, accountid, content, imageUrl, timestamp, and messageTYPE.

설명

본 기능은 사용자 간 실시간 소통을 지원하기 위한 채팅 기능으로, 채팅방 생성, 참여자 관리, 메시지 전송 및 읽음 처리 기능을 포함한다.

채팅방 정보는 ChatRoom 엔티티를 통해 관리되며, 채팅의 유형에 따라 1:1 채팅(PRIVATE), 그룹 채팅(GROUP), 지역 기반 채팅(GROUP_STREET)으로 구분된다. 각 채팅방은 특정 도메인과 연관될 수 있도록 refType과 refId를 통해 상점, 게시글 등 다양한 기능과 연결 가능하도록 설계하였다.

사용자는 채팅방에 참여할 수 있으며, 참여자 정보는 ChatParticipant 엔티티를 통해 관리된다. 각 참여자는 사용자(Account)와 채팅방(ChatRoom)에 연관되며, 마지막으로 메시지를 확인한 시간(lastReadTime)과 참여 상태(status)를 통해 읽음 처리 및 참여 상태를 관리할 수 있도록 설계하였다.

채팅 메시지는 ChatMessage 엔티티를 통해 관리되며, 각 메시지는 채팅방(ChatRoom)과 작성자(Account)와 연관된다. 메시지는 텍스트, 이미지, 시스템 메시지 등 다양한 유형을 지원하도록 messageType으로 구분되며, 전송 시간(timestamp)을 통해 메시지 상태를 관리한다.

또한 이미지 메시지를 지원하기 위해 imageUrl 속성을 포함하였으며, 이를 통해 다양한 형태의 콘텐츠 전송이 가능하도록 설계하였다.

이와 같은 구조를 통해 사용자 간 실시간 소통을 지원하며, 다양한 서비스 기능과 연계 가능한 유연한 채팅 시스템을 구현할 수 있도록 하였다.

식별 번호	DCOM-8
기능	알림
설계 다이어그램	Notification - notificationId : Long - accountId : FK - refType : String - refId : Long - content : String - isRead : boolean - type : ENUM (CHAT, ORDER, REVIEW, RESERVATION, COMMENT, LIKE, INQUIRY, STORE_PRODUCT)

설명

본 기능은 서비스 내에서 발생하는 주요 이벤트를 사용자에게 전달하기 위한 기능으로, 주문 상태 변경, 채팅 메시지 수신, 리뷰 작성, 공지사항 등 다양한 알림을 관리한다.

알림 정보는 Notification 엔티티를 통해 관리되며, 각 알림은 사용자(Account)와 연관되어 개별적으로 전달된다. 알림은 특정 도메인과 연결될 수 있도록 refType과 refId를 통해 주문, 채팅, 게시글, 리뷰 등 다양한 기능과 유연하게 연관되도록 설계하였다.

사용자는 자신에게 전달된 알림 목록을 조회할 수 있으며, 이를 통해 서비스 내에서 발생한 주요 이벤트를 실시간 또는 비동기적으로 확인할 수 있도록 하였다.

또한 알림은 이벤트 발생 시 자동으로 생성되도록 설계하여, 사용자 경험을 향상시키고 서비스의 반응성을 높일 수 있도록 하였다.

이와 같은 구조를 통해 다양한 기능에서 발생하는 이벤트를 통합적으로 관리하고, 사용자에게 필요한 정보를 적시에 전달할 수 있도록 설계하였다.

식별 번호	DCOM-9
기능	찜
설계 다이어그램	Favorite - favoriteId : Long - accountId : FK - targetId : Long - targetType : ENUM (STORE, USED_PRODUCT)
설명	본 기능은 사용자가 관심 있는 상점, 상품을 저장하여 빠르게 다시 확인할 수 있도록 하는 즐겨찾기 기능으로, 관심 대상 등록, 조회 및 해제 기능을 포함한다. 사용자는 서비스 이용 중 관심 있는 대상을 선택하여 즐겨찾기에 추가할 수 있으며, 해당 정보는 Favorite 엔티티를 통해 관리된다. 즐겨찾기 대상은 상점(Store), 상품(Product) 등 다양한 도메인을 포함할 수 있도록 설계하였다. 각 즐겨찾기 정보는 사용자(Account)와 대상 엔티티를 기반으로 생성되며, 대상의 유형을 구분하기 위해 타입 정보를 함께 저장하도록 하였다. 이를 통해 하나의 구조로 다양한 기능의 즐겨찾기를 통합 관리할 수 있도록 설계하였다. 사용자는 저장한 즐겨찾기 목록을 조회할 수 있으며, 이를 통해 관심 있는 상점이나 상품, 게시글에 빠르게 접근할 수 있도록 하였다. 또한 필요에 따라 즐겨찾기 해제 기능을 제공하여 목록을 유연하게 관리할 수 있도록 하였다. 이와 같은 구조를 통해 사용자 맞춤형 정보 접근성을 향상시키고, 서비스 이용 편의성을 높일 수 있도록 설계하였다.

식별 번호	DCOM-10
기능	문의
설계 다이어그램	<pre> classDiagram class Inquiry { -inquiryId : Long -accountId : FK -content : String -status : ENUM (PENDING, ANSWERED) } class InquiryReply { -inquiryReplyId : Long -inquiryId : FK -accountId : FK -content : String } Inquiry "1" --> "*" InquiryReply </pre> The diagram illustrates the relationship between Inquiry and InquiryReply entities. The Inquiry entity contains attributes: <code>- inquiryId : Long</code>, <code>- accountId : FK</code>, <code>- content : String</code>, and <code>- status : ENUM (PENDING, ANSWERED)</code>. The InquiryReply entity contains attributes: <code>- inquiryreplyId : Long</code>, <code>- inquiryId : FK</code>, <code>- accountId : FK</code>, and <code>- content : String</code>. A directed association connects Inquiry to InquiryReply, with a multiplicity of 1 at the Inquiry end and * at the InquiryReply end.

설명

본 기능은 사용자가 서비스 이용 중 발생하는 문의 사항을 등록하고, 관리자 또는 운영자가 이에 대해 답변할 수 있도록 하는 고객 문의 관리 기능으로, 문의 등록, 조회 및 답변 기능을 포함한다.

사용자는 서비스 이용 중 발생한 문제나 궁금한 사항에 대해 문의를 작성할 수 있으며, 문의 정보는 Inquiry 엔티티를 통해 관리된다. 각 문의는 작성자(Account)와 연관되며, 제목과 내용 등의 정보를 포함하여 관리된다.

등록된 문의는 관리자 또는 운영자가 확인할 수 있으며, 이에 대한 답변은 InquiryReply 엔티티를 통해 관리된다. 하나의 문의에 대해 여러 개의 답변이 가능하도록 설계하여 추가 설명이나 보완 답변을 유연하게 제공할 수 있도록 하였다.

각 답변은 특정 문의와 연관되며, 작성자(Account) 정보를 포함하여 누가 답변을 작성했는지 확인할 수 있도록 설계하였다. 이를 통해 사용자와 관리자 간의 원활한 소통이 가능하도록 하였다.

또한 문의 및 답변 상태를 통해 처리 진행 상황을 관리할 수 있으며, 이를 통해 사용자에게 문의 처리 여부를 명확하게 전달할 수 있도록 설계하였다.

이와 같은 구조를 통해 사용자 문의를 체계적으로 관리하고, 서비스 품질 향상 및 사용자 만족도를 높일 수 있도록 설계하였다.

식별 번호	DCOM-11
기능	신고
설계 다이어그램	Report - reportId : Long - accountId : FK - content : String - targetType : ENUM (USER, POST, TRADE, CHAT, Inquiry, InquiryReply) - targetId : Long - status : ENUM (PENDING, PROCESSED, DISMISSED)

설명

본 기능은 서비스 내에서 발생할 수 있는 부적절한 게시글, 댓글, 채팅, 사용자 행위 등을 신고하고 이를 관리하기 위한 기능으로, 사용자 신고 접수 및 관리자 처리 기능을 포함한다.

사용자는 서비스 이용 중 부적절한 콘텐츠나 사용자에게 대해 신고를 할 수 있으며, 신고 정보는 Report 엔티티를 통해 관리된다. 신고 대상은 게시글, 댓글, 채팅 메시지, 사용자 등 다양한 도메인을 포함할 수 있도록 설계하였다.

각 신고는 신고자(Account)와 신고 대상 정보를 포함하며, 신고 사유 및 상세 내용을 기록할 수 있도록 구성하였다. 이를 통해 서비스 내 문제 상황을 체계적으로 수집할 수 있도록 하였다.

관리자는 접수된 신고를 확인하고 처리할 수 있으며, 신고 상태는 대기(PENDING), 처리 완료(RESOLVED), 반려(REJECTED) 등의 상태로 관리된다. 이를 통해 신고 처리 진행 상황을 명확하게 구분할 수 있도록 설계하였다.

또한 신고 대상에 대한 제재 조치(게시글 삭제, 사용자 정지 등)를 통해 서비스 품질을 유지하고, 사용자 간 건전한 커뮤니티 환경을 조성할 수 있도록 하였다.

이와 같은 구조를 통해 사용자 참여 기반의 신고 시스템을 구축하고, 관리자의 효율적인 모니터링 및 대응이 가능하도록 설계하였다.

5. 설계 클래스 명세

5.1 Account 도메인

- Entity

«Entity» Account			
속성명	타입	제약조건	설명
accountId	Long	PK, NOT NULL	회원 ID
primaryRegionId	Long	FK, nullable	대표 지역
email	String	UNIQUE, NOT NULL	로그인 이메일
password	String	nullable	일반 로그인 시 필수, OAuth 로그인 시 "null"
name	String	NOT NULL	실명
provider	OAuthProvider	NOT NULL	OAuth 로그인 시 필수, 일반 로그인 시 LOCAL (KAKAO / NAVER / LOCAL)
providerId	String	nullable	LOCAL → providerId = null, OAuth → providerId NOT NULL
profileImageUrl	String	DEFAULT "/default-profile.png"	사용자 프로필 이미지 URL
nickname	String	UNIQUE, NOT NULL	닉네임
role	AccountRole	NOT NULL	ROLE_USER / ROLE_OWNER / ROLE_ADMIN
status	AccountStatus	NOT NULL	ACTIVE / SUSPENDED / WITHDRAWN
emailVerified	boolean	NOT NULL, DEFAULT false	이메일 인증 여부
fcmToken	String	nullable	푸시 발송용 디바이스 토큰

createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속
deletedAt	LocalDateTime	nullable	탈퇴 시점

«Entity» OwnerInfo			
속성명	타입	제약조건	설명
ownerinfoId	Long	PK, NOT NULL	사장 회원 고유 식별자
accountId	Long	FK, UNIQUE, NOT NULL	연관 회원 (1:1)
phone	String	NOT NULL	사업장 연락처
businessNumber	String	UNIQUE, NOT NULL	사업자 등록번호
approvalStatus	ApprovalStatus	NOT NULL	PENDING / APPROVED / REJECTED
rejectionReason	String	nullable	거절 사유
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» AccountRegion

속성명	타입	제약조건	설명
accountregionId	Long	PK, NOT NULL	회원 지역 고유 식별자
accountId	Long	FK, NOT NULL	연관 회원
regionId	Long	FK, NOT NULL	연관 지역
verified	boolean	NOT NULL	GPS 인증 완료 여부
verifiedAt	LocalDateTime	nullable	인증 시각
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» Region

속성명	타입	제약조건	설명
regionId	Long	PK, NOT NULL	지역 고유 식별자
regionCode	String	UNIQUE, NOT NULL	행정동 코드
siDo	String	NOT NULL	시/도
gunGu	String	NOT NULL	시/군/구
dong	String	NOT NULL	행정동명
radius	int	NOT NULL	서비스 반경(m)

«Entity» Location			
속성명	타입	제약조건	설명
locationId	Long	PK, NOT NULL	좌표 고유 식별자
regionId	Long	FK, NOT NULL	연관 지역(1:1)
latitude	Double	NOT NULL	위도
longitude	Double	NOT NULL	경도

«Entity» AdminAuditLog			
속성명	타입	제약조건	설명
auditlogId	Long	PK, NOT NULL	고유 식별자
accountId	Long	FK, NOT NULL	작업한 관리자 (Account 참조)
actionType	AuditActionType	NOT NULL	작업 종류 ENUM
targetType	String	NOT NULL	대상 도메인 (ACCOUNT/STORE/COMMUNITY_POST 등)
targetId	Long	nullable	대상 ID (전역 작업은 null — 시스템 공지 등)
parameters	String (TEXT)	nullable	메서드 파라미터 JSON 직렬화 (예: 거절 사유)
result	AuditResult	NOT NULL	SUCCESS/FAILURE
failureReason	String	nullable	실패 사유 (예외 메시지)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» **Account (도메인 메서드)**

메서드명	반환타입	설명
activate()	void	상태를 ACTIVE로 변경 (정지 해제 포함)
suspend()	void	상태를 SUSPENDED로 변경
withdraw()	void	상태를 WITHDRAWN으로 변경
updateInfo (nickname, profileImageUrl)	void	회원 정보 수정
changePassword(encoded)	void	암호화된 새 비밀번호로 변경
isActive()	boolean	ACTIVE 상태 여부 반환
verifyEmail()	void	이메일 인증 완료 처리 (emailVerified=true)
updateProfileImage(url)	void	프로필 이미지 URL만 변경
scheduleWithdrawal()	void	탈퇴 예약 (status=WITHDRAWN, deletedAt=now)
isWithdrawn()	boolean	탈퇴 여부 반환
isAdmin()	boolean	관리자 여부 반환
isOwner()	boolean	사장 여부 반환
setPrimaryRegion(regionId)	void	인증된 지역인지 검증 후 설정
clearPrimaryRegion()	void	메인 지역 제거

«Entity» OwnerInfo (도메인 메서드)		
메서드명	반환타입	설명
approve()	void	상태를APPROVED로 변경
reject(reason)	void	상태를REJECTED로 변경, 사유 저장

«Entity» AccountRegion (도메인 메서드)		
메서드명	반환타입	설명
verify()	void	GPS 인증 완료 처리 (verified=true)
isVerified()	boolean	인증 여부 반환

- **Controller**

«Controller» AuthController			
MAP 방식	API명	반환타입	설명
POST	/auth/signup	ResponseEntity<AccountResponseDto>	일반 회원가입 요청 처리
POST	/auth/signup/owner	ResponseEntity<AccountResponseDto>	사장 회원가입 요청 처리
POST	/auth/login	ResponseEntity<TokenResponseDto>	이메일/비밀번호 로그인 및 토큰 발급
GET	/auth/check-email?email=	ResponseEntity<CommonResponseDto>	이메일 중복체크
GET	/auth/check-nickname?nickname=	ResponseEntity<CommonResponseDto>	닉네임 중복 체크
POST	/auth/login/oauth	ResponseEntity<TokenResponseDto>	OAuth 로그인 및 토큰 발급
POST	/auth/token/reissue	ResponseEntity<TokenResponseDto>	Refresh Token을 이용한 Access Token 재발급
POST	/auth/logout	ResponseEntity<CommonResponseDto>	로그아웃 (Redis Refresh Token 삭제)
POST	/auth/reauth	ResponseEntity<ReAuthResponseDto>	사용자 재인증 요청 처리 (비밀번호 또는 OAuth 기반 검증 후 re-auth 토큰 발급)
POST	/auth/email/send-verification	ResponseEntity<CommonResponseDto>	이메일 인증 코드 발송 (회원가입 전 이메일 소유 확인용 6자리 코드, Redis에 이메일과 확인용 코드 임시 저장 TTL 3분)
POST	/auth/email/verify	ResponseEntity<CommonResponseDto>	이메일 인증 코드 검증 (인증 성공 시 짧은 인증 토큰(Redis) 발급, signup 시 이 토큰 함께 전달)
POST	/auth/password/reset-request	ResponseEntity<CommonResponseDto>	비밀번호 재설정 메일 발송 (재설정 토큰 포함 링크)
POST	/auth/password/reset-verify	ResponseEntity<CommonResponseDto>	재설정 토큰 유효성 검증
POST	/auth/password/reset	ResponseEntity<CommonResponseDto>	토큰 기반 비밀번호 재설정 (로그인 없이 가능)

«Controller» AccountController			
MAP 방식	API명	반환타입	설명
GET	/accounts/me	ResponseEntity<MyPageResponseDto>	회원 정보 조회
PATCH	/accounts/me	ResponseEntity<AccountResponseDto>	회원 정보 수정 (닉네임/프로필 이미지 등, 재인증 불필요)
DELETE	/accounts/me	ResponseEntity<CommonResponseDto>	회원 탈퇴 (재인증 토큰 필요)
PATCH	/accounts/me/regions/{regionId}/primary	ResponseEntity<AccountResponseDto>	대표 활동 지역 설정 (Account.primaryRegionId 갱신, 인증된 지역만 가능)
GET	/accounts/me/orders	ResponseEntity<Page<MyOrderResponseDto>>	사용자 동네상점 주문/예약 내역
GET	/accounts/me/orders/{orderId}	ResponseEntity<MyOrderDetailResponseDto>	사용자 동네상점 주문 상세
GET	/accounts/me/store-reviews	ResponseEntity<Page<StoreReviewResponseDto>>	사용자가 동네상점에 작성한 상점 리뷰
GET	/accounts/me/used-reviews/received	ResponseEntity<Page<UsedProductResponseDto>>	받은 중고거래 리뷰
GET	/accounts/me/used-reviews/written	ResponseEntity<Page<UsedProductResponseDto>>	작성한 중고거래 리뷰
GET	/accounts/{accountId}/reviews/received	ResponseEntity<Page<UsedProductReviewResponseDto>>	타인 프로필 조회
GET	/accounts/me/owner	ResponseEntity<OwnerResponseDto>	Owner 승인 상태 표시
POST	/accounts/me/owner	ResponseEntity<OwnerResponseDto>	사업자 정보 등록(ROLE_USER → PENDING 상태로 승인 대기, 관리자 승인 후 ROLE_OWNER로 변경)
PUT	/accounts/me/owner	ResponseEntity<OwnerResponseDto>	사업자 정보 수정
PATCH	/accounts/me/password	ResponseEntity<CommonResponseDto>	비밀번호 변경(재인증 토큰 필요)

«Controller» AccountRegionController			
MAP 방식	API명	반환타입	설명
POST	/accounts/me/regions	ResponseEntity<AccountRegionResponseDto>	활동 지역 등록
GET	/accounts/me/regions	ResponseEntity<List<AccountRegionResponseDto>>	활동 지역 목록 조회
GET	/accounts/me/regions/{regionId}	ResponseEntity<AccountRegionResponseDto>	특정 활동 지역 조회 (대표 지역 여부(Account.primaryRegionId와 비교하여 판단)
PATCH	/accounts/me/regions/{regionId}/verify	ResponseEntity<AccountRegionResponseDto>	활동 지역 GPS 인증 (현재 위치 좌표 검증 후 verified=true 설정)
DELETE	/accounts/me/regions/{regionId}	ResponseEntity<CommonResponseDto>	활동 지역 삭제

«Controller» **AdminAccountController**

MAP 방식	API명	반환타입	설명
GET	/admin/accounts	ResponseEntity<Page<AccountResponseDto>>	회원 목록 조회
GET	/admin/accounts/{accountId}	ResponseEntity<AccountDetailResponseDto>	회원 상세 조회
PATCH	/admin/accounts/{accountId}/suspend	ResponseEntity<CommonResponseDto>	회원 정지 (SUSPENDED)
PATCH	/admin/accounts/{accountId}/activate	ResponseEntity<CommonResponseDto>	회원 정지 해제 (ACTIVE)
GET	/admin/accounts/withdrawn	ResponseEntity<Page<AccountResponseDto>>	탈퇴 예정 회원 관리
PATCH	/admin/accounts/{accountId}/withdrawal/cancel	ResponseEntity<CommonResponseDto>	탈퇴 취소 (사용자 요청 시)
DELETE	/admin/accounts/{accountId}	ResponseEntity<CommonResponseDto>	강제 탈퇴 (30일 유예 무시)
GET	/admin/owners/applications	ResponseEntity<Page<OwnerResponseDto>>	사장 승인 신청 목록 (PENDING)
GET	/admin/owners/{ownerInfold}	ResponseEntity<OwnerResponseDto>	사장 신청 상세 조회
PATCH	/admin/owners/{ownerInfold}/approve	ResponseEntity<CommonResponseDto>	사장 승인 (APPROVED)
PATCH	/admin/owners/{ownerInfold}/reject	ResponseEntity<CommonResponseDto>	사장 거절 (REJECTED + 사유)

«Controller» **AdminAuditLogController**

MAP 방식	API명	반환타입	설명
GET	/admin/audit-logs	Page<AdminAuditLogResponseDto>	감사 로그 목록 (필터: adminId/actionType/targetType/기간)
GET	/admin/audit-logs/{auditlogId}	ResponseEntity<AdminAuditLogDetailResponseDto>	감사 로그 상세
GET	/admin/audit-logs/admin/{accountId}	ResponseEntity<Page<AdminAuditLogResponseDto>>	특정 관리자 활동 이력
GET	/admin/audit-logs/target	ResponseEntity<Page<AdminAuditLogResponseDto>>	특정 대상 변경 이력 (?targetType=ACCOUNT&targetId=123)

- Service

«Service» AuthService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
signup(SignupRequestDto)	AccountResponseDto	REQUIRED	EmailVerified	입력값 검증→ 이메일 중복 확인→ BCrypt 암호화→ Account 저장(ACTIVE)
signupOwner(OwnerSignupRequestDto)	AccountResponseDto	REQUIRED	EmailVerified + ExternalApi	사업자번호 외부API 검증→ Account 저장(PENDING) → OwnerInfo 저장 (@Transactional)
sendVerificationEmail(email)	void	-	RateLimit(1분당 1회)	6자리 인증코드 생성 → Redis 저장(TTL 3분) → 이메일 발송
verifyEmailCode(email, code)	void	-	-	Redis 코드 조회/비교 → 성공 시 인증 완료 상태 저장(TTL 30분)
login(LoginRequestDto)	TokenResponseDto	-	RateLimit(IP당 5분 5회)	이메일/비밀번호 검증→ ACTIVE 확인→ Access+Refresh Token 발급→ Redis 저장
loginOAuth(OAuthLoginRequestDto)	TokenResponseDto	REQUIRED	ExternalApi	OAuth 코드로 사용자 조회→ 없으면 신규 생성→ 토큰 발급
logout(accessToken, refreshToken)	void	-	-	로그아웃 처리 (Refresh Token 삭제 + Access Token 블랙리스트 등록)
reissueToken(accessToken)	TokenResponseDto	-	-	Redis Refresh Token 검증→ 새 Access Token 발급
reAuth(accountId, ReAuthRequestDto)	ReAuthResponseDto	readOnly	-	사용자 재인증 처리 (비밀번호 또는 OAuth 검증 후 re-auth 토큰 발급)
requestPasswordReset(email)	void	-	RateLimit(이메일당 5분 1회)	재설정 토큰 생성 → Redis 저장 → 재설정 링크 이메일 발송
verifyPasswordResetToken(token)	void	-	-	재설정 토큰 유효성 검증
resetPassword(token, newPassword)	void	REQUIRED	-	토큰 검증 후 비밀번호 재설정 (로그인 없이)
checkEmailDuplicate(email)	boolean	readOnly	-	이메일 중복 여부 확인

checkNicknameDuplicate(nickname)	boolean	readOnly	-	닉네임 중복 여부 확인
----------------------------------	---------	----------	---	--------------

«Service» AccountService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getMyInfo(accountId)	AccountResponseDto	readOnly	-	회원 기본 정보 조회
updateInfo(accountId, UpdateInfoRequestDto)	AccountResponseDto	REQUIRED	-	회원 정보 수정 (닉네임/프로필 이미지 등, 재인증 불필요)
changePassword(accountId, ChangePasswordRequestDto)	void	REQUIRED	ReAuthRequired	BCrypt 암호화→ 비밀번호 변경→ 전체 Refresh Token 삭제 (강제 로그아웃)
withdraw(accountId, ReAuthTokenDto)	void	REQUIRED	ReAuthRequired + Event	→ re-auth 토큰 검증 → status=WITHDRAWN + deletedAt 기록 + Refresh Token 삭제
deleteWithdrawnAccountsAfter30Days()	int	REQUIRED	Internal	Scheduler 호출용 — deletedAt < now() - 30일인 WITHDRAWN 계정 일괄 삭제
getOwnerInfo(accountId)	OwnerResponseDto	readOnly	-	내 사업자 정보 및 승인 상태 조회
registerOwnerInfo(accountId, OwnerInfoDto)	OwnerResponseDto	REQUIRED	ExternalApi	사업자 정보 등록 (PENDING 상태로)
updateOwnerInfo(accountId, OwnerInfoDto)	OwnerResponseDto	REQUIRED	ExternalApi	사업자 정보 수정 (승인 상태 재검토)

«Service» **AdminAccountService**

메서드명	반환타입	트랜잭션	정책 / AOP	설명
getAccountList(Pageable,filter)	Page<AccountResponseDto>	readOnly	Admin	회원 목록 조회 (status/role 필터링)
getAccountDetail(accountId)	AccountDetailResponseDto	readOnly	Admin	회원 상세 조회
suspendAccount(accountId)	void	REQUIRED	Admin + @AdminAudit(ACCOUNT_SUSPEND)	회원 정지 처리
activateAccount(accountId)	void	REQUIRED	Admin + @AdminAudit(ACCOUNT_ACTIVATE)	회원 정지 해제
getWithdrawnAccounts(Pageable)	Page<AccountResponseDto>	readOnly	Admin	탈퇴 예정 회원 목록
cancelWithdrawal(accountId)	void	REQUIRED	Admin + @AdminAudit(ACCOUNT_WITHDRAWAL_CANCEL)	탈퇴 취소 (사용자 요청 시 처리)
forceDeleteAccount(accountId)	void	REQUIRED	Admin + @AdminAudit(ACCOUNT_FORCE_DELETE)	강제 탈퇴 (30일 유예 무시, CASCADE 정리)
getOwnerApplications(Pageable)	Page<OwnerResponseDto>	readOnly	Admin	사장 승인 대기 목록
getOwnerApplicationDetail(ownerInfold)	OwnerResponseDto	readOnly	Admin	사장 신청 상세 조회
approveOwner(ownerInfold)	void	REQUIRED	Admin + @AdminAudit(OWNER_APPROVE) + Event	사장 승인 (APPROVED + ROLE_OWNER 변경)
rejectOwner(ownerInfold,reason)	void	REQUIRED	Admin + @AdminAudit(OWNER_APPROVE) + Event	사장 거절(REJECTED + 사유 저장)

«Service» AccountRegionService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
registerRegion(accountId, RegionRequestDto)	AccountRegionResponseDto	readOnly	-	활동 지역 등록 (GPS 위치 기반 검증 후 등록, 동일 지역 중복 등록 방지 및 최대 등록 개수 제한)
getRegion(accountId, regionId)	AccountRegionResponseDto	readOnly	Ownership	특정 활동 지역 조회
setRegion(accountId, regionId)	AccountResponseDto	REQUIRED	MaxRegionLimit(2)	대표 활동 지역 설정 (인증된 지역 여부 검증 후 Account.primaryRegionId 갱신)
verifyRegion(accountId, regionId, LocationDto)	AccountRegionResponseDto	REQUIRED	Ownership + GpsValidation	GPS 위치 검증 → verified=true 설정
getMyRegions(accountId)	List<AccountRegionResponseDto>	REQUIRED	Ownership + Verified	회원의 활동 지역 목록 조회
deleteRegion(accountId, regionId)	void	REQUIRED	Ownership	활동 지역 삭제 (본인 소유 및 인증 여부 검증 후 삭제, primary 지역 삭제 시 다른 지역을 자동으로 primary로 재설정)

«Service» **TokenService**

메서드명	반환타입	트랜잭션	정책 / AOP	설명
generateToken(account)	TokenResponseDto	-	-	Access + Refresh 생성
reissueToken(refreshToken)	TokenResponseDto	-	-	Refresh 검증 후 Access 재발급
invalidateToken(accountId, accessToken)	void	-	-	Refresh Token 삭제, Access Token 블랙리스트 등록
generateReAuthToken(accountId)	ReAuthResponseDto	-	-	민감 작업 수행을 위한 re-auth 토큰을 생성하고 Redis에 TTL 기반으로 저장
validateReAuthToken(token)	Long	-	-	전달받은 re-auth 토큰의 유효성을 검증하고, 유효하면 해당 accountId를 반환
invalidateReAuthToken(accountId)	void	-	-	사용자의 re-auth 토큰을 Redis에서 삭제하여 재사용을 방지
generatePasswordResetToken(email)	String	-	-	Reset 토큰 생성 (UUID + Redis)
validatePasswordResetToken(token)	String	-	-	Reset 토큰 검증 후 email 반환
invalidatePasswordResetToken(token)	void	-	-	Reset 토큰 삭제 (일회용)

«Service» **AdminAuditService**

메서드명	반환타입	트랜잭션	정책 / AOP	설명
recordAction(AuditActionDto)	void	REQUIRED	Internal	감사 로그 저장 (별도 트랜잭션, 본 작업 실패 시에도 로그는 기록)
getAuditLogs(AuditSearchDto, Pageable)	Page<AdminAuditLogResponseDto>	readOnly	Admin	감사 로그 조회 (필터: adminId/actionType/targetType/기간)
getAuditLogDetail(auditlogId)	AdminAuditLogDetailResponseDto	readOnly	Admin	감사 로그 상세
deleteOldAuditLogs(threshold)	int	REQUIRED	Internal	Scheduler에서 호출, 1년 이전 로그 정리

- Repository

«Repository» AccountRepository		
메서드명	반환타입	설명
findById(Long accountId)	Optional<Account>	회원 ID로 회원 조회
findByEmail(String email)	Optional<Account>	이메일로 회원 조회
findByNickname(String nickname)	Optional<Account>	닉네임 조회
existsByEmail(String email)	boolean	이메일 중복 확인
existsByNickname(String nickname)	boolean	닉네임 중복 확인
findByProviderAndProviderId(OAuthProvider provider, String providerId)	Optional<Account>	OAuth 로그인 시 기존 회원 조회
findAllByDeletedAtIsNotNull(Pageable pageable)	Page<Account>	탈퇴 예정 회원 목록 조회(관리자용)
findAllByDeletedAtBefore(LocalDateTime threshold, Pageable pageable)	Page<Account>	30일 지난 탈퇴 회원(물리 삭제 스케줄러용)
findAllByStatus(AccountStatus status, Pageable pageable)	Page<Account>	상태별 회원 목록 조회(관리자용)
findAllByRole(AccountRole role, Pageable pageable)	Page<Account>	역할별 회원 목록 조회(관리자용)

«Repository» OwnerInfoRepository		
메서드명	반환타입	설명
findByAccount_AccountId(Long accountId)	Optional<OwnerInfo>	회원 ID로 사장 회원 정보 조회
existsByBusinessNumber(String businessNumber)	boolean	사업자등록번호 중복 확인
findByBusinessNumber(String businessNumber)	Optional<OwnerInfo>	사업자등록번호로 사장 회원 정보 조회
findAllByApprovalStatus(ApprovalStatus approvalStatus, Pageable pageable)	Page<OwnerInfo>	승인 상태별 사장 회원 목록 조회(관리자용)

«Repository» OwnerInfoRepositoryCustom (QueryDSL)		
메서드명	반환타입	설명
searchOwnerInfos(OwnerInfoSearchDto condition, Pageable pageable)	Page<OwnerInfo>	관리자 복합 필터(approvalStatus / 사업자번호/ 상호명/ 신청일 기간)

«Repository» AccountRegionRepository		
메서드명	반환타입	설명
findAllByAccount_AccountId(Long accountId)	List<AccountRegion>	회원의 활동 지역 목록 조회
findByAccount_AccountIdAndRegion_RegionId(Long accountId, Long regionId)	Optional<AccountRegion>	회원의 특정 활동 지역 조회
existsByAccount_AccountIdAndRegion_RegionId(Long accountId, Long regionId)	boolean	동일 지역 등록 여부 확인
existsByAccount_AccountIdAndRegion_RegionIdAndVerifiedTrue(Long accountId, Long regionId)	boolean	대표 지역 변경 시 본인 인증 지역 검증
countByAccount_AccountId(Long accountId)	Long	회원 활동 지역 개수 조회(최대 2개 제한 확인)
findFirstByAccount_AccountIdAndVerifiedTrueOrderByVerifiedAtDesc(Long accountId)	Optional<AccountRegion>	deleteRegion 후 가장 최근 인증 지역 자동 재설정용
findAllByRegion_RegionId(Long regionId)	List<AccountRegion>	특정 지역을 등록한 회원 목록 조회
deleteByAccount_AccountIdAndRegion_RegionId(Long accountId, Long regionId)	void	회원의 특정 활동 지역 삭제

«Repository» RegionRepository		
메서드명	반환타입	설명
findById(Long regionId)	Optional<Region>	지역 ID로 조회
findByRegionCode(String regionCode)	Optional<Region>	행정동 코드로 조회
findAllBySiDoAndGunGu(String siDo, String gunGu)	List<Region>	시/도 + 시/군/구 기준 지역 조회
findAllByDongContaining(String dong)	List<Region>	행정동명 검색
existsByRegionCode(String regionCode)	boolean	행정동 코드 존재 여부 확인

«Repository» LocationRepository		
메서드명	반환타입	설명
findByRegion_RegionId(Long regionId)	Optional<Location>	지역 ID로 대표 좌표 조회

«Repository» **TokenRepository**

메서드명	반환타입	설명
saveRefreshToken(Long accountId, String refreshToken, Duration ttl)	void	Refresh Token 저장
findRefreshToken(Long accountId)	Optional<String>	Refresh Token 조회
deleteRefreshToken(Long accountId)	void	Refresh Token 삭제
saveReAuthToken(Long accountId, String reAuthToken, Duration ttl)	void	re-auth 토큰 저장
findReAuthToken(Long accountId)	Optional<String>	re-auth 토큰 조회
deleteReAuthToken(Long accountId)	void	re-auth 토큰 삭제
savePasswordResetToken(String token, String email, Duration ttl)	void	비밀번호 재설정 토큰 저장
findPasswordResetToken(String token)	Optional<String>	재설정 토큰으로 이메일 조회
deletePasswordResetToken(String token)	void	재설정 토큰 삭제 (일회용)
saveEmailVerificationCode(String email, String code, Duration ttl)	void	이메일 인증 코드 저장
findEmailVerificationCode(String email)	Optional<String>	이메일로 인증 코드 조회
deleteEmailVerificationCode(String email)	void	인증 코드 삭제
saveEmailVerified(String email, Duration ttl)	void	이메일 인증 완료 상태 저장
isEmailVerified(String email)	boolean	이메일 인증 완료 여부 확인
deleteEmailVerified(String email)	void	이메일 인증 상태 삭제 (회원가입 완료 후)
addTokenBlacklist(String accessToken, Duration ttl)	void	Access Token 블랙리스트 등록 (로그아웃 시)
isTokenBlacklisted(String accessToken)	boolean	Access Token 블랙리스트 확인

«Repository» AdminAuditLogRepository

메서드명	반환타입	설명
findById(Long auditlogId)	Optional<AdminAuditLog>	조회
findAllByAccountIdOrderByCreatedAtDesc(Long accountId, Pageable pageable)	Page<AdminAuditLog>	관리자별 이력
findAllByTargetTypeAndTargetIdOrderByCreatedAtDesc(String targetType, Long, targetId, Pageable pageable)	Page<AdminAuditLog>	대상별 이력
deleteAllByCreatedAtBefore(LocalDateTime threshold)	int	정리 배치용 (AuditLogCleanupScheduler 매월 1일, 1년 이전 삭제)

«Repository» AdminAuditLogRepositoryCustom (QueryDSL)

메서드명	반환타입	설명
searchAuditLogs(AuditSearchDto condition, Pageable pageable)	Page<AdminAuditLog>	복합 필터 (adminId/actionType/targetType/기간)
countByActionTypeAndPeriod(AuditActionType actionType, LocalDateTime from, LocalDateTime to)	Long	통계용 — 작업 유형별 발생 건수

• Aspect

«Aspect» AdminAuditAspect

@Around("@annotation(adminAudit)") log(ProceedingJoinPoint, AdminAudit adminAudit): Object

처리 흐름:

1. SecurityContext에서 adminId 추출
2. 메서드 파라미터 직렬화 (Jackson, 민감정보 마스킹)
3. 메서드 실행
4. 성공: result=SUCCESS, 응답 처리
5. 실패: result=FAILURE, failureReason 기록 후 예외 재던짐
6. AdminAuditService.recordAction() 호출 (별도 트랜잭션)

- **Component**

«Component» AuditLogCleanupScheduler				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
cleanupWithdrawnAccounts()	void	-	@Scheduled(cron = "0 0 2 * * *")	매일 새벽 2시 — AccountService.deleteWithdrawnAccountsAfter30Days() 호출 (deletedAt < now() - 30일인 WITHDRAWN 계정 물리 삭제 + CASCADE)

«Component» AuditLogCleanupScheduler				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
cleanupOldAuditLogs()	void	-	@Scheduled(cron = "0 0 4 1 * *")	매월 1일 새벽 4시 — AdminAuditService.deleteOldAuditLogs(1년 전) 호출

5.2 Store 도메인

- Entity

«Entity» Store			
속성명	타입	제약조건	설명
storeId	Long	PK, NOT NULL	상점 고유 식별자
accountId	Long	FK, UNIQUE, NOT NULL	상점 소유자 (사장 1:1 상점 제약)
regionId	Long	FK, NOT NULL	상점 소속 지역
categoryId	Long	FK, NOT NULL	상점 카테고리
latitude	Double	NOT NULL	상점 위도
longitude	Double	NOT NULL	상점 경도
name	String	NOT NULL	상점명
address	String	NOT NULL	주소
phone	String	NOT NULL	상점 연락처
description	String	nullable	상점 소개
businessHours	String	NOT NULL	영업시간
rating	Double	NOT NULL,Default 0.0	평균 평점 수(캐싱) -> 리뷰 변경 시 갱신
favoriteCount	int	NOT NULL,Default 0	상점 좋아요 수
reviewCount	int	NOT NULL,Default 0	리뷰 수(캐싱) -> 리뷰 변경 시 갱신

status	StoreStatus	NOT NULL	OPEN / CLOSED / TEMP_CLOSED
version	Long	NOT NULL	낙관적 락용 (@Version) — 평점 갱신 동시성 제어
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» StoreNotice			
속성명	타입	제약조건	설명
noticeId	Long	PK, NOT NULL	고유 식별자
storeId	Long	FK, NOT NULL	연관 상점
noticeType	NoticeType	NOT NULL, ENUM	공지 유형 (GENERAL, HOLIDAY, EVENT 등)
content	String	NOT NUL	공지 상세 내용
isActive	boolean	NOT NULL, DEFAULT true	공지 노출 여부 (true: 노출, false: 비노출)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» StoreImage

속성명	타입	제약조건	설명
storeimageld	Long	PK, NOT NULL	이미지 고유 식별자(ImageBase 상속)
storeld	Long	FK,, NOT NULL	연관 상점
imageUrl	String	NOT NULL	이미지 URL(ImageBase 상속)
displayOrder	int	NOT NULL, DEFAULT 0	표시 순서(ImageBase 상속)
isThumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부(ImageBase 상속)
createdAt	LocalDateTime	NOT NULL	ImageBase 상속
modifiedAt	LocalDateTime	NOT NULL	ImageBase 상속

«Entity» ProductCategory			
속성명	타입	제약조건	설명
productcategoryId	Long	PK, NOT NULL	상품 카테고리 고유 식별자
storeId	Long	FK, NOT NULL	소속 상점
name	String	NOT NULL	상품 카테고리
displayOrder	int	NOT NULL, DEFAULT 0	표시 순서
isActive	boolean	NOT NULL, DEFAULT true	노출 여부
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» Product			
속성명	타입	제약조건	설명
productId	Long	PK, NOT NULL	상품 고유 식별자
storeId	Long	FK, NOT NULL	소속 상점
productcategoryId	Long	FK, NOT NULL	상품 카테고리
name	String	NOT NULL	상품명
description	String	NOT NULL	상품 설명
price	BigDecimal	NOT NULL, ≥0	가격
stock	int	NULLABLE, ≥0	재고 수량 (null=무제한)
viewCount	int	NOT NULL, DEFAULT 0	조회수 (Redis 실시간 집계 → 5분 주기 배치 동기화)
status	ProductStatus	NotNull,DEFAULT 'ACTIVE'	ACTIVE/SOLD_OUT
version	Long	NOT NULL	낙관적 락용 (@Version) — 재고 차감 동시성 제어
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» ProductImage			
속성명	타입	제약조건	설명
productimageId	Long	PK, NOT NULL	이미지 고유 식별자(ImageBase 상속)
productId	Long	FK, NOT NULL	연관 상품
imageUrl	String	NOT NULL	이미지 URL(ImageBase 상속)
displayOrder	int	NOT NULL, DEFAULT 0	표시 순서(ImageBase 상속)
isThumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부(ImageBase 상속)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» ProductOption			
속성명	타입	제약조건	설명
productoptionId	Long	PK, NOT NULL	고유 식별자
productId	Long	FK, NOT NULL	연관 상품
groupName	String	NOT NULL	옵션 그룹 이름 (예: 용량, 매운맛)
selectionType	OptionSelectionType	NOT NULL, DEFAULT 0	옵션 선택 방식 (단일 선택 / 다중 선택)
isRequired	boolean	NOT NULL, DEFAULT false	해당 옵션 선택 필수 여부
displayOrder	int	NOT NULL, DEFAULT SINGLE	옵션 항목 노출 순서
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» ProductOptionItem			
속성명	타입	제약조건	설명
productoptionitemId	Long	PK, NOT NULL	고유 식별자
productoptionId	Long	FK, NOT NULL	연관 상품옵션
itemName	String	NOT NULL	옵션 선택 값 (예: 300g, 매운맛)
additionalPrice	BigDecimal	NOT NULL, DEFAULT 0	선택 시 추가되는 금액
isDefault	boolean	NOT NULL, DEFAULT false	기본 선택 여부
displayOrder	int	NOT NULL, DEFAULT 0	옵션 항목 노출 순서
isAvailable	boolean	DEFAULT true	옵션 선택 가능 여부 (품질/비활성 상태)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» EventProduct

속성명	타입	제약조건	설명
eventproductId	Long	PK, NOT NULL	고유 식별자
productId	Long	FK, NOT NULL	연관 상품 (동시에 ACTIVE 1개만)
activeProductId	Long	GENERATED COLUMN, unique	status = 'ACTIVE'인 경우 productId 값, 그 외 NULL → 동일 productId에 ACTIVE 이벤트 1개만 허용
discountRate	int	NOT NULL, 0~99	할인율(%)
stock	int	NULLABLE, ≥0	이벤트 수량 (null=무제한)
startDate	LocalDateTime	NOT NULL	이벤트 시작일
endDate	LocalDateTime	NOT NULL	이벤트 종료일
status	EventStatus	NOT NULL	ACTIVE / ENDED
version	Long	NOT NULL	낙관적 락용 (@Version) — 이벤트 재고 차감
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» StoreReview

속성명	타입	제약조건	설명
storereviewId	Long	PK, NOT NULL	고유 식별자
storeId	Long	FK, NOT NULL	연관 상점
accountId	Long	FK, NOT NULL	작성자
orderId	Long	FK, NOT NULL, UNIQUE	1주문 1리뷰 보장
rating	int	NOT NULL, 1~5	별점
content	String	NOT NULL	리뷰 내용
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» **StoreReviewImage**

속성명	타입	제약조건	설명
storereviewimageId	Long	PK, NOT NULL	이미지 고유 식별자(ImageBase 상속)
storeReviewId	Long	FK, NOT NULL	연관 상점 리뷰
imageUrl	String	NOT NULL	이미지 URL(ImageBase 상속)
displayOrder	int	NOT NULL, DEFAULT 0	표시 순서(ImageBase 상속)
isThumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부(ImageBase 상속)
createdAt	LocalDateTime	NOT NULL	ImageBase 상속
modifiedAt	LocalDateTime	NOT NULL	ImageBase 상속

«Entity» **StoreReviewReply**

속성명	타입	제약조건	설명
storereplyId	Long	PK, NOT NULL	고유 식별자
storereviewId	Long	FK, UNIQUE, NOT NULL	연관 리뷰(1:1)
accountId	Long	FK, NOT NULL	답글 작성자(사장 회원)
content	String	NOT NULL	답글 내용
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» Store (도메인 메서드)		
메서드명	반환 타입	설명
open()	void	상태 OPEN 변경
close()	void	상태 CLOSED 변경
temporaryClose()	void	상태 TEMP_CLOSED 변경
updateStoreInfo(dto)	void	상점 정보 수정 (이름, 주소, 전화번호 등)
updateRating(newRating, newCount)	void	평균 평점 재계산
increaseReviewCount()	void	리뷰 작성 시 호출
decreaseReviewCount()	void	리뷰 삭제 시 호출
isOwnedBy(accountId)	boolean	소유권 확인

«Entity» ProductCategory (도메인 메서드)		
메서드명	반환 타입	설명
activate()	void	활성화
deactivate()	void	비활성화 (Soft Delete)
updateDisplayOrder(int)	void	표시 순서 변경
isOwnedBy(storeId)	boolean	본인 매장 카테고리 검증

«Entity» Product (도메인 메서드)		
메서드명	반환 타입	설명
decreaseStock(qty)	void	재고 차감→ 0 이하 시 SOLD_OUT 처리
increaseStock(qty)	void	재고 복구(취소/환불 시)
isSoldOut()	boolean	품절 여부 반환
markAsSoldOut()	void	수동 품절 처리 (stock이 null일 때 사장이 직접)
markAsAvailable()	void	판매 재개
hasUnlimitedStock()	boolean	재고 무제한 여부 (stock == null)

«Entity» EventProduct (도메인 메서드)		
메서드명	반환 타입	설명
endEvent()	void	이벤트 종료— status=ENDED
isExpired()	boolean	현재 시각이 endDate 초과 여부 반환
getDiscountedPrice(originalPrice)	BigDecimal	할인 적용 가격 계산
isActive()	boolean	진행 중 여부 (현재 시간이 start~end 사이 + status=ACTIVE)
decreaseStock(qty)	void	재고 차감→ 0 이하 시 SOLD_OUT 처리
isSoldOut()	boolean	품절 여부 반환

- **Controller**

«Controller» StoreController			
MAP 방식	API명	반환타입	설명
GET	/stores	ResponseEntity<Page<StoreResponseDto>>	상점 목록 조회 (지역/카테고리/키워드 필터링)
GET	/stores/nearby	ResponseEntity<List<StoreMapResponseDto>>	주변 상점 조회 (지도 마커용, 위도/경도/반경)
GET	/stores/{storeId}	ResponseEntity<StoreDetailResponseDto>	상점 상세 정보 조회 (평점, 리뷰 수 포함)
GET	/stores/{storeId}/products	ResponseEntity<List<StoreProductResponseDto>>	상점 상품 목록 조회 (이벤트 상품 포함)
GET	/stores/{storeId}/product-categories	ResponseEntity<List<ProductCategoryResponseDto>>	매장 메뉴 카테고리 (활성만, 사용자 화면용)
GET	/stores/{storeId}/products/{productId}	ResponseEntity<StoreProductDetailResponseDto>	상품 상세 조회
GET	/stores/{storeId}/products/{productId}/options	ResponseEntity<List<StoreProductResponseDto>>	상품 상세 진입 시 옵션 트리 조회
GET	/stores/{storeId}/events	ResponseEntity<List<StoreEventProductResponseDto>>	진행 중인 이벤트 상품 목록
GET	/stores/{storeId}/notices	ResponseEntity<List<StoreNoticeResponseDto>>	매장 활성 공지 목록 (매장 상세 진입 시)

«Controller» OwnerStoreController			
MAP 방식	API명	반환타입	설명
POST	/owner/stores	ResponseEntity<StoreResponseDto>	상점 등록 (imageUrls 포함)
PATCH	/owner/stores/{storeId}	ResponseEntity<StoreResponseDto>	상점 정보 수정
GET	/owner/stores/{storeId}	ResponseEntity<StoreDashboardResponseDto>	상점 대시보드 조회 (요약 정보)
POST	/owner/stores/{storeId}/images	ResponseEntity<StoreImageResponseDto>	상점 이미지 추가
DELETE	/owner/stores/{storeId}/images/{imageId}	ResponseEntity<CommonResponseDto>	상점 이미지 삭제
PATCH	/owner/stores/{storeId}/images/{imageId}/thumbnail	ResponseEntity<CommonResponseDto>	상점 대표 이미지 지정
PATCH	/owner/stores/{storeId}/status	ResponseEntity<StoreResponseDto>	상점 상태 변경
DELETE	/owner/stores/{storeId}	ResponseEntity<CommonResponseDto>	상점 삭제
POST	/owner/stores/{storeId}/products	ResponseEntity<StoreProductResponseDto>	상품 등록 (imageUrls 포함)
PATCH	/owner/stores/{storeId}/products/{productId}	ResponseEntity<StoreProductResponseDto>	상품 정보 수정
POST	/owner/stores/{storeId}/products/{productId}/images	ResponseEntity<StoreProductImageResponseDto>	상품 이미지 추가
DELETE	/owner/stores/{storeId}/products/{productId}/images/{imageId}	ResponseEntity<CommonResponseDto>	상품 이미지 삭제
PATCH	/owner/stores/{storeId}/products/{productId}/images/{imageId}/thumbnail	ResponseEntity<CommonResponseDto>	상품 대표 이미지 지정
GET	/owner/stores/{storeId}/reviews	ResponseEntity<Page<OwnerStoreReviewResponseDto>>	상점 리뷰 전체 조회
GET	/owner/stores/{storeId}/reviews/{reviewId}	ResponseEntity<OwnerStoreReviewDetailResponseDto>	상점 리뷰 상세
PATCH	/owner/stores/{storeId}/products/{productId}/sold-out	ResponseEntity<StoreProductResponseDto>	수동 품절 처리 (stock null인 경우)
PATCH	/owner/stores/{storeId}/products/{productId}/available	ResponseEntity<StoreProductResponseDto>	판매 재개
DELETE	/owner/stores/{storeId}/products/{productId}	ResponseEntity<CommonResponseDto>	상품 삭제
POST	/owner/stores/{storeId}/events	ResponseEntity<StoreEventProductResponseDto>	이벤트 상품 등록
PATCH	/owner/stores/{storeId}/events/{eventId}	ResponseEntity<StoreEventProductResponseDto>	이벤트 상품 수정
DELETE	/owner/stores/{storeId}/events/{eventId}	ResponseEntity<CommonResponseDto>	이벤트 상품 삭제

GET	/owner/stores/{storeId}/orders	ResponseEntity<Page<StoreOrderResponseDto>>	주문/예약 목록 조회 (필터링: 기간, 상태)
GET	/owner/stores/{storeId}/orders/{orderId}	ResponseEntity<StoreOrderResponseDto>	주문/예약 상세 조회

«Controller» OwnerStoreNoticeController			
MAP 방식	API명	반환타입	설명
GET	/owner/notices	ResponseEntity<List<StoreNoticeResponseDto>>	내 매장 공지 이력
POST	/owner/notices	ResponseEntity<StoreNoticeResponseDto>	공지 등록
PATCH	/owner/notices/{noticeId}	ResponseEntity<StoreNoticeResponseDto>	공지 수정
DELETE	/owner/notices/{noticeId}	ResponseEntity<CommonResponseDto>	공지 삭제

«Controller» OwnerProductOptionController			
MAP 방식	API명	반환타입	설명
GET	/owner/products/{productId}/options	ResponseEntity<List<ProductOptionDto>>	상품 옵션 목록 조회
POST	/owner/products/{productId}/options	ResponseEntity<ProductOptionDto>	옵션 그룹 생성 (선택지 포함)
PUT	/owner/products/{productId}/options/{optionId}	ResponseEntity<ProductOptionDto>	옵션 그룹 전체 교체
PATCH	/owner/products/{productId}/options/{optionId}	ResponseEntity<ProductOptionDto>	옵션 그룹 부분 수정
DELETE	/owner/products/{productId}/options/{optionId}	ResponseEntity<CommonResponseDto>	옵션 그룹 삭제
PATCH	/owner/products/{productId}/options/items/{itemId}/availability	ResponseEntity<CommonResponseDto>	선택지 품질 토글

«Controller» OwnerProductCategoryController			
MAP 방식	API명	반환타입	설명
GET	/owner/stores/{storeId}/product-categories	ResponseEntity<List<ProductCategoryResponseDto>>	본인 매장 카테고리 목록
POST	/owner/stores/{storeId}/product-categories	ResponseEntity<ProductCategoryResponseDto>	카테고리 생성
PATCH	/owner/product-categories/{categoryId}	ResponseEntity<ProductCategoryResponseDto>	수정 (이름 / 표시순서)
DELETE	/owner/product-categories/{categoryId}	ResponseEntity<CommonResponseDto>	삭제 (소속 상품 검증)
DELETE	/owner/product-categories/{categoryId}/hard	ResponseEntity<CommonResponseDto>	물리 삭제 (소속 상품 0개일 때만)
PATCH	/owner/product-categories/{categoryId}/activate	ResponseEntity<CommonResponseDto>	활성화
PATCH	/owner/product-categories/{categoryId}/deactivate	ResponseEntity<CommonResponseDto>	비활성화

«Controller» StoreReviewController			
MAP 방식	API명	반환타입	설명
GET	/stores/{storeId}/reviews	ResponseEntity<Page<StoreReviewResponseDto>>	상점 리뷰 목록 조회
GET	/stores/{storeId}/reviews/{reviewId}	ResponseEntity<StoreReviewDetailResponseDto>	리뷰 상세 조회
POST	/stores/{storeId}/reviews	ResponseEntity<StoreReviewResponseDto>	리뷰 작성 (거래 완료 orderId 필요, imageUrls 포함)
PATCH	/stores/{storeId}/reviews/{reviewId}	ResponseEntity<StoreReviewResponseDto>	리뷰 수정 (본인만)
DELETE	/stores/{storeId}/reviews/{reviewId}	ResponseEntity<CommonResponseDto>	리뷰 삭제 (본인만)
POST	/stores/{storeId}/reviews/{reviewId}/images	ResponseEntity<StoreImageResponseDto>	리뷰 이미지 추가
DELETE	/stores/{storeId}/reviews/{reviewId}/images/{imageId}	ResponseEntity<CommonResponseDto>	리뷰 이미지 삭제
PATCH	/stores/{storeId}/reviews/{reviewId}/images/{imageId}/thumbnail	ResponseEntity<CommonResponseDto>	대표 이미지 지정
POST	/stores/{storeId}/reviews/{reviewId}/reply	ResponseEntity<StoreReviewReplyResponseDto>	리뷰 답글 작성 (상점 사장만)
PATCH	/stores/{storeId}/reviews/{reviewId}/reply	ResponseEntity<StoreReviewReplyResponseDto>	리뷰 답글 수정 (상점 사장만)
DELETE	/stores/{storeId}/reviews/{reviewId}/reply	ResponseEntity<CommonResponseDto>	리뷰 답글 삭제 (상점 사장만)

«Controller» AdminStoreController			
MAP 방식	API명	반환타입	설명
GET	/admin/stores	ResponseEntity<Page<StoreResponseDto>>	관리자 - 전체 상점 조회
PATCH	/admin/stores/{storeId}/suspend	ResponseEntity<CommonResponseDto>	상점 정지 (신고 처리 시)
PATCH	/admin/stores/{storeId}/activate	ResponseEntity<CommonResponseDto>	상점 정지 해제

- Service

«Service» StoreService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getStoreList(storeSearchDto)	Page<StoreResponseDto>	readOnly	QueryDSL	상점 목록 조회 (지역/카테고리/키워드/평점/정렬)
getNearbyStores(latitude, longitude, radius)	List<StoreMapResponseDto>	readOnly	-	주변 상점 조회 (지도용)
getStoreDetail(storeId)	StoreDetailResponseDto	readOnly	CheckStoreAccess	상점 상세 정보 조회 (평점/리뷰 수 포함)
getProductList(storeId)	List<ProductResponseDto>	readOnly	-	상점의 상품 목록 (이벤트 포함)
getActiveProductCategoriesByStore(storeId)	List<ProductCategoryResponseDto>	readOnly	-	사용자 화면용 활성 카테고리 (displayOrder 정렬)
getProductDetail(productId)	ProductDetailResponseDto	readOnly	CheckStoreAccess	상품 상세 조회 (조회수 증가)
getActiveEvents(storeId)	List<EventProductResponseDto>	readOnly	-	진행 중인 이벤트 상품 목록

«Service» OwnerStoreService

메서드명	반환타입	트랜잭션	정책 / AOP	설명
registerStore(accountId, StoreCreateRequestDto)	StoreResponseDto	REQUIRED	Ownership	상점 등록 (1인 1상점 검증 후 저장)
updateStore(accountId, storeId, StoreUpdateRequestDto)	StoreResponseDto	REQUIRED	Ownership	상점 정보 수정
updateStoreStatus(accountId, storeId, status)	StoreResponseDto	REQUIRED	Ownership	상점 상태 변경
deleteStore(accountId, storeId)	void	REQUIRED	Ownership	상점 delete
getProductCategoriesByStore(storeId)	List<ProductCategoryResponseDto>	readOnly	Ownership	본인 매장 카테고리 목록
createProductCategory(storeId, ProductCategoryCreateRequestDto)	ProductCategoryResponseDto	REQUIRED	Ownership	카테고리 생성 (이름 중복 검증)
updateProductCategory(categoryId, ProductCategoryUpdateRequestDto)	ProductCategoryResponseDto	REQUIRED	Ownership	수정
deactivateProductCategory(categoryId)	void	REQUIRED	Ownership	Soft Delete (기본 삭제 = isActive=false)
hardDeleteProductCategory(categoryId)	void	REQUIRED	Ownership	물리 삭제 (소속 Product 0개 검증)
activateProductCategory(categoryId)	void	REQUIRED	Ownership	활성화
deactivateProductCategory(categoryId)	void	REQUIRED	Ownership	비활성화
addStoreImage(accountId, storeId, imageUrl)	StoreImageResponseDto	REQUIRED	ImageService	상점 이미지 추가
deleteStoreImage(accountId, storeId, imageId)	void	REQUIRED	ImageService	상점 이미지 삭제
setStoreImageThumbnail(accountId, storeId, imageId)	void	REQUIRED	ImageService	상점 대표 이미지 지정

«Service» StoreNoticeService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
createNotice(Long storeId, CreateNoticeDto dto)	StoreNoticeDto	REQUIRED	Ownership	공지 등록 (사장)
updateNotice(Long storeId, Long noticeId, UpdateNoticeDto dto)	StoreNoticeDto	REQUIRED	Ownership	공지 수정
deleteNotice(Long storeId, Long noticeId)	void	REQUIRED	Ownership	공지 삭제
getNoticeHistory(Long storeId, Pageable pageable)	Page<StoreNoticeDto>	readOnly	Ownership	공지 이력 (사장 페이지)
getActiveNotices(Long storeId)	List<StoreNoticeDto>	readOnly	-	활성 공지 (사용자 매장 상세)
getLatestActiveNotice(Long storeId)	Optional<StoreNoticeDto>	readOnly	-	최신 공지

«Service» ProductService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
registerProduct(accountId, storeId, ProductCreateRequestDto)	ProductResponseDto	REQUIRED	Ownership	상품 등록
updateProduct(accountId, storeId, productId, ProductUpdateRequestDto)	ProductResponseDto	REQUIRED	Ownership + OptimisticLock	상품 수정
deleteProduct(accountId, storeId, productId)	void	REQUIRED	Ownership	상품 삭제
markProductAsSoldOut(accountId, storeId, productId)	ProductResponseDto	REQUIRED	-	수동 품절 처리
markProductAsAvailable(accountId, storeId, productId)	ProductResponseDto	REQUIRED	-	판매 재개
addProductImage(accountId, storeId, productId, imageUrl)	ProductImageResponseDto	REQUIRED	ImageService	상품 이미지 추가
deleteProductImage(accountId, storeId, productId, imageId)	void	REQUIRED	ImageService	상품 이미지 삭제
setProductImageThumbnail(accountId, storeId, productId, imageId)	void	REQUIRED	ImageService	상품 대표 이미지 지정

«Service» ProductOptionService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getProductOptions(Long productId)	List<ProductOptionDto>	READ ONLY	-	상품 옵션 트리 조회 (그룹 + 선택지)
createOption(Long storeId, Long productId, CreateOptionDto dto)	ProductOptionDto	REQUIRED	OWNER	옵션 그룹 + 선택지 일괄 생성
updateOption(Long storeId, Long productId, UpdateOptionDto dto)	ProductOptionDto	REQUIRED	OWNER	옵션 그룹 + 선택지 일괄 수정
deleteOption(Long storeId, Long productId)	void	REQUIRED	OWNER	옵션 그룹 삭제 (CASCADE)
toggleItemAvailability(Long storeId, Long itemId)	void	REQUIRED	OWNER	선택지 품절 토글
validateAndCalculatePrice(Long productId, List<Long> selectedItemIds)	OptionValidationResult	READ ONLY	OptionValidation	주문/장바구니 시 검증 + 가격 계산
buildOptionsSnapshot(List<Long> selectedItemIds)	String	READ ONLY	Snapshot	OrderItem 스냅샷 JSON 생성

«Service» ProductService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
registerProduct(accountId, storeId, ProductCreateRequestDto)	ProductResponseDto	REQUIRED	Ownership	상품 등록
updateProduct(accountId, storeId, productId, ProductUpdateRequestDto)	ProductResponseDto	REQUIRED	Ownership + OptimisticLock	상품 수정
deleteProduct(accountId, storeId, productId)	void	REQUIRED	Ownership	상품 삭제
markProductAsSoldOut(accountId, storeId, productId)	ProductResponseDto	REQUIRED	-	수동 품절 처리
markProductAsAvailable(accountId, storeId, productId)	ProductResponseDto	REQUIRED	-	판매 재개
addProductImage(accountId, storeId, productId, imageUrl)	ProductImageResponseDto	REQUIRED	ImageService	상품 이미지 추가
deleteProductImage(accountId, storeId, productId, imageId)	void	REQUIRED	ImageService	상품 이미지 삭제
setProductImageThumbnail(accountId, storeId, productId, imageId)	void	REQUIRED	ImageService	상품 대표 이미지 지정

«Service» EventProductService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
registerEvent(accountId, storeId, EventCreateRequestDto)	EventProductResponseDto	REQUIRED	PESSIMISTIC_WRITE + UNIQUE	동일 상품 ACTIVE 이벤트 1개 보장
updateEvent(accountId, storeId, eventId, EventUpdateRequestDto)	EventProductResponseDto	REQUIRED	-	종료 이벤트 수정 불가
deleteEvent(accountId, storeId, eventId)	void	REQUIRED	-	이벤트 삭제
autoEndExpiredEvents()	void	REQUIRED	Scheduler	만료 이벤트 자동 종료

«Service» **StoreDashboardService**

메서드명	반환타입	트랜잭션	정책 / AOP	설명
getDashboardSummary(accountId)	StoreDashboardResponseDto	readOnly	Cacheable	매출/주문/리뷰 통합 조회
getSalesStatistics(accountId, StatisticsPeriod)	SalesStatisticsResponseDto	readOnly	-	기간별 매출 통계
getProductStatistics(accountId)	ProductStatisticsResponseDto	readOnly	-	인기상품/재고 분석
getStoreOrders(accountId, OrderSearchDto, Pageable)	Page<StoreOrderResponseDto>	readOnly	-	주문 목록 조회
getStoreOrderDetail(accountId, orderId)	StoreOrderDetailResponseDto	readOnly	-	주문 상세 조회
getReviewStatistics(accountId)	ReviewStatisticsResponseDto	readOnly	-	평점/리뷰 분석

«Service» StoreReviewService

메서드명	반환타입	트랜잭션	정책 / AOP	설명
getReviewList(storeId, Pageable)	Page<StoreReviewResponseDto>	readOnly	-	삭제 제외 리뷰 조회
getMyStoreReviews(accountId, Pageable)	Page<StoreReviewResponseDto>	readOnly	-	내가 작성한 상점 리뷰
getReviewDetail(reviewId)	StoreReviewDetailResponseDto	readOnly	-	리뷰 상세 조회
registerReview(accountId, storeId, StoreReviewCreateRequestDto)	StoreReviewResponseDto	REQUIRED	UNIQUE(orderId) + Event	리뷰 등록
updateReview(accountId, reviewId, StoreReviewUpdateRequestDto)	StoreReviewResponseDto	REQUIRED	Ownership + Event	리뷰 수정
deleteReview(accountId, reviewId)	void	REQUIRED	Ownership + Event	리뷰 delete
addReviewImage(accountId, reviewId, imageUrl)	StoreReviewImageResponseDto	REQUIRED	ImageService	리뷰 이미지 추가
deleteReviewImage(accountId, reviewId, imageId)	void	REQUIRED	ImageService	리뷰 이미지 삭제
setReviewImageThumbnail(accountId, reviewId, imageId)	void	REQUIRED	ImageService	대표 이미지 설정

«Service» StoreReplyService

메서드명	반환타입	트랜잭션	정책 / AOP	설명
registerReply(accountId, reviewId, StoreReviewReplyCreateRequestDto)	StoreReviewReplyResponseDto	REQUIRED	OwnerOnly	리뷰 답글 등록
updateReply(accountId, reviewId, StoreReviewReplyUpdateRequestDto)	StoreReviewDetailResponseDto	REQUIRED	OwnerOnly	답글 수정
deleteReply(accountId, reviewId)	CommonResponseDto	REQUIRED	OwnerOnly	답글 삭제

«Service» **AdminStoreService**

메서드명	반환타입	트랜잭션	정책 / AOP	설명
getAllStores(Pageable, filter)	Page<StoreResponseDto>	readOnly	Admin	전체 상점 조회
suspendStore(storeId)	void	REQUIRED	Admin	상점 정지
forceDeleteReview(reviewId)	void	REQUIRED	Admin	리뷰 영구 삭제

«Helper» **StoreAccessHelper**

메서드명	반환타입	설명
verifyStoreOwnership(accountId, storeId)	Store	소유권 검증 → 실패 시 ForbiddenException
verifyProductOwnership(accountId, storeId, productId)	Product	상품 소유권 검증
verifyActiveStore(storeId)	Store	상점 접근 가능 여부 (CLOSED 차단)
verifyProductCategoryOwnership(accountId, productcategoryId)	ProductCategory	본인 매장 카테고리 검증

- Repository

«Repository» StoreRepository		
메서드명	반환타입	설명
findById(Long storeId)	Optional<Store>	상점 ID로 조회
findByAccountId(Long accountId)	Optional<Store>	사장 ID로 상점 조회
existsByAccountId(Long accountId)	boolean	사장의 상점 중복 등록 방지
incrementFavoriteCount(Long storeId)	int	찜 카운트 +1 (DB 원자 UPDATE, 영향받은 행 수 반환)
decrementFavoriteCount(Long storeId)	int	찜 카운트 -1 (favoriteCount > 0 가드, 영향받은 행 수 반환)
findNearbyStores(Double latitude, Double longitude, Double radius)	List<Store>	위치 기반 주변 상점 조회 (지도용)
findAllByStatus(StoreStatus status, Pageable pageable)	Page<Store>	상태별 상점 목록 (관리자용)

«Repository» StoreRepositoryCustom (QueryDSL)		
메서드명	반환타입	설명
searchStores(StoreSearchDto condition, Pageable pageable)	Page<Store>	지역+카테고리+키워드+정렬 복합 검색
findNearbyStoresWithFilter(NearbySearchCondition condition)	List<Store>	위치 + 필터 복합 검색

«Repository» StoreNoticeRepository

메서드명	반환타입	설명
findById(Long noticeId)	Optional<StoreNotice>	공지 조회
findByIdAndStoreId(Long noticeId, Long storeId)	Optional<StoreNotice>	소유권 검증
findAllByStoreId(Long storeId)	List<StoreNotice>	매장 공지 이력 (사장용)
findAllByStoreIdAndIsActiveTrue(Long storeId)	Page<StoreNotice>	매장 활성 공지 (사용자 화면 노출용)
findFirstByStoreIdAndIsActiveTrueOrderByCreatedAtDesc(Long storeId)	Optional<StoreNotice>	최신 활성 공지 1개 (목록 화면 배치용)
deleteAllByStoreId(Long storeId)	void	매장 삭제 시 CASCADE

«Repository» StoreImageRepository

메서드명	반환타입	설명
findAllByStoreIdOrderByDisplayOrder(Long storeId)	List<StoreImage>	상점 이미지 목록 (순서대로)
findByStoreIdAndIsThumbnailTrue(Long storeId)	Optional<StoreImage>	대표 이미지 조회
findByIdAndStoreId(Long storeImageId, Long storeId)	Optional<StoreImage>	소유권 확인용 조회
countByStoreId(Long storeId)	Long	상점 이미지 개수 (제한 검증용)
findFirstByStoreIdOrderByDisplayOrderAsc(Long storeId)	Optional<StoreImage>	대표 자동 승격용(대표 삭제 시)
deleteAllByStoreId(Long storeId)	void	상점 이미지 전체 삭제 (상점 삭제 시)

«Repository» **ProductCategoryRepository**

메서드명	반환타입	설명
findById(Long categoryId)	Optional<ProductCategory>	카테고리 조회
findAllByStoreId(Long storeId)	List<ProductCategory>	매장별 전체 (사장용, displayOrder 정렬)
findAllByStoreIdAndIsActiveTrue(Long storeId)	List<ProductCategory>	매장별 활성 카테고리 (사용자용)
existsByStoreIdAndName(Long storeId, String name)	boolean	중복 이름 검증
countByStoreId(Long storeId)	Long	매장별 카테고리 수
deleteAllByStoreId(Long storeId)	void	매장 삭제 시 CASCADE

«Repository» ProductRepository		
메서드명	반환타입	설명
findById(Long productId)	Optional<Product>	상품 ID로 조회
findByIdAndStoreId(Long productId, Long storeId)	Optional<Product>	상점-상품 소유권 확인용
findByIdForUpdate(productId)	Optional<Product>	재고 차감용 락
findAllByStoreId(Long storeId, Pageable pageable)	Page<Product>	상점의 상품 목록
findAllByStoreIdAndStatus(Long storeId, ProductStatus status, Pageable pageable)	Page<Product>	상점 상품 목록 (상태별, 페이징)
findAllByStoreIdAndProductCategoryId(Long storeId, Long productCategoryId)	List<Product>	상점 내 카테고리별 상품
countByStoreIdAndStatusNot(Long storeId, ProductStatus status)	Long	삭제되지 않은 상품 수 (대시보드용)
countByProductCategoryId(Long productCategoryId)	Long	ProductCategory Hard Delete 검증용(소속 Product 0개 확인)

«Repository» **ProductImageRepository**

메서드명	반환타입	설명
findAllByProductIdOrderByDisplayOrder(Long productId)	List<ProductImage>	상품 이미지 목록 (순서대로)
findByProductIdAndIsThumbnailTrue(Long productId)	Optional<ProductImage>	대표 이미지 조회
findByIdAndProductId(Long productImageId, Long productId)	Optional<ProductImage>	소유권 확인용 조회
countByProductId(Long productId)	Long	상품 이미지 개수 (제한 검증용)
findFirstByProductIdOrderByDisplayOrderAsc(Long productId)	Optional<ProductImage>	대표 자동 승격용
deleteAllByProductId(Long productId)	void	상품 이미지 전체 삭제 (상품 삭제 시)

«Repository» **ProductOptionRepository**

메서드명	반환타입	설명
findById(Long productOptionId)	Optional<ProductOption>	옵션 그룹 조회
findAllByProductIdOrderByDisplayOrder(Long productId)	List<ProductOption>	상품의 옵션 그룹 목록
existsByProductIdAndGroupName(Long productId, String groupName)	boolean	중복 그룹명 검증
countByProductId(Long productId)	Long	옵션 그룹 수
deleteAllByProductId(Long productId)	void	상품 삭제 시 CASCADE

«Repository» ProductOptionItemRepository		
메서드명	반환타입	설명
findById(Long productOptionItemId)	Optional<ProductOptionItem>	선택지 조회
findAllByProductOptionIdOrderByDisplayOrder(Long productOptionId)	List<ProductOptionItem>	그룹의 선택지 목록
findAllByIdIn(List<Long> itemIds)	List<ProductOptionItem>	여러 선택지 일괄 조회 (주문 시 검증용)
findAllByProductOptionId_ProductId(Long productId)	List<ProductOptionItem>	상품의 모든 선택지 (한 번에 로딩)
existsByProductOptionIdAndItemName(Long productOptionId, String itemName)	boolean	중복 이름 검증
countByProductOptionId(Long productOptionId)	Long	선택지 개수
deleteAllByProductOptionId(Long productOptionId)	void	그룹 삭제 시 CASCADE

«Repository» EventProductRepository		
메서드명	반환타입	설명
findById(Long eventProductId)	Optional<EventProduct>	이벤트 상품 ID로 조회
findActiveByProductId(Long productId)	Optional<EventProduct>	특정 상품의 진행 중(ACTIVE) 이벤트 조회
findActiveByProductIdForUpdate(Long productId)	Optional<EventProduct>	@Lock(PESSIMISTIC_WRITE) / 이벤트 등록 시 동시성 제어
existsByActiveProductId(Long productId)	boolean	ACTIVE 이벤트 존재 여부 확인 (동시성 제어 + 중복 생성 방지)
findAllByStatusAndEndDateBefore(EventStatus status, LocalDateTime now)	List<EventProduct>	만료 이벤트 자동 종료용 (스케줄러)
bulkUpdateExpiredEvents()	int	만료 이벤트 일괄 종료 처리 (@Modifying 스케줄러)

«Repository» StoreReviewRepository		
메서드명	반환타입	설명
findById(Long storeReviewId)	Optional<StoreReview>	리뷰 ID로 조회
findAllByStoreId(Long storeId, Pageable pageable)	Page<StoreReview>	상점 리뷰 목록 (페이징)
findAllByAccountId(Long accountId, Pageable pageable)	Page<StoreReview>	내가 작성한 리뷰 목록 (마이페이지용)
existsByOrderId(Long orderId)	boolean	주문에 대한 리뷰 존재 여부 (중복 작성 방지)
findByOrderId(Long orderId)	Optional<StoreReview>	주문 ID로 리뷰 조회
findAverageRatingByStoreId(Long storeId)	Double	상점 평균 평점 계산 (갱신용)
countByStoreId(Long storeId)	Long	상점 리뷰 수 (캐싱용)

«Repository» StoreReviewImageRepository		
메서드명	반환타입	설명
findAllByStoreReviewIdOrderByDisplayOrder(Long storeReviewId)	List<StoreReviewImage>	리뷰 이미지 목록
findByStoreReviewIdAndIsThumbnailTrue(Long storeReviewId)	Optional<StoreReviewImage>	대표 이미지 조회
findByIdAndStoreReviewId(Long imageId, Long reviewId)	Optional<StoreReviewImage>	소유권 확인용
countByStoreReviewId(Long storeReviewId)	Long	리뷰 이미지 개수
findFirstByStoreReviewIdOrderByDisplayOrderAsc(Long storeReviewId)	Optional<ProductImage>	대표 자동 승격용
deleteAllByStoreReviewId(Long storeReviewId)	void	리뷰 이미지 전체 삭제

«Repository» StoreReviewReplyRepository		
메서드명	반환타입	설명
findByStoreReviewId(Long storeReviewId)	Optional<StoreReviewReply>	리뷰의 답글 조회
existsByStoreReviewId(Long storeReviewId)	boolean	답글 존재 여부 (중복 작성 방지)
deleteByStoreReviewId(Long storeReviewId)	void	리뷰 삭제 시 답글도 함께 삭제

5.3 Order/Payment 도메인

- Entity

«Entity» Order			
속성명	타입	제약조건	설명
orderId	Long	PK, NOT NULL	고유 식별자
accountId	Long	FK, NOT NULL	구매자
storeId	Long	FK, NOT NULL	주문 상점
totalPrice	BigDecimal	NOT NULL	총 결제 금액
orderNumber	String	UNIQUE, NOT NULL	사용자 표시용 주문번호
version	Long	NOT NULL	낙관적 락용 (@Version)
status	OrderStatus	NOT NULL	PENDING / PAID / FAILED / CANCELLED / REFUNDED
paidAt	LocalDateTime	nullable	결제 완료 시점
cancelledAt	LocalDateTime	nullable	취소 시점
cancelReason	String	nullable	취소 사유
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» OrderItem			
속성명	타입	제약조건	설명
orderid	Long	PK, NOT NULL	고유 식별자
productId	Long	FK, NOT NULL	연관 주문
eventproductId	Long	FK, nullable	연관 상품
productName	String	NOT NULL	주문 시점 상품명
selectedOptions	String	NOT NULL	주문 시 선택된 옵션 스냅샷(JSON)
quantity	int	NOT NULL	수량
price	BigDecimal	NOT NULL	주문 시점 가격 고정

«Entity» Payment			
속성명	타입	제약조건	설명
paymentId	Long	PK, NOT NULL	고유 식별자
orderId	Long	FK, NOT NULL	연관 주문(1:1)
accountId	Long	FK, NOT NULL	결제자
portOnePaymentId	String	UNIQUE, NOT NULL	PortOne 결제 고유 번호
idempotencyKey	String	UNIQUE, NOT NULL	중복 결제 방지 (멱등성 키)
amount	BigDecimal	NOT NULL	결제 금액
status	PaymentStatus	NOT NULL	PENDING / PAID / FAILED / CANCELLED / REFUNDED
pgProvider	String	nullable	PG사 이름
paymentMethod	PaymentMethod (ENUM)	NOT NULL	CARD / KAKAOPAY / NAVERPAY / TOSS 등
paidAt	LocalDateTime	nullable	결제 완료 시점
cancelledAt	LocalDateTime	nullable	취소 시점
failReason	String	nullable	결제 실패 사유 (PortOne 응답)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» Reservation			
속성명	타입	제약조건	설명
reservationId	Long	PK, NOT NULL	고유 식별자
accountId	Long	FK, NOT NULL	예약자
storeId	Long	FK, NOT NULL	예약 상점
productId	Long	FK, nullable	예약 상품(상품 예약 시)
orderId	Long	FK, nullable	연관 주문
reservedAt	LocalDateTime	NOT NULL	예약 일시
status	ReservationStatus	NOT NULL	PENDING / CONFIRMED / EXPIRED / CANCELLED
reservationType	ReservationType	NOT NULL	VISIT / PRODUCT
totalPrice	BigDecimal	nullable	PRODUCT 타입일 때만 값 존재, VISIT 타입은 결제 없으므로 null
capacity	int	nullable	예약 최대 가능 인원 (VISIT 타입에서만 사용)
version	Long	NOT NULL	낙관적 락
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» Cart			
속성명	타입	제약조건	설명
cartId	Long	PK, NOT NULL	고유 식별자
accountId	Long	FK, NOT NULL, UNIQUE	회원
storeId	Long	FK, NOT NULL, UNIQUE	동일 상점 제약 관리
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

UNIQUE 제약: (cartId, productId, eventproductId, selectedOptionsHash)
CHECK 제약: productId IS NOT NULL XOR eventproductId IS NOT NULL

«Entity» CartItem			
속성명	타입	제약조건	설명
cartItemId	Long	PK, NOT NULL	고유 식별자
cartId	Long	FK, NOT NULL	연관 장바구니
productId	Long	FK, nullable	연관 상품 UNIQUE 제약: (cartId, productId), (cartId, eventproductId)
eventproductId	Long	FK, nullable	연관 이벤트 상품 UNIQUE 제약: (cartId, productId), (cartId, eventproductId)
selectedOptionItemIds	String	nullable	선택한 옵션 ID 목록(JSON)
optionsTotalPrice	BigDecimal	NOT NULL	옵션 추가 가격 총합
selectedOptionsHash	String	NOT NULL	옵션 조합 구분용 해시값
quantity	int	NOT NULL, ≥1	수량
price	BigDecimal	NOT NULL	추가할 당시 가격 고정

«Entity» Order (도메인 메서드)		
메서드명	타입	제약조건
markPaid()	void	status = PAID
cancel()	void	status = CANCELLED
refund()	void	status = REFUNDED
isCancelable()	boolean	취소 가능 상태 반환

«Entity» Payment (도메인 메서드)		
메서드명	타입	제약조건
markPaid()	void	status = PAID
markFailed()	void	status = FAILED
cancel()	void	status = CANCELLED
refund()	void	status = REFUNDED
isAlreadyProcessed()	boolean	이미 처리된 웹훅 여부 반환

- **Controller**

«Controller» OrderController			
MAP 방식	API명	반환타입	설명
POST	/orders	ResponseEntity<OrderResponseDto>	주문 생성 (장바구니 → 주문)
GET	/orders/me	ResponseEntity<Page<OrderResponseDto>>	내 주문 목록 (필터: 기간, 상태)
GET	/orders/me/{orderId}	ResponseEntity<OrderDetailResponseDto>	내 주문 상세
PATCH	/orders/me/{orderId}/cancel	ResponseEntity<CommonResponseDto>	주문 취소 (PENDING 상태만)
POST	/orders/me/{orderId}/refund	ResponseEntity<CommonResponseDto>	환불 요청 (PAID 상태만)

«Controller» PaymentController			
MAP 방식	API명	반환타입	설명
POST	/payments	ResponseEntity<PaymentResponseDto>	결제 요청
POST	/payments/webhook	ResponseEntity<CommonResponseDto>	PortOne 웹훅 수신(PortOne webhook 서명 검증, 위변조 요청 차단)
GET	/payments/me	ResponseEntity<Page<PaymentResponseDto>>	내 결제 내역
GET	/payments/me/{paymentId}	ResponseEntity<PaymentResponseDto>	결제 상세 조회
PATCH	/payments/{paymentId}/cancel	ResponseEntity<CommonResponseDto>	결제 취소
POST	/payments/{paymentId}/refund	ResponseEntity<CommonResponseDto>	환불 요청

«Controller» ReservationController			
MAP 방식	API명	반환타입	설명
POST	/reservations/product	ResponseEntity<ReservationResponseDto>	상품 예약 생성
POST	/reservations/store	ResponseEntity<ReservationResponseDto>	상점 방문 예약 생성
GET	/reservations/me	ResponseEntity<Page<ReservationResponseDto>>	내 예약 목록 (필터: 기간, 상태, 타입)
GET	/reservations/me/{reservationId}	ResponseEntity<ReservationResponseDto>	내 예약 상세
PATCH	/reservations/me/{reservationId}/cancel	ResponseEntity<CommonResponseDto>	예약 취소

«Controller» CartController			
MAP 방식	API명	반환타입	설명
GET	/cart	ResponseEntity<CartResponseDto>	장바구니 조회 (목록 + 총 금액)
GET	/cart/count	ResponseEntity<CartCountResponseDto>	장바구니 상품 수
POST	/cart/items	ResponseEntity<CartItemResponseDto>	장바구니 상품 추가
PATCH	/cart/items/{cartItemId}	ResponseEntity<CartItemResponseDto>	장바구니 수량 변경
DELETE	/cart/items/{cartItemId}	ResponseEntity<CommonResponseDto>	장바구니 상품 삭제
DELETE	/cart	ResponseEntity<CommonResponseDto>	장바구니 전체 비우기

«Controller» **OwnerOrderController**

MAP 방식	API명	반환타입	설명
GET	/stores/me/{storeId}/orders	ResponseEntity<Page<StoreOrderResponseDto>>	사장용 주문 목록 (필터: 기간, 상태)
GET	/stores/me/{storeId}/orders/{orderId}	ResponseEntity<StoreOrderDetailResponseDto>	사장용 주문 상세
PATCH	/stores/me/{storeId}/orders/{orderId}/confirm	ResponseEntity<CommonResponseDto>	주문 확정
PATCH	/stores/me/{storeId}/orders/{orderId}/refund/approve	ResponseEntity<CommonResponseDto>	환불 요청 승인
PATCH	/stores/me/{storeId}/orders/{orderId}/refund/reject	ResponseEntity<CommonResponseDto>	환불 요청 거절

«Controller» **OwnerReservationController**

MAP 방식	API명	반환타입	설명
GET	/owner/stores/me/{storeId}/reservations	ResponseEntity<Page<StoreReservationResponseDto>>	사장용 예약 목록
GET	/owner/stores/me/{storeId}/reservations/{reservationId}	ResponseEntity<StoreReservationDetailResponseDto>	사장용 예약 상세
PATCH	/owner/stores/me/{storeId}/reservations/{reservationId}/confirm	ResponseEntity<CommonResponseDto>	예약 확정
PATCH	/owner/stores/{storeId}/reservations/{reservationId}/reject	ResponseEntity<CommonResponseDto>	예약 거절

- Service

«Service» OrderService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
createOrder(accountId, OrderCreateRequestDto)	OrderResponseDto	REQUIRED	RedisLock(productId) + StockCheck + Event	장바구니 → 주문 생성 (재고 차감 + Order/OrderItem 저장 + orderNumber 발급)
getMyOrders(accountId, OrderSearchDto, Pageable)	Page<MyOrderResponseDto>	readOnly	-	내 동네상점 주문/예약 내역 (필터: 기간, 상태)
getMyOrderDetail(accountId, orderId)	MyOrderDetailResponseDto	readOnly	Ownership	내동네 상점 주문 상세 (OrderItem + Payment 정보 포함)
cancelOrder(accountId, orderId, reason)	void	REQUIRED	Ownership + Cancellable + Event	주문 취소 (PENDING/PAID 상태) → 재고 복구 + 결제 취소 이벤트 발행
refundOrder(accountId, orderId, reason)	void	REQUIRED	Ownership + Refundable + Event	환불 요청 → 환불 이벤트 발행 (PaymentService가 처리)
markAsPaid(orderId)	void	REQUIRED	Internal + OptimisticLock	Payment Webhook에서 호출 → status=PAID + paidAt 기록

«Service» **PaymentService**

메서드명	반환타입	트랜잭션	정책 / AOP	설명
requestPayment(accountId, PaymentRequestDto)	PaymentResponseDto	REQUIRED	Idempotent + RedisLock(orderId)	먹등성 키 검증 → Redis 락 → 재고 확인 → PortOne 결제 요청 → Payment 생성
handleWebhook(WebhookRequestDto)	void	REQUIRED	Idempotent + Event	중복 웹훅 확인 (portOnePaymentId) → 금액 검증 → Order 상태 변경 → 재고 차감 확정
getMyPayments(accountId, Pageable)	Page<PaymentResponseDto>	readOnly	-	내 결제 내역
getMyPaymentDetail(accountId, paymentId)	PaymentResponseDto	readOnly	Ownership	내 결제 상세
cancelPayment(accountId, paymentId)	void	REQUIRED	Ownership + RedisLock + Event	Redis 락 → PortOne 취소 호출 → status=CANCELLED → Order 상태 동기화
refundPayment(accountId, paymentId, reason)	void	REQUIRED	Ownership + Event	PortOne 환불 호출 → status=REFUNDED → 재고 복구
verifyPayment(paymentId, amount)	boolean	readOnly	-	PortOne 결제 검증 (금액 일치 확인)

«Service» ReservationService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
createProductReservation(accountId, ReservationCreateRequestDto)	ReservationResponseDto	REQUIRED	StockCheck + Event	상품 예약 생성 (재고 확인 → 예약 생성 → status=PENDING)
createStoreReservation(accountId, ReservationCreateRequestDto)	ReservationResponseDto	REQUIRED	RedisLock ("reservation:{storeId}: {reservedAt:yyyy-MM-dd-HH-mm}") + CapacityCheck + Event	매장 방문 예약 생성 (시간 중복 확인 + capacity 검증 → 예약 생성)
getMyReservations(accountId, ReservationSearchDto, Pageable)	Page<ReservationResponseDto>	readOnly	-	내 예약 목록 (필터: 기간, 상태, 타입)
getMyReservationDetail(accountId, reservationId)	ReservationResponseDto	readOnly	Ownership	내 예약 상세
cancelReservation(accountId, reservationId, reason)	void	REQUIRED	Ownership + Cancellable + Event	예약 취소 (status=CANCELLED → 필요 시 재고 복구)

«Service» CartService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getCart(accountId)	CartResponseDto	readOnly	-	장바구니 목록 + 총 금액
getCartItemCount(accountId)	CartCountResponseDto	readOnly	-	장바구니 상품 수
addToCart(accountId, CartAddRequestDto)	CartItemResponseDto	REQUIRED	SameStoreCheck + StockCheck	동일 상점 검증 → 다르면 기존 초기화 → 재고 확인 → CartItem 저장
updateCartItemQuantity(accountId, cartItemId, CartItemUpdateRequestDto)	CartItemResponseDto	REQUIRED	Ownership + StockCheck	본인 카트 검증 + 재고 확인 + 수량 갱신
deleteCartItem(accountId, cartItemId)	void	REQUIRED	Ownership	장바구니 항목 삭제
clearCart(accountId)	void	REQUIRED	-	장바구니 전체 비우기

«Service» OwnerOrderService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getStoreOrders(accountId, storeId, OrderSearchDto, Pageable)	Page<StoreOrderResponseDto>	REQUIRED	Ownership	사장용 주문 목록 (필터: 기간, 상태)
getStoreOrderDetail(accountId, storeId, orderId)	StoreOrderDetailResponseDto	readOnly	Ownership	사장용 주문 상세
confirmOrder(accountId, storeId, orderId)	void	REQUIRED	Ownership + Event	주문 확정 → 사용자에게 알림 발송

«Service» OwnerReservationService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getStoreReservations(accountId, storeId, ReservationSearchDto, Pageable)	Page<StoreReservationResponseDto>	readOnly	Ownership	사장용 예약 목록 (필터: 기간, 상태, 타입)
getStoreReservationDetail(accountId, storeId, reservationId)	StoreReservationDetailResponseDto	readOnly	Ownership	사장용 예약 상세
confirmReservation(accountId, storeId, reservationId)	void	REQUIRED	Ownership + Event	예약 확정 (사장) → status=CONFIRMED → 사용자 알림

«Helper» OrderAccessHelper		
메서드명	반환타입	설명
verifyOrderOwnership(accountId, orderId)	Order	주문 소유권 검증 (구매자) 본인 검증 → 실패 시 ForbiddenException
verifyStoreOrderAccess(accountId, storeId, orderId)	Order	사장용: 본인 상점 + 해당 주문 검증
verifyCancellable(orderId)	Order	취소 가능 상태 검증 (PENDING/PAID)
verifyRefundable(orderId)	Order	환불 가능 상태 검증 (PAID, 환불 기간 내)
verifyPaymentOwnership(accountId, paymentId)	Payment	결제 소유권 검증

«Helper» ReservationAccessHelper		
메서드명	반환타입	설명
verifyReservationOwnership(accountId, reservationId)	Reservation	예약 소유권 검증 (예약자)
verifyStoreReservationAccess(accountId, storeId, reservationId)	Reservation	사장용: 본인 상점 + 해당 예약 검증
verifyCancellable(reservationId)	Reservation	취소 가능 상태 검증 (PENDING/CONFIRMED)
verifyConfirmable(reservationId)	Reservation	확정 가능 상태 검증 (PENDING만 허용)
verifyNotExpired(reservationId)	Reservation	예약 만료 여부 검증 (reservedAt 기준)

- Repository

«Repository» OrderRepository		
메서드명	반환타입	설명
findById(Long orderId)	Optional<Order>	주문 ID로 조회
findByIdAndAccountId(Long orderId, Long accountId)	Optional<Order>	본인 주문 검증용
findByOrderNumber(String orderNumber)	Optional<Order>	주문번호로 조회
findByIdForUpdate(Long orderId)	Optional<Order>	@Lock(PESSIMISTIC_WRITE) / 결제 처리·취소 시 동시성 제어
existsByOrderNumber(String orderNumber)	boolean	orderNumber 채번 시 충돌 검사용
findAllByAccountId(Long accountId, Pageable pageable)	Page<Order>	사용자 주문 목록
findAllByAccountIdAndStatus(Long accountId, OrderStatus status, Pageable pageable)	Page<Order>	상태별 사용자 주문 목록
findAllByAccountIdAndStatusIn(Long accountId, List<OrderStatus> statuses, Pageable pageable)	Page<Order>	다중 상태 필터 사용자 주문 조회 (예: PAID + REFUND_REQUESTED 동시 조회)
findAllByAccountIdAndCreatedAtBetween(Long accountId, LocalDateTime start, LocalDateTime end, Pageable pageable)	Page<Order>	기간별 사용자 주문
findAllByStoreId(Long storeId, Pageable pageable)	Page<Order>	사장용 상점 주문 목록
findAllByStoreIdAndStatus(Long storeId, OrderStatus status, Pageable pageable)	Page<Order>	사장용 상태별 조회
findAllByStoreIdAndStatusIn(Long storeId, List<OrderStatus> statuses, Pageable pageable)	Page<Order>	다중 상태 필터 사장용 주문 조회
findAllByStoreIdAndCreatedAtBetween(Long storeId, LocalDateTime start, LocalDateTime end, Pageable pageable)	Page<Order>	사장용 기간별 매출 조회
countByStoreIdAndStatus(Long storeId, OrderStatus status)	Long	사장 대시보드 집계용
sumTotalPriceByStoreIdAndStatusAndPaidAtBetween(Long storeId, OrderStatus status, LocalDateTime start, LocalDateTime end)	BigDecimal	기간별 매출 합계 (사장 대시보드)

«Repository» OrderItemRepository		
메서드명	반환타입	설명
findAllByOrderId(Long orderId)	List<OrderItem>	주문에 포함된 상품 목록
findAllByOrderIdIn(List<Long> orderIds)	List<OrderItem>	다수 주문의 상품 일괄 조회 (N+1 방지)
findAllByOrderIdInWithProduct(List<Long> orderIds)	List<OrderItem>	Product 동시 페치 @EntityGraph(attributePaths={"product"})
countByProductId(Long productId)	Long	상품 판매 횟수 (인기 상품 집계)
findTopProductsByStoreId(Long storeId, int limit)	List<ProductSalesDto>	상점 인기 상품 TOP-N (대시보드용)
existsByAccountIdAndProductIdAndStatus(Long accountId, Long productId, OrderStatus status)	boolean	사용자가 특정 상품을 구매했는지 검증 (리뷰 작성 권한 등)

«Repository» PaymentRepository		
메서드명	반환타입	설명
findById(Long paymentId)	Optional<Payment>	결제 ID로 조회
findByIdAndAccountId(Long paymentId, Long accountId)	Optional<Payment>	본인 결제 검증용
findByOrderId(Long orderId)	Optional<Payment>	주문에 연결된 결제 조회 (1:1)
findByPortOnePaymentId(String portOnePaymentId)	Optional<Payment>	PortOne ID로 조회 (Webhook 중복 확인)
findByIdempotencyKey(String idempotencyKey)	Optional<Payment>	멥등성 키로 중복 결제 방지
existsByIdempotencyKey(String idempotencyKey)	boolean	중복 처리 여부 확인
existsByPortOnePaymentId(String portOnePaymentId)	boolean	Webhook 중복 수신 확인
findAllByAccountId(Long accountId, Pageable pageable)	Page<Payment>	사용자 결제 내역
findAllByAccountIdAndStatus(Long accountId, PaymentStatus status, Pageable pageable)	Page<Payment>	상태별 사용자 결제 내역
findAllByStatusAndCreatedAtBefore(PaymentStatus status, LocalDateTime threshold)	List<Payment>	미완결 결제 정리용 (스케줄러)
countByStatusAndCreatedAtBetween(PaymentStatus, LocalDateTime, LocalDateTime)	Long	PortOne Webhook 처리 결과 모니터링 (관리자 대시보드)

«Repository» **ReservationRepository**

메서드명	반환타입	설명
findById(Long reservationId)	Optional<Reservation>	예약 ID로 조회
findByIdAndAccountId(Long reservationId, Long accountId)	Optional<Reservation>	본인 예약 검증용
findByIdForUpdate(Long reservationId)	Optional<Reservation>	@Lock(PESSIMISTIC_WRITE) / 예약 취소·확정 시 동시성 제어
findAllByAccountId(Long accountId, Pageable pageable)	Page<Reservation>	사용자 예약 목록
findAllByAccountIdAndStatus(Long accountId, ReservationStatus status, Pageable pageable)	Page<Reservation>	상태별 사용자 예약 목록
findAllByAccountIdAndReservationType(Long accountId, ReservationType type, Pageable pageable)	Page<Reservation>	타입별 사용자 예약 (VISIT/PRODUCT)
findAllByStoreId(Long storeId, Pageable pageable)	Page<Reservation>	사장용 예약 목록
findAllByStoreIdAndStatus(Long storeId, ReservationStatus status, Pageable pageable)	Page<Reservation>	사장용 상태별 조회
findAllByStoreIdAndReservedAtBetween(Long storeId, LocalDateTime start, LocalDateTime end)	List<Reservation>	매장 예약 일정 조회 (캘린더용)
countByStoreIdAndReservedAtBetweenAndStatus(Long storeId, LocalDateTime startOfMinute, LocalDateTime endOfMinute, ReservationStatus status)	Long	분 단위 동시 예약 수 (capacity 검증용, Redis Lock과 함께 사용)
existsByOrderId(Long orderId)	boolean	주문 연결 예약 중복 방지
findByOrderId(Long orderId)	Optional<Reservation>	주문 연결 예약 조회

«Repository» CartRepository		
메서드명	반환타입	설명
findByAccountIdAndStoreId(Long accountId, Long storeId)	Optional<Cart>	특정 가게의 장바구니 조회
findAllByAccountId(Long accountId)	List<Cart>	다른storeId 기존 장바구니 정리용
existsByAccountIdAndStoreId(Long accountId, Long storeId)	boolean	특정 가게 장바구니 존재 여부
deleteByAccountIdAndStoreId(Long accountId, Long storeId)	void	장바구니 삭제
deleteAllByAccountId(Long accountId)	void	회원 탈퇴 시CASCADE

«Repository» CartItemRepository		
메서드명	반환타입	설명
findById(Long cartItemId)	Optional<CartItem>	장바구니 항목 조회
findByIdAndCart_AccountId(Long cartItemId, Long accountId)	Optional<CartItem>	본인 장바구니 항목 검증용
findAllByCartId(Long cartId)	List<CartItem>	장바구니의 모든 항목
findByCartIdAndProductIdAndEventProductId(Long cartId, Long productId, Long eventProductId)	Optional<CartItem>	일반/이벤트 상품 별도 카트 항목
existsByCartIdAndProductIdAndEventProductId(Long cartId, Long productId, Long eventProductId)	boolean	상품 중복 확인
countByCartId(Long cartId)	Long	장바구니 상품 수
deleteAllByCartId(Long cartId)	void	장바구니 전체 비우기
deleteByCartIdAndProductIdAndEventProductId(Long cartId, Long productId, Long eventProductId)	void	특정 조합 카트 항목 제거

5.4 UsedProduct 도메인

- Entity

«Entity» UsedProduct			
속성명	타입	제약조건	설명
usedproductId	Long	PK, NOT NULL	고유 식별자
accountId	Long	FK, NOT NULL	판매자
buyerId	Long	FK, nullable	구매자 (예약/완료 시 설정)
regionId	Long	FK, NOT NULL	게시 활동 지역
categoryId	Long	FK,NOT NULL	Category 테이블 참조, type=USED 검증은 Service 레벨
title	String	NOT NULL	제목
description	String	NOT NULL	상품 설명
price	BigDecimal	NOT NULL, ≥0	희망 가격
tradeLocation	String	nullable	거래 희망 장소
viewCount	int	DEFAULT 0	조회수 (Redis 캐싱 대상)
status	UsedProductStatus	NOT NULL	ON_SALE / RESERVED / SOLD
favoriteCount	int	NOT NULL,Default 0	중고거래 좋아요 수
isBuyerConfirmed	boolean	NOT NULL, DEFAULT false	구매자 거래 완료 확인 여부
version	Long	NOT NULL	낙관적 락

createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» UsedProductReview			
속성명	타입	제약조건	설명
usedproductreviewId	Long	PK, NOT NULL	고유 식별자
usedproductId	Long	FK, NOT NULL, UNIQUE	연관 주문(중복 방지)
accountId	Long	FK, NOT NULL	작성자
rating	int	NOT NULL, 1~5	별점
content	String	NOT NULL	리뷰 내용
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» UsedProduct (도메인 메서드)		
메서드명	타입	설명
reserve(buyerId)	void	ON_SALE 상태에서만 예약 처리 (buyerId 설정, RESERVED 변경)
cancelReservation()	void	RESERVED 상태에서 판매자가 예약 취소 → ON_SALE 복귀
confirmPurchase(buyerId)	void	구매자 거래 완료 확인 (isBuyerConfirmed = true)
completeSale(accountId)	void	구매자 확인 이후 판매자가 SOLD 처리
changeStatus(status)	void	내부 상태 변경 (외부 직접 호출 금지)
incrementViewCount()	void	조회수 증가 (Redis 경유)
isEditable(accountId)	boolean	판매자이며 ON_SALE 상태일 때만 수정 가능
isOwnedBy(accountId)	boolean	판매자 본인 여부 확인
isDeletable(accountId)	boolean	판매자이며 SOLD가 아닐 경우 삭제 가능
isPurchasable()	boolean	ON_SALE 상태 여부 반환

- **Controller**

«Controller» UsedProductController			
MAP 방식	API명	반환타입	설명
POST	/used-products	ResponseEntity<UsedProductResponseDto>	중고 게시글 등록
PATCH	/used-products/{productId}	ResponseEntity<UsedProductResponseDto>	중고 게시글 수정
DELETE	/used-products/{productId}	ResponseEntity<CommonResponseDto>	중고 게시글 삭제
GET	/used-products	ResponseEntity<Page<UsedProductResponseDto>>	중고 게시글 목록 조회
GET	/used-products/{productId}	ResponseEntity<UsedProductDetailResponseDto>	중고 게시글 상세 조회
POST	/used-products/{productId}/reserve	ResponseEntity<CommonResponseDto>	중고 거래 예약
PATCH	/used-products/{productId}/reserve/cancel	ResponseEntity<CommonResponseDto>	거래 예약 취소 (RESERVED → ON_SALE)
POST	/used-products/{productId}/confirm	ResponseEntity<CommonResponseDto>	거래 완료 처리 (구매자 확정 → SOLD)
POST	/used-products/{productId}/images	ResponseEntity<UsedProductImageResponseDto>	중고 거래 상품 이미지 추가
DELETE	/used-products/{productId}/images/{imageId}	ResponseEntity<CommonResponseDto>	중고 거래 상품 이미지 삭제
PATCH	/used-products/{productId}/images/{imageId}/thumbnail	ResponseEntity<CommonResponseDto>	중고 거래 상품 대표 이미지 지정

«Controller» UsedProductReviewController			
MAP 방식	API명	반환타입	설명
GET	/used-products/{productId}/reviews/{reviewId}	ResponseEntity<UsedProductReviewDetailResponseDto>	리뷰 상세 조회
POST	/used-products/{productId}/reviews	ResponseEntity<UsedProductReviewResponseDto>	중고 거래 리뷰 작성
PATCH	/used-products/{productId}/reviews/{reviewId}	ResponseEntity<UsedProductReviewResponseDto>	중고 거래 리뷰 수정 (본인만)
DELETE	/used-products/{productId}/reviews/{reviewId}	ResponseEntity<CommonResponseDto>	중고 거래 리뷰 삭제 (본인만)

«Controller» AdminUsedProductController			
MAP 방식	API명	반환타입	설명
GET	/admin/used-products	ResponseEntity<Page<UsedProductResponseDto>>	전체 중고 상품 조회 (상태/지역/기간 필터)
GET	/admin/used-products/{productId}	ResponseEntity<UsedProductDetailResponseDto>	중고 상품 상세 조회
DELETE	/admin/used-products/{productId}	ResponseEntity<CommonResponseDto>	중고 상품 강제 삭제 (신고 처리)
GET	/admin/used-products/reviews	ResponseEntity<Page<UsedProductReviewResponseDto>>	전체 리뷰 조회
DELETE	/admin/used-products/reviews/{reviewId}	ResponseEntity<CommonResponseDto>	리뷰 강제 삭제

- Service

«Service» UsedProductService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getUsedProductList(UsedProductSearchDto)	Page<UsedProductResponseDto>	readOnly	QueryDSL	중고 상품 목록 조회
getUsedProductDetail(usedProductId)	UsedProductDetailResponseDto	readOnly	-	중고 상품 상세 조회 (조회수 증가 포함)
registerUsedProduct(accountId, UsedProductCreateRequestDto)	UsedProductResponseDto	REQUIRED	-	중고 상품 등록
updateUsedProduct(accountId, usedProductId, UsedProductUpdateRequestDto)	UsedProductResponseDto	REQUIRED	Ownership	중고 상품 수정
deleteUsedProduct(accountId, usedproductId)	void	REQUIRED	Ownership	중고 상품 삭제
reserveUsedProduct(accountId, usedProductId)	CommonResponseDto	REQUIRED	OptimisticLock + Event	중고 상품 예약
cancelReservation(accountId, usedproductId)	CommonResponseDto	REQUIRED	Ownership + RESERVED	중고 상품 예약 취소
confirmPurchase(accountId, usedProductId)	CommonResponseDto	REQUIRED	BuyerOnly + RESERVED + Event	구매자 거래 완료 확정
getMyUsedProducts(accountId, Pageable)	Page<UsedProductResponseDto>	readOnly	-	내가 등록한 중고 상품
getMyPurchasedUsedProducts(accountId, Pageable)	Page<UsedProductResponseDto>	readOnly	-	내가 거래한 중고 상품
incrementViewCount(usedProductId)	void	REQUIRED	REDIS	조회수 증가 (Redis 집계 → 5분 주기 배치 동기화)
addUsedProductImage(accountId, usedProductId, imageUrl)	UsedProductImageResponseDto	REQUIRED	Ownership + ImageService	상품 이미지 추가
deleteUsedProductImage(accountId, usedProductId, imageId)	void	REQUIRED	Ownership + ImageService	상품 이미지 삭제
setUsedProductImageThumbnail(accountId, usedProductId, imageId)	void	REQUIRED	Ownership + ImageService	상품 대표 이미지 지정

incrementFavoriteCount(usedProductId)	void	REQUIRED	Internal	상품 찜 개수 증가
decrementFavoriteCount(usedProductId)	void	REQUIRED	Internal	상품 찜 개수 감소

«Service» UsedProductReviewService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getUsedProductReviewDetail(reviewId)	UsedProductReviewDetailResponseDto	readOnly	-	리뷰 상세 조회
registerUsedProductReview(accountId, usedproductId, UsedProductReviewCreateRequestDto)	UsedProductReviewResponseDto	REQUIRED	UNIQUE(productId) + BuyerOnly	리뷰 등록
updateUsedProductReview(accountId, reviewId, UsedProductReviewUpdateRequestDto)	UsedProductReviewResponseDto	REQUIRED	Ownership	리뷰 수정
deleteUsedProductReview(accountId, reviewId)	void	REQUIRED	Ownership	리뷰 삭제
getMyReceivedUsedReviews(accountId, Pageable)	Page<UsedProductReviewResponseDto>	readOnly		내가 받은 중고거래 리뷰
getMyWrittenUsedReviews(accountId, Pageable)	Page<UsedProductReviewResponseDto>	readOnly		내가 작성한 중고거래 리뷰

«Service» **AdminUsedProductService**

메서드명	반환타입	트랜잭션	정책 / AOP	설명
getAllUsedProducts(Pageable, filter)	Page<UsedProductResponseDto>	readOnly	Admin	전체 중고 상품 조회
getUsedProductDetail(usedProductId)	UsedProductDetailResponseDto	readOnly	Admin	중고 상품 상세
forceDeleteUsedProduct(usedProductId)	void	REQUIRED	Admin	중고 상품 강제 삭제
getAllReviews(Pageable)	Page<UsedProductReviewResponseDto>	readOnly	Admin	전체 리뷰 조회
forceDeleteReview(reviewId)	void	REQUIRED	Admin	중고 상품 리뷰 강제 삭제

«Helper» **UsedProductAccessHelper**

메서드명	반환타입	설명
verifyOwnership(accountId, usedProductId)	UsedProduct	판매자 본인 검증 → 실패 시 ForbiddenException
verifyBuyer(accountId, usedProductId)	UsedProduct	예약된 구매자 본인 검증
verifyOnSale(usedProductId)	UsedProduct	ON_SALE 상태 검증
verifyReserved(usedProductId)	UsedProduct	RESERVED 상태 검증

- Repository

«Repository» UsedProductRepository		
메서드명	반환타입	설명
findById(Long usedproductId)	Optional<UsedProduct>	기본 조회
findByIdForUpdate(Long usedproductId)	Optional<UsedProduct>	@Lock(PESSIMISTIC_WRITE) / 거래 상태 변경 시 사용
findAll(Pageable pageable)	Page<UsedProduct>	전체 상품 목록 (페이징)
findAllByRegionId(Long regionId, Pageable pageable)	Page<UsedProduct>	지역 기반 상품 목록
findAllByCategoryId(Long categoryId, Pageable pageable)	Page<UsedProduct>	카테고리 기반 상품 목록
findAllByStatus(UsedProductStatus status, Pageable pageable)	Page<UsedProduct>	상태별 상품 목록 (ON_SALE / RESERVED / SOLD)
countByAccountId(Long accountId)	Long	판매자 상품 수
countByBuyerId(Long buyerId)	Long	구매자 거래 수
incrementFavoriteCount(Long usedproductId)	int	찜 카운트 +1 (DB 원자 UPDATE, 영향받은 행 수 반환)
decrementFavoriteCount(Long usedproductId)	int	찜 카운트 -1 (favoriteCount > 0 가드, 영향받은 행 수 반환)

«Repository» UsedProductRepositoryCustom (QueryDSL)		
메서드명	반환타입	설명
searchProducts(ProductSearchDto condition, Pageable pageable)	Page<UsedProduct>	지역 + 카테고리 + 키워드 + 정렬 복합 검색
findNearbyProducts(NearbySearchCondition condition)	List<UsedProduct>	위치 기반 상품 검색

«Repository» UsedProductImageRepository		
메서드명	반환타입	설명
findAllByUsedproductIdOrderByDisplayOrder(Long usedproductId)	List<UsedProductImage>	중고상품 이미지 목록 (순서대로)
findByUsedproductIdAndIsThumbnailTrue(Long usedproductId)	Optional<UsedProductImage>	대표 이미지 조회
findByIdAndUsedproductId(Long imageId, Long usedproductId)	Optional<UsedProductImage>	소유권 확인용 조회
countByUsedproductId(Long usedproductId)	Long	상품 이미지 개수 (제한 검증용)
findFirstByUsedproductIdOrderByDisplayOrderAsc(Long usedproductId)	Optional<UsedProductImage>	대표 자동 승격용
deleteAllByUsedproductId(Long usedproductId)	void	게시글 삭제 시 CASCADE

«Repository» UsedProductReviewRepository		
메서드명	반환타입	설명
findById(Long usedproductReviewId)	Optional<UsedProductReview>	리뷰 ID로 조회
findByUsedProductId(Long usedproductId)	Optional<UsedProductReview>	상품별 리뷰 조회
existsByUsedProductId(Long usedproductId)	boolean	리뷰 존재 여부
countByAccountId(Long accountId)	Long	사용자 리뷰 수

5.5 Community 도메인

- Entity

«Entity» CommunityPost			
속성명	타입	제약조건	설명
communitypostId	Long	PK, NOT NULL	고유 식별자
accountId	Long	FK, NOT NULL	작성자
categoryId	Long	FK, NOT NULL	카테고리 (Category, type=COMMUNITY)
regionId	Long	FK, NOT NULL	게시 활동 지역
title	String	NOT NULL	제목
content	String	NOT NULL	본문
viewCount	int	NOT NULL, DEFAULT 0	조회수 (Redis 캐싱)
likeCount	int	NOT NULL, DEFAULT 0	좋아요 수 (Redis 캐싱)
commentCount	int	NOT NULL, DEFAULT 0	댓글 수 (Redis 캐싱)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» CommunityImage			
속성명	타입	제약조건	설명
communityimageId	Long	PK, NOT NULL	이미지 고유 식별자(ImageBase 상속)
communitypostId	Long	FK, NOT NULL	연관 게시글
imageUrl	String	NOT NULL	이미지 URL(ImageBase 상속)
displayOrder	int	NOT NULL, DEFAULT 0	표시 순서(ImageBase 상속)
isThumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부(ImageBase 상속)
createdAt	LocalDateTime	NOT NULL	ImageBase 상속
modifiedAt	LocalDateTime	NOT NULL	ImageBase 상속

«Entity» CommunityComment			
속성명	타입	제약조건	설명
communitycommentId	Long	PK, NOT NULL	고유 식별자
communitypostId	Long	FK, NOT NULL	연관 게시글
accountId	Long	FK, NOT NULL	작성자
content	String	NOT NULL	댓글 내용
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» CommunityCommentReply			
속성명	타입	제약조건	설명
communitcommentreplyld	Long	PK, NOT NULL	고유 식별자
communitycommentld	Long	FK, NOT NULL	연관 댓글
accountld	Long	FK, NOT NULL	작성자
content	String	NOT NULL	대댓글 내용
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» PostLike			
속성명	타입	제약조건	설명
postlikeld	Long	PK, NOT NULL	고유 식별자
accountld	Long	FK, NOT NULL	좋아요 누른 사용자
communitypostld	Long	FK, NOT NULL	좋아요 대상 게시글

- **Controller**

«Controller» CommunityController			
MAP 방식	API명	반환타입	설명
POST	/community/posts	ResponseEntity<CommunityPostResponseDto>	게시글 작성
PATCH	/community/posts/{postId}	ResponseEntity<CommunityPostResponseDto>	게시글 수정 (본인만)
DELETE	/community/posts/{postId}	ResponseEntity<CommonResponseDto>	게시글 삭제 (본인만)
GET	/community/posts	ResponseEntity<Page<CommunityPostResponseDto>>	게시글 목록 (필터: 카테고리, 지역, 키워드)
GET	/community/posts/{postId}	ResponseEntity<CommunityPostDetailResponseDto>	게시글 상세 (조회수 증가)
POST	/community/posts/{postId}/like	ResponseEntity<CommonResponseDto>	좋아요 등록/해제 토글(Favorite와 별개)
POST	/community/posts/{postId}/images	ResponseEntity<CommunityImageResponseDto>	게시글 이미지 추가
DELETE	/community/posts/{postId}/images/{imageId}	ResponseEntity<CommonResponseDto>	게시글 이미지 삭제
PATCH	/community/posts/{postId}/images/{imageId}/thumbnail	ResponseEntity<CommonResponseDto>	대표 이미지 지정
GET	/community/posts/me	ResponseEntity<Page<CommunityPostResponseDto>>	내가 쓴 게시글
GET	/community/posts/me/liked	ResponseEntity<Page<CommunityPostResponseDto>>	좋아요(PostLike) 누른 게시글 (Favorite와 별개)

«Controller» CommunityCommentController			
MAP 방식	API명	반환타입	설명
GET	/community/posts/{postId}/comments	ResponseEntity<Page<CommunityCommentResponseDto>>	댓글 목록 (대댓글 포함)
POST	/community/posts/{postId}/comments	ResponseEntity<CommunityCommentResponseDto>	댓글 작성
PATCH	/community/comments/{commentId}	ResponseEntity<CommunityCommentResponseDto>	댓글 수정 (본인만)
DELETE	/community/comments/{commentId}	ResponseEntity<CommonResponseDto>	댓글 삭제 (본인만, CASCADE)
POST	/community/comments/{commentId}/replies	ResponseEntity<CommunityCommentReplyResponseDto>	대댓글 작성
PATCH	/community/replies/{replyId}	ResponseEntity<CommunityCommentReplyResponseDto>	대댓글 수정 (본인만)
DELETE	/community/replies/{replyId}	ResponseEntity<CommonResponseDto>	대댓글 삭제 (본인만)
GET	/community/comments/me	ResponseEntity<Page<CommunityCommentResponseDto>>	내가 쓴 댓글

«Controller» AdminCommunityController			
MAP 방식	API명	반환타입	설명
GET	/admin/community/posts	ResponseEntity<Page<CommunityPostResponseDto>>	전체 게시물 조회 (필터: 카테고리/지역/기간)
DELETE	/admin/community/posts/{postId}	ResponseEntity<CommonResponseDto>	게시글 강제 삭제 (신고 처리)
GET	/admin/community/comments	ResponseEntity<Page<CommunityCommentResponseDto>>	전체 댓글 조회
DELETE	/admin/community/comments/{commentId}	ResponseEntity<CommonResponseDto>	댓글 강제 삭제
DELETE	/admin/community/replies/{replyId}	ResponseEntity<CommonResponseDto>	대댓글 강제 삭제

- Service

«Service» CommunityPostService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getCommunityPostList(CommunityPostSearchDto, Pageable)	Page<CommunityPostResponseDto>	readOnly	QueryDSL	게시글 목록 (카테고리/지역/키워드 복합 필터)
getCommunityPostDetail(postId)	CommunityPostDetailResponseDto	readOnly	-	게시글 상세 조회 (조회수는 별도 incrementViewCount 메서드 호출)
registerCommunityPost(accountId, CommunityPostCreateRequestDto)	CommunityPostResponseDto	REQUIRED	-	게시글 등록
updateCommunityPost(accountId, postId, CommunityPostUpdateRequestDto)	CommunityPostResponseDto	REQUIRED	Ownership	게시글 수정
deleteCommunityPost(accountId, postId)	void	REQUIRED	Ownership	게시글 삭제 (CASCADE로 이미지/댓글/좋아요 동시 삭제)
togglePostLike(accountId, postId)	void	REQUIRED	Event	좋아요 토글 (likeCount 갱신 + 알림 이벤트)
incrementViewCount(postId)	void	REQUIRED	REDIS	조회수 증가 (Redis 집계 → 5분 주기 배치 동기화)
getMyPosts(accountId, Pageable)	Page<CommunityPostResponseDto>	readOnly	-	내가 쓴 게시글
getLikedPosts(accountId, Pageable)	Page<CommunityPostResponseDto>	readOnly	-	좋아요한 게시글
addPostImage(accountId, postId, imageUrl)	CommunityImageResponseDto	REQUIRED	Ownership + ImageService	게시글 이미지 추가
deletePostImage(accountId, postId, imageId)	void	REQUIRED	Ownership + ImageService	게시글 이미지 삭제
setPostImageThumbnail(accountId, postId, imageId)	void	REQUIRED	Ownership + ImageService	대표 이미지 지정

«Service» **CommunityCommentService**

메서드명	반환타입	트랜잭션	정책 / AOP	설명
getCommentsByPost(postId, Pageable)	Page<CommunityCommentResponseDto>	readOnly	-	게시글 댓글 목록 (대댓글 포함, N+1 방지)
registerComment(accountId, postId, CommunityCommentCreateRequestDto)	CommunityCommentResponseDto	REQUIRED	Event	댓글 작성 (commentCount 증가 + 게시글 작성자 알림)
updateComment(accountId, commentId, CommunityCommentUpdateRequestDto)	CommunityCommentResponseDto	REQUIRED	Ownership	댓글 수정
deleteComment(accountId, commentId)	void	REQUIRED	Ownership	댓글 삭제 + CASCADE 대댓글 삭제 + commentCount 감소
registerReply(accountId, commentId, CommunityCommentReplyCreateRequestDto)	CommunityCommentReplyResponseDto	REQUIRED	Event	대댓글 작성 (commentCount 증가 + 댓글 작성자 알림)
updateReply(accountId, replyId, CommunityCommentReplyUpdateRequestDto)	CommunityCommentReplyResponseDto	REQUIRED	Ownership	대댓글 수정
deleteReply(accountId, replyId)	void	REQUIRED	Ownership	대댓글 삭제 (commentCount 감소)
getMyComments(accountId, Pageable)	Page<CommunityCommentResponseDto>	readOnly	-	내가 쓴 댓글

«Service» AdminCommunityService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getAllPosts(Pageable, filter)	Page<CommunityPostResponseDto>	readOnly	Admin	전체 게시물 조회
forceDeletePost(postId)	void	REQUIRED	Admin	게시글 강제 삭제 (신고 처리)
getAllComments(Pageable)	Page<CommunityCommentResponseDto>	readOnly	Admin	전체 댓글 + 대댓글 일괄 조회
forceDeleteComment(commentId)	void	REQUIRED	Admin	댓글 강제 삭제
forceDeleteReply(replyId)	void	REQUIRED	Admin	대댓글 강제 삭제

«Helper» CommunityAccessHelper		
메서드명	반환타입	설명
verifyPostOwnership(accountId, postId)	CommunityPost	게시글 작성자 본인 검증 → 실패 시 ForbiddenException
verifyCommentOwnership(accountId, commentId)	CommunityComment	댓글 작성자 본인 검증
verifyReplyOwnership(accountId, replyId)	CommunityCommentReply	대댓글 작성자 본인 검증

- Repository

«Repository» CommunityPostRepository		
메서드명	반환타입	설명
findById(Long postId)	Optional<CommunityPost>	게시글 ID로 조회
findByIdAndAccountId(Long postId, Long accountId)	Optional<CommunityPost>	본인 게시글 검증용
findAllByAccountId(Long accountId, Pageable pageable)	Page<CommunityPost>	사용자 게시글 목록 (내가 쓴 글)
findAllByCategoryId(Long categoryId, Pageable pageable)	Page<CommunityPost>	카테고리별 게시글
findAllByRegionId(Long regionId, Pageable pageable)	Page<CommunityPost>	지역별 게시글
countByAccountId(Long accountId)	Long	사용자 게시글 수

«Repository» CommunityPostRepositoryCustom (QueryDSL)		
메서드명	반환타입	설명
searchPosts(CommunityPostSearchDto, Pageable)	Page<CommunityPost>	카테고리 + 지역 + 키워드(제목/본문) + 정렬 복합 검색
findLikedPostsByAccountId(Long accountId, Pageable)	Page<CommunityPost>	좋아요한 게시글 (PostLike JOIN)

«Repository» CommunityImageRepository		
메서드명	반환타입	설명
findById(Long imageId)	Optional<CommunityImage>	이미지 조회
findAllByCommunityPostId(Long postId)	List<CommunityImage>	게시글 이미지 목록 (displayOrder 정렬)
findThumbnailByCommunityPostId(Long postId)	Optional<CommunityImage>	대표 이미지 조회 (목록 화면용)
countByCommunityPostId(Long postId)	Long	이미지 개수 (업로드 제한 검증)
findFirstByCommunityPostIdOrderByDisplayOrderAsc(Long postId)	Optional<CommunityImage>	대표 자동 승격용
deleteAllByCommunityPostId(Long postId)	void	게시글 삭제 시 CASCADE

«Repository» CommunityCommentRepository		
메서드명	반환타입	설명
findById(Long commentId)	Optional<CommunityComment>	댓글 조회
findByIdAndAccountId(Long commentId, Long accountId)	Optional<CommunityComment>	본인 댓글 검증용
findAllByCommunityPostId(Long postId, Pageable pageable)	Page<CommunityComment>	게시글 댓글 목록
findAllByAccountId(Long accountId, Pageable pageable)	Page<CommunityComment>	사용자 댓글 목록
countByCommunityPostId(Long postId)	Long	게시글 댓글 수 (commentCount 동기화 검증용)
deleteAllByCommunityPostId(Long postId)	void	게시글 삭제 시 CASCADE

«Repository» CommunityCommentReplyRepository		
메서드명	반환타입	설명
findById(Long replyId)	Optional<CommunityCommentReply>	대댓글 조회
findByIdAndAccountId(Long replyId, Long accountId)	Optional<CommunityCommentReply>	본인 대댓글 검증용
findAllByCommunityCommentId(Long commentId)	List<CommunityCommentReply>	댓글의 대댓글 목록
findAllByCommunityCommentIdIn(List<Long> commentIds)	List<CommunityCommentReply>	다수 댓글의 대댓글 일괄 조회 (N+1 방지)
countByCommunityCommentId(Long commentId)	Long	대댓글 수
deleteAllByCommunityCommentId(Long commentId)	void	댓글 삭제 시 CASCADE

«Repository» PostLikeRepository		
메서드명	반환타입	설명
findByAccountIdAndCommunityPostId(Long accountId, Long postId)	Optional<PostLike>	좋아요 조회 (토글용)
existsByAccountIdAndCommunityPostId(Long accountId, Long postId)	boolean	좋아요 여부 확인
countByCommunityPostId(Long postId)	Long	게시글 좋아요 수 (likeCount 동기화 검증용)
findAllByAccountId(Long accountId, Pageable pageable)	Page<PostLike>	사용자가 좋아요한 목록
deleteByAccountIdAndCommunityPostId(Long accountId, Long postId)	void	좋아요 해제
deleteAllByCommunityPostId(Long postId)	void	게시글 삭제 시 CASCADE

5.6 Chat 도메인

- Entity

«Entity» ChatRoom			
속성명	타입	제약조건	설명
chatroomId	Long	PK, NOT NULL	고유 식별자
createdBy	Long	FK, NOT NULL	채팅방 생성자
type	ChatRoomType	NOT NULL	PRIVATE / GROUP / GROUP_STREET
refType	ChatRoomRefType	ENUM	Polymorphic 참조 타입 (TRADE / STORE / COMMUNITY 등)
refId	Long	nullable	연관 도메인 ID (Polymorphic 참조, FK 아님)
name	String	nullable	채팅방 이름 (GROUP 타입에서 사용)
isActive	boolean	NOT NULL, DEFAULT True	채팅방 활성 상태 (GROUP 전체 퇴장 시 false)
lastMessageAt	LocalDateTime	nullable	마지막 메시지 시각 (목록 정렬용)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» ChatParticipant			
속성명	타입	제약조건	설명
chatparticipantId	Long	PK, NOT NULL	고유 식별자
chatroomId	Long	FK, NOT NULL	연관 채팅방
accountId	Long	FK, NOT NULL	참여 사용자
lastReadTime	LocalDateTime	nullable	마지막 읽은 시각 (안 읽은 메시지 수 계산)
status	ParticipantStatus	NOT NULL, DEFAULT ACTIVE	ACTIVE / LEFT
joinedAt	LocalDateTime	NOT NULL	참여 시각
leftAt	LocalDateTime	nullable	퇴장 시각
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» ChatMessage			
속성명	타입	제약조건	설명
chatmessageld	Long	PK, NOT NULL	고유 식별자
chatroomId	Long	FK, NOT NULL	연관 채팅방
accountId	Long	FK, NOT NULL	발신자
content	String	nullable	메시지 내용 (TEXT/SYSTEM 타입)
imageUrl	String	nullable	이미지 URL (IMAGE 타입)
messageType	MessageType	NOT NULL	TEXT / IMAGE / SYSTEM
isDeleted	boolean	NOT NULL, DEFAULT false	발신자 본인 삭제 여부 (Soft Delete)
sentAt	LocalDateTime	NOT NULL	발신 시각 (인덱스 대상)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» ChatRoom (도메인 메서드)

메서드명	반환타입	설명
updateLastMessageAt(LocalDateTime)	void	마지막 메시지 시각 갱신
deactivate()	void	채팅방 비활성화 (isActive=false, GROUP 전체 퇴장 시)
isPrivate()	boolean	PRIVATE 타입 여부
isGroup()	boolean	GROUP 또는 GROUP_STREET 타입 여부

«Entity» ChatParticipant (도메인 메서드)

메서드명	반환타입	설명
leave()	void	퇴장 처리 (status=LEFT, leftAt 기록)
updateLastReadTime(LocalDateTime)	void	마지막 읽은 시각 갱신
isActive()	boolean	ACTIVE 상태 여부

«Entity» ChatMessage (도메인 메서드)

메서드명	반환타입	설명
markDeleted()	void	Soft Delete (isDeleted=true)
isOwnedBy(accountId)	boolean	발신자 본인 여부 확인
isDeletable(accountId)	boolean	본인 발신 + 미삭제 상태일 때만 삭제 가능

- **Controller**

«Controller» ChatRoomController			
MAP 방식	API명	반환타입	설명
POST	/chat/rooms/private	ResponseEntity<ChatRoomResponseDto>	1:1 채팅방 생성/조회 (refType+refId 기준 역등)
POST	/chat/rooms/group	ResponseEntity<ChatRoomResponseDto>	GROUP 채팅방 생성 (참여자 다수 초대)
GET	/chat/rooms	ResponseEntity<Page<ChatRoomResponseDto>>	내 채팅방 목록 (lastMessageAt 정렬, unreadCount 포함)
GET	/chat/rooms/{roomId}	ResponseEntity<ChatRoomDetailResponseDto>	채팅방 상세 (참여자 목록 포함)
PATCH	/chat/rooms/{roomId}/leave	ResponseEntity<CommonResponseDto>	채팅방 나가기 (PRIVATE→LEFT, GROUP 전체 퇴장→방 비활성)
POST	/chat/rooms/{roomId}/participants	ResponseEntity<CommonResponseDto>	GROUP 채팅방 참여자 초대
PATCH	/chat/rooms/{roomId}/read	ResponseEntity<CommonResponseDto>	읽음 처리 (lastReadTime 갱신)

«Controller» ChatMessageController			
MAP 방식	API명	반환타입	설명
GET	/chat/rooms/{roomId}/messages	ResponseEntity<Page<ChatMessageResponseDto>>	메시지 목록 조회 (커서 페이징, 최신→과거)
POST	/chat/rooms/{roomId}/messages	ResponseEntity<ChatMessageResponseDto>	메시지 발송 (REST 폴백 — WebSocket 실패 시)
POST	/chat/rooms/{roomId}/messages/image	ResponseEntity<ChatMessageResponseDto>	이미지 메시지 발송
DELETE	/chat/messages/{messageId}	ResponseEntity<CommonResponseDto>	메시지 삭제 (본인만, Soft Delete)
GET	/chat/messages/unread/count	ResponseEntity<UnreadCountResponseDto>	전체 안 읽은 메시지 수 (배지 표시용)

«Controller» **ChatStompController (STOMP / WebSocket)**

MAPPING	메시지 매핑	Payload	설명
@MessageMapping	/chat/{roomId}/send	ChatMessageRequestDto	메시지 전송 → /topic/chat/{roomId} 브로드캐스트
@MessageMapping	/chat/{roomId}/read	ChatReadRequestDto	읽음 이벤트 전송 → /topic/chat/{roomId}/read
@MessageMapping	/chat/{roomId}/typing	TypingEventDto	타이핑 인디케이터 → /topic/chat/{roomId}/typing
@SubscribeMapping	/topic/chat/{roomId}	-	채팅방 구독 (인증·참여자 검증 인터셉터)

«Controller» **AdminChatController**

MAP 방식	API명	반환타입	설명
GET	/admin/chat/rooms	ResponseEntity<Page<ChatRoomResponseDto>>	전체 채팅방 조회 (필터: 타입/기간)
GET	/admin/chat/rooms/{roomId}/messages	ResponseEntity<Page<ChatMessageResponseDto>>	신고된 채팅방 메시지 조회
DELETE	/admin/chat/messages/{messageId}	ResponseEntity<CommonResponseDto>	메시지 강제 삭제 (신고 처리)
DELETE	/admin/chat/rooms/{roomId}	ResponseEntity<CommonResponseDto>	채팅방 강제 비활성화

- Service

«Service» ChatRoomService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
createPrivateRoom(accountId, ChatRoomCreateRequestDto)	ChatRoomResponseDto	REQUIRED	RedisLock("chat:private:{refType}:{refId}")	1:1 채팅방 생성/조회 (refType+refId 기준 먹등 — 기존 방 있으면 반환)
createGroupRoom(accountId, ChatRoomCreateRequestDto)	ChatRoomResponseDto	REQUIRED	-	GROUP 채팅방 생성 + 참여자 일괄 초대
getMyRooms(accountId, Pageable)	Page<ChatRoomResponseDto>	readOnly	-	내 채팅방 목록 (ACTIVE 참여자 기준, lastMessageAt 내림차순, unreadCount 계산)
getRoomDetail(accountId, roomId)	ChatRoomDetailResponseDto	readOnly	Participant	채팅방 상세 (참여자 목록 포함)
inviteParticipants(accountId, roomId, List<Long> accountIds)	void	REQUIRED	Participant + GroupOnly + Event	GROUP 채팅방 참여자 초대 + 입장 SYSTEM 메시지 발행
leaveRoom(accountId, roomId)	void	REQUIRED	Participant + Event	퇴장 (status=LEFT, leftAt 기록). GROUP에서 전체 퇴장 시 isActive=false 처리 + 퇴장 SYSTEM 메시지
markRoomAsRead(accountId, roomId)	void	REQUIRED	Participant	lastReadTime 갱신 → 해당 방 unread 캐시 0 으로 리셋 + 사용자 전체 unread에서 차감

«Service» ChatMessageService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
sendMessage(accountId, roomId, ChatMessageSendRequestDto)	ChatMessageResponseDto	REQUIRED	Participant + Event	메시지 저장 → ChatRoom.lastMessageAt 갱신 → STOMP 브로드캐스트 + 알림 이벤트
sendImageMessage(accountId, roomId, MultipartFile)	ChatMessageResponseDto	REQUIRED	Participant + ImageService + Event	S3 업로드는 트랜잭션 외부에서 수행 (또는 Presigned URL 방식). DB 트랜잭션은 메시지 저장만 책임
getMessages(accountId, roomId, ChatMessageSearchDto, Pageable)	Page<ChatMessageResponseDto>	readOnly	Participant	메시지 목록 (커서 페이징, isDeleted=true는 "삭제된 메시지" 표시)
deleteMessage(accountId, messageId)	void	REQUIRED	Ownership	본인 메시지 Soft Delete → 브로드캐스트
countUnreadByAccountIdFromDb(Long accountId)	UnreadCountResponseDto	readOnly	-	전체 안 읽은 메시지 수 Redis에서 unread:account:{accountId} 조회 → 캐시 미스 시 DB 쿼리 후 Redis 복구
broadcastTyping(accountId, roomId)	void	-	Participant	타이핑 이벤트 STOMP 브로드캐스트 (DB 저장 X)

«Service» AdminChatService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getAllRooms(Pageable, filter)	Page<ChatRoomResponseDto>	readOnly	Admin	전체 채팅방 조회
getRoomMessages(roomId, Pageable)	Page<ChatMessageResponseDto>	readOnly	Admin	신고된 채팅방 메시지 조회
forceDeleteMessage(messageId)	void	REQUIRED	Admin	메시지 강제 삭제
forceDeactivateRoom(roomId)	void	REQUIRED	Admin	채팅방 강제 비활성화

«Helper» ChatAccessHelper

메서드명	반환타입	설명
verifyParticipant(accountId, roomId)	ChatParticipant	채팅방 참여자 (ACTIVE) 검증 → 실패 시 ForbiddenException
verifyMessageOwnership(accountId, messageId)	ChatMessage	메시지 발신자 본인 검증
verifyRoomCreator(accountId, roomId)	ChatRoom	채팅방 생성자 검증 (GROUP 관리 권한)
verifyGroupRoom(roomId)	ChatRoom	GROUP/GROUP_STREET 타입 검증 (PRIVATE 초대 차단)

- Repository

«Repository» ChatRoomRepository		
메서드명	반환타입	설명
findById(Long roomId)	Optional<ChatRoom>	채팅방 조회
findByRefTypeAndRefIdAndType(ChatRoomRefType refType, Long refId, ChatRoomType type)	Optional<ChatRoom>	PRIVATE 채팅방 먹등 검증용 (거래/상점 1:1)
existsByRefTypeAndRefIdAndType(ChatRoomRefType refType, Long refId, ChatRoomType type)	boolean	PRIVATE 채팅방 존재 여부
countActiveParticipants(Long roomId)	Long	ACTIVE 참여자 수 (GROUP 전체 퇴장 판정용)
updateLastMessageAt(Long roomId, LocalDateTime sentAt)	int	@Modifying / 메시지 발송 시lastMessageAt 갱신(목록 정렬용)

«Repository» ChatRoomRepositoryCustom(QueryDSL)		
메서드명	반환타입	설명
findMyRoomsWithUnreadCount(Long accountId, Pageable pageable)	Page<ChatRoomDto>	내 채팅방 목록 (ChatParticipant JOIN + unreadCount 서브쿼리, lastMessageAt 정렬)
searchRoomsByAdmin(ChatRoomSearchDto, Pageable pageable)	Page<ChatRoom>	관리자용 채팅방 검색 (타입/기간/생성자 필터)

«Repository» ChatParticipantRepository

메서드명	반환타입	설명
findByChatRoomIdAndAccountId(Long roomId, Long accountId)	Optional<ChatParticipant>	본인 참여 여부 검증
existsByChatRoomIdAndAccountIdAndStatus(Long roomId, Long accountId, ParticipantStatus status)	boolean	ACTIVE 참여자 검증 (Helper용)
findAllByChatRoomId(Long roomId)	List<ChatParticipant>	채팅방 참여자 목록
findAllByChatRoomIdAndStatus(Long roomId, ParticipantStatus status)	List<ChatParticipant>	ACTIVE 참여자만 조회 (브로드캐스트 대상)
findAllByAccountIdAndStatus(Long accountId, ParticipantStatus status, Pageable pageable)	Page<ChatParticipant>	사용자의 ACTIVE 채팅방 목록
countByChatRoomIdAndStatus(Long roomId, ParticipantStatus status)	Long	ACTIVE 참여자 수 (GROUP 전체 퇴장 판정)
deleteAllByChatRoomId(Long roomId)	void	채팅방 강제 삭제 시 CASCADE

«Repository» ChatMessageRepository

메서드명	반환타입	설명
findById(Long messageId)	Optional<ChatMessage>	메시지 조회
findByIdAndAccountId(Long messageId, Long accountId)	Optional<ChatMessage>	본인 메시지 검증용
findAllByChatRoomIdOrderBySentAtDesc(Long roomId, Pageable pageable)	Page<ChatMessage>	채팅방 메시지 목록 (최신순)
findAllByChatRoomIdAndSentAtBefore(Long roomId, LocalDateTime cursor, Pageable pageable)	List<ChatMessage>	커서 페이징 (과거 메시지 로드)
countByChatRoomIdAndSentAtAfter(Long roomId, LocalDateTime lastReadTime)	Long	채팅방별 안 읽은 메시지 수
findFirstByChatRoomIdOrderBySentAtDesc(Long roomId)	Optional<ChatMessage>	마지막 메시지 (목록 화면 미리보기)
deleteAllByChatRoomId(Long roomId)	void	채팅방 강제 삭제 시 CASCADE

5.7 공통 도메인

- Entity

«MappedSuperclass» ImageBase			
속성명	타입	제약조건	설명
imageUrl	String	NOT NULL	S3 이미지 URL
displayOrder	int	NOT NULL, DEFAULT 0	표시 순서
isThumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» Category			
속성명	타입	제약조건	설명
categoryId	Long	PK, NOT NULL	고유 식별자
type	CategoryType	NOT NULL	STORE / USED / COMMUNITY
parentId	Long	FK, nullable	상위 카테고리 (대>중>소, 최상위는 null)
name	String	NOT NULL	카테고리명
displayOrder	int	NOT NULL, DEFAULT 0	같은 부모 내 표시 순서
depth	int	NOT NULL, DEFAULT 1	1=대분류, 2=중분류, 3=소분류 (최대 3)
isActive	boolean	NOT NULL, DEFAULT true	노출 여부 (Soft Delete 기본값)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» Favorite			
속성명	타입	제약조건	설명
favoriteId	Long	PK, NOT NULL	고유 식별자
accountId	Long	FK, NOT NULL	찜한 사용자
refType	FavoriteRefType	ENUM	Polymorphic 타입 (STORE / USED_PRODUCT)
refId	Long	NOT NULL	Polymorphic 참조 ID (FK 아님, Service에서 검증)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» Notification			
속성명	타입	제약조건	설명
notificationId	Long	PK, NOT NULL	고유 식별자
accountId	Long	FK, NOT NULL	알림 수신자
type	NotificationType	NOT NULL	알림 종류 ENUM
title	String	NOT NULL	알림 제목 (푸시 표시용)
content	String	NOT NULL	알림 본문
refType	NotificationRefType	ENUM	Polymorphic 참조 타입
refId	Long	nullable	Polymorphic 참조 ID (FK 아님, Service에서 검증)
linkUrl	String	nullable	클라이언트 딥링크 URL
isRead	boolean	NOT NULL, DEFAULT false	읽음 여부
readAt	LocalDateTime	nullable	읽은 시각
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» NotificationSettings			
속성명	타입	제약조건	설명
notificationsettingsId	Long	PK, NOT NULL	고유 식별자
accountId	Long	FK, NOT NULL, UNIQUE	사용자
orderEnabled	boolean	NOT NULL, DEFAULT true	주문/결제 알림 (필수)
reservationEnabled	boolean	NOT NULL, DEFAULT true	예약 알림 (필수)
chatEnabled	boolean	NOT NULL, DEFAULT true	채팅 알림
communityEnabled	boolean	NOT NULL, DEFAULT true	커뮤니티 알림 (댓글/대댓글/좋아요)
storeReviewEnabled	boolean	NOT NULL, DEFAULT true	상점/리뷰 알림
usedProductEnabled	boolean	NOT NULL, DEFAULT true	중고거래 알림
systemEnabled	boolean	NOT NULL, DEFAULT true	시스템 공지 (필수)
marketingEnabled	boolean	NOT NULL, DEFAULT false	마케팅/이벤트 알림 (명시적 수신 동의 필요)
marketingAgreedAt	LocalDateTime	nullable	마케팅 수신 동의 시각 (법적 증빙용)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» Inquiry			
속성명	타입	제약조건	설명
inquiryId	Long	PK, NOT NULL	고유 식별자
accountId	Long	FK, NOT NULL	문의 작성자
inquiryType	InquiryType	NOT NULL	STORE / SYSTEM (대상 구분)
targetId	Long	nullable	대상 ID (STORE면 storeId, SYSTEM이면 null)
category	InquiryCategory	NOT NULL	문의 카테고리 ENUM
title	String	NOT NULL	문의 제목
content	String	NOT NULL, TEXT	문의 본문
status	InquiryStatus	NOT NULL, DEFAULT PENDING	PENDING / ANSWERED / CLOSED
answeredAt	LocalDateTime	nullable	답변 시각 (status=ANSWERED 전이 시 기록)
closedAt	LocalDateTime	nullable	종료 시각 (status=CLOSED 전이 시 기록)
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» InquiryReply			
속성명	타입	제약조건	설명
inquiryreplyId	Long	PK, NOT NULL	고유 식별자
inquiryId	Long	FK, NOT NULL, UNIQUE	문의 (1:1 보장)
accountId	Long	FK, NOT NULL	답변자 (사장 또는 관리자)
content	String	NOT NULL, TEXT	답변 본문
createdAt	LocalDateTime	NOT NULL	BaseEntity 상속
modifiedAt	LocalDateTime	NOT NULL	BaseEntity 상속

«Entity» InquiryImage			
속성명	타입	제약조건	설명
inquiryimageId	Long	PK, NOT NULL	이미지 고유 식별자(ImageBase 상속)
inquiryId	Long	FK,, NOT NULL	연관 문의
imageUrl	String	NOT NULL	이미지 URL(ImageBase 상속)
displayOrder	int	NOT NULL, DEFAULT 0	표시 순서(ImageBase 상속)
isThumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부(ImageBase 상속)
createdAt	LocalDateTime	NOT NULL	ImageBase 상속
modifiedAt	LocalDateTime	NOT NULL	ImageBase 상속

«MappedSuperclass» ImageBase (도메인 메서드)		
메서드명	반환타입	설명
markAsThumbnail()	void	대표 이미지로 설정
unmarkThumbnail()	void	대표 이미지 해제
updateDisplayOrder(int)	void	표시 순서 변경
«Entity» Category (도메인 메서드)		
메서드명	반환타입	설명
activate()	void	노출 활성화
deactivate()	void	노출 비활성화 (Soft Delete)
isRoot()	boolean	최상위 여부 (parentId == null)
canHaveChild()	boolean	자식 추가 가능 여부 (depth < 3)
updateDisplayOrder(int)	void	표시 순서 변경
«Entity» Favorite (도메인 메서드)		
메서드명	반환타입	설명
isOwnedBy(accountId)	boolean	본인 찜 여부 확인
matches(refType, refId)	boolean	타입+ID 일치 여부 (토글 검증용)

«Entity» Notification (도메인 메서드)		
메서드명	반환타입	설명
markAsRead()	void	읽음 처리 (isRead=true, readAt 기록)
isOwnedBy(accountId)	boolean	본인 알림 여부
isUnread()	boolean	안 읽음 여부

«Entity» NotificationSettings (도메인 메서드)		
메서드명	반환타입	설명
isAllowed(NotificationType type)	boolean	해당 타입 알림 수신 동의 여부 검증
agreeToMarketing()	void	마케팅 수신 동의 (marketingEnabled=true, marketingAgreedAt 기록)
disagreeToMarketing()	void	마케팅 수신 거부 (marketingEnabled=false, marketingAgreedAt=null)
toggleChatEnabled()	void	채팅 알림 토글
toggleCommunityEnabled()	void	커뮤니티 알림 토글
toggleStoreReviewEnabled()	void	상점/리뷰 알림 토글
toggleUsedProductEnabled()	void	중고거래 알림 토글

«Entity» Inquiry (도메인 메서드)		
메서드명	반환타입	설명
markAsAnswered()	void	PENDING → ANSWERED 전이 (answeredAt 기록)
markAsClosed()	void	ANSWERED → CLOSED 전이 (closedAt 기록)
isOwnedBy(accountId)	boolean	본인 문의 여부
isAnswerable()	boolean	답변 가능 상태 (PENDING)
isClosable()	boolean	종료 가능 상태 (ANSWERED)
isStoreInquiry()	boolean	STORE 타입 여부
isSystemInquiry()	boolean	SYSTEM 타입 여부

«Entity» InquiryReply (도메인 메서드)		
메서드명	반환타입	설명
isOwnedBy(accountId)	boolean	답변자 본인 여부
updateContent(String)	void	답변 본문 수정

- **Controller**

«Controller» CategoryController			
MAP 방식	API명	반환타입	설명
GET	/categories/{type}	ResponseEntity<List<CategoryTreeResponseDto>>	타입별 카테고리 트리 (활성만, 캐싱 응답)
GET	/categories/{type}/roots	ResponseEntity<List<CategoryResponseDto>>	최상위 카테고리 목록
GET	/categories/{categoryId}/children	ResponseEntity<List<CategoryResponseDto>>	직속 자식 카테고리

«Controller» AdminCategoryController			
MAP 방식	API명	반환타입	설명
POST	/admin/categories	ResponseEntity<CategoryResponseDto>	카테고리 생성 (depth 검증)
PATCH	/admin/categories/{categoryId}	ResponseEntity<CategoryResponseDto>	수정 (이름 / 표시순서)
DELETE	/admin/categories/{categoryId}	ResponseEntity<CommonResponseDto>	삭제 (기본: Soft Delete, 참조 0개일 때만 Hard Delete 가능)
PATCH	/admin/categories/{categoryId}/activate	ResponseEntity<CommonResponseDto>	활성화
PATCH	/admin/categories/{categoryId}/deactivate	ResponseEntity<CommonResponseDto>	비활성화
GET	/admin/categories	ResponseEntity<List<CategoryResponseDto>>	전체 조회 (비활성 포함)

«Controller» FavoriteController			
MAP 방식	API명	반환타입	설명
POST	/favorites	ResponseEntity<FavoriteResponseDto>	찜 등록/해제 토글 (refType + refId)
DELETE	/favorites/{favoriteId}	ResponseEntity<CommonResponseDto>	찜 삭제
GET	/favorites/me	ResponseEntity<Page<FavoriteResponseDto>>	내 찜 목록 (refType 필터, 최신순)
GET	/favorites/me/{refType}	ResponseEntity<Page<XxxResponseDto>>	타입별 찜 목록 (Store/UsedProduct 정보 조회)
GET	/favorites/check	ResponseEntity<FavoriteCheckResponseDto>	특정 대상 찜 여부 조회 (?refType=STORE&refId=123)
GET	/favorites/check/batch	ResponseEntity<List<FavoriteCheckResponseDto>>	다수 대상 찜 여부 일괄 조회 (목록 화면용 N+1 방지)

«Controller» AdminFavoriteController			
MAP 방식	API명	반환타입	설명
GET	/admin/favorites/stats	ResponseEntity<List<FavoriteStatResponseDto>>	인기 항목 통계 (refType별 상위 N개, 운영지표)

«Controller» NotificationController			
MAP 방식	API명	반환타입	설명
GET	/notifications	ResponseEntity<Page<NotificationResponseDto>>	내 알림 목록 (최신순, 페이지)
GET	/notifications/unread/count	ResponseEntity<UnreadCountResponseDto>	안 읽은 알림 수 (배지)
GET	/notifications/unread	ResponseEntity<Page<UnreadNotificationResponseDto>>	안 읽은 알림
PATCH	/notifications/{notificationId}/read	ResponseEntity<CommonResponseDto>	알림 읽음 처리
PATCH	/notifications/read/all	ResponseEntity<CommonResponseDto>	모두 읽음 처리
DELETE	/notifications/{notificationId}	ResponseEntity<CommonResponseDto>	알림 삭제
DELETE	/notifications/all	ResponseEntity<CommonResponseDto>	전체 삭제 (사용자가 알림함 비우기)

«Controller» NotificationSettingsController			
MAP 방식	API명	반환타입	설명
GET	/notifications/settings	ResponseEntity<NotificationSettingsResponseDto>	내 알림 수신 설정
PATCH	/notifications/settings	ResponseEntity<NotificationSettingsResponseDto>	카테고리별 ON/OFF 변경

«Controller» AdminNotificationController			
MAP 방식	API명	반환타입	설명
POST	/admin/notifications/system	ResponseEntity<CommonResponseDto>	시스템 공지 발송 (전체 또는 필터 대상)
POST	/admin/notifications/event	ResponseEntity<CommonResponseDto>	이벤트 알림 발송 (지역/연령대 등 타겟팅)
GET	/admin/notifications	ResponseEntity<Page<NotificationResponseDto>>	발송 이력 조회 (감사용)

«Controller» InquiryController			
MAP 방식	API명	반환타입	설명
POST	/inquiries	ResponseEntity<InquiryResponseDto>	문의 등록 (inquiryType + targetId 지정)
GET	/inquiries/me	ResponseEntity<Page<InquiryResponseDto>>	내 문의 목록 (status 필터)
GET	/inquiries/{inquiryId}	ResponseEntity<InquiryDetailResponseDto>	문의 상세 (답변 + 이미지 포함)
PATCH	/inquiries/{inquiryId}	ResponseEntity<InquiryResponseDto>	문의 수정 (PENDING 상태일 때만)
DELETE	/inquiries/{inquiryId}	ResponseEntity<CommonResponseDto>	문의 삭제 (PENDING 상태일 때만)
PATCH	/inquiries/{inquiryId}/close	ResponseEntity<CommonResponseDto>	문의 종료 (ANSWERED → CLOSED, 본인만)
POST	/inquiries/{inquiryId}/images	ResponseEntity<InquiryImageResponseDto>	이미지 추가
DELETE	/inquiries/{inquiryId}/images/{imageId}	ResponseEntity<CommonResponseDto>	이미지 삭제

«Controller» OwnerInquiryController			
MAP 방식	API명	반환타입	설명
GET	/owner/stores/{storeId}/inquiries	ResponseEntity<Page<InquiryResponseDto>>	본인 매장 문의 목록 (status 필터)
GET	/owner/inquiries/{inquiryId}	ResponseEntity<InquiryDetailResponseDto>	매장 문의 상세
POST	/owner/inquiries/{inquiryId}/reply	ResponseEntity<InquiryReplyResponseDto>	답변 작성 (PENDING → ANSWERED)
PATCH	/owner/inquiries/{inquiryId}/reply	ResponseEntity<InquiryReplyResponseDto>	답변 수정
DELETE	/owner/inquiries/{inquiryId}/reply	ResponseEntity<CommonResponseDto>	답변 삭제

«Controller» AdminInquiryController			
MAP 방식	API명	반환타입	설명
GET	/admin/inquiries	ResponseEntity<Page<InquiryResponseDto>>	전체 문의 조회 (type/status/category 필터)
GET	/admin/inquiries/system	ResponseEntity<Page<InquiryResponseDto>>	SYSTEM 문의 목록 (관리자 답변 대상)
GET	/admin/inquiries/{inquiryId}	ResponseEntity<InquiryDetailResponseDto>	문의 상세
POST	/admin/inquiries/{inquiryId}/reply	ResponseEntity<InquiryReplyResponseDto>	SYSTEM 문의 답변 작성
PATCH	/admin/inquiries/{inquiryId}/reply	ResponseEntity<InquiryReplyResponseDto>	답변 수정
DELETE	/admin/inquiries/{inquiryId}/reply	ResponseEntity<CommonResponseDto>	답변 삭제
DELETE	/admin/inquiries/{inquiryId}	ResponseEntity<CommonResponseDto>	문의 강제 삭제
PATCH	/admin/inquiries/{inquiryId}/close	ResponseEntity<CommonResponseDto>	문의 강제 종료

- Service

«Service» ImageService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
generatePresignedUrl(PresignedUrlRequestDto)	PresignedUrlResponseDto	-	-	기본 업로드 방식 — S3 Presigned URL 발급 (도메인별 디렉토리 분기)
uploadImage(MultipartFile, ImageType)	ImageUploadResponseDto	-	S3Upload	서버 경유 업로드 (예외 케이스 — 관리자 업로드 등)
deleteImage(String imageUrl)	void	-	S3Delete	S3에서 이미지 물리 삭제
validateImageFile(MultipartFile)	void	-	-	확장자(jpg/png/webp) / 크기(10MB) / MIME 검증
validateImageUrl(String imageUrl)	boolean	-	-	Presigned URL로 업로드된 객체 존재 검증 (DB 저장 전)

«Service» CategoryService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getCategoryTree(CategoryType type)	List<CategoryTreeResponseDto>	readOnly	RedisCache	타입별 활성 카테고리 트리 (Key: category:tree:{type}, TTL 1h)
getRootCategories(CategoryType type)	List<CategoryResponseDto>	readOnly	RedisCache	최상위 활성 카테고리
getChildCategories(Long categoryId)	List<CategoryResponseDto>	readOnly	-	직속 자식 카테고리
getCategoryById(Long categoryId)	Category	readOnly	-	카테고리 조회 (다른 도메인 검증용)
validateCategoryType(Long categoryId, CategoryType expected)	void	readOnly	-	타입 일치 검증
validateCategoryActive(categoryId)	void	readOnly	-	활성 카테고리인지 검증 (게시글/상점 등록 시 사용)

«Service» **AdminCategoryService**

메서드명	반환타입	트랜잭션	정책 / AOP	설명
createCategory(CategoryCreateRequestDto)	CategoryResponseDto	REQUIRED	Admin + CacheEvict	생성 (parent 검증, depth ≤ 3, 중복 이름 검증)
updateCategory(Long categoryId, CategoryUpdateRequestDto)	CategoryResponseDto	REQUIRED	Admin + CacheEvict	수정 (이름 / 순서)
deactivateCategory(Long categoryId)	void	REQUIRED	Admin + CacheEvict	Soft Delete (기본 삭제)
activateCategory(Long categoryId)	void	REQUIRED	Admin + CacheEvict	재활성화
hardDeleteCategory(Long categoryId)	void	REQUIRED	Admin + CacheEvict	물리 삭제 (자식 0 + 참조 0 검증, 실패 시 예외)
getAllCategories(CategoryType type)	List<CategoryResponseDto>	readOnly	Admin	비활성 포함 전체 조회

«Service» FavoriteService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
toggleFavorite(accountId, FavoriteToggleRequestDto)	FavoriteToggleResponseDto	REQUIRED	RefValidation + Event	찜 토글 (있으면 해제, 없으면 등록). refType + refId 유효성 검증 후 처리
deleteFavorite(accountId, favoriteId)	void	REQUIRED	Ownership	찜 삭제
getMyFavorites(accountId, refType, Pageable)	Page<FavoriteResponseDto>	readOnly	-	내 찜 목록 (refType 필터 옵션, 최신순)
getMyFavoritesByType(accountId, refType, Pageable)	Page<XxxResponseDto>	readOnly	-	타입별 찜 목록 + 대상 정보 JOIN (Store / UsedProduct별 분기)
checkFavorite(accountId, refType, refId)	FavoriteCheckResponseDto	readOnly	-	찜 여부 조회 (상세 화면 진입 시)
checkFavoritesBatch(accountId, refType, List<Long> refIds)	List<FavoriteCheckResponseDto>	readOnly	-	다수 대상 찜 여부 일괄 조회 (목록 화면 N+1 방지)
getFavoriteCount(refType, refId)	Long	readOnly	-	대상별 찜 수 (Store / UsedProduct 상세 화면용)
deleteAllByRefTypeAndRefId(refType, refId)	void	REQUIRED	-	대상 도메인 삭제 시 CASCADE — 다른 도메인 Service에서 호출
deleteAllByAccountId(accountId)	void	REQUIRED	Internal	회원 탈퇴 시 본인 찜 일괄 삭제 (CASCADE)

«Service» AdminFavoriteService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getFavoriteStats(refType, FavoriteStatsRequestDto)	List<FavoriteStatResponseDto>	readOnly	Admin	인기 항목 통계 (기간 + refType + 상위 N)

«Helper» FavoriteAccessHelper		
메서드명	반환타입	설명
verifyFavoriteOwnership(accountId, favoriteId)	Favorite	본인 찜 검증
validateRefTypeAndRefId(refType, refId)	void	refType별로 해당 도메인 존재 검증

«Service» NotificationService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
createNotification(NotificationCreateRequestDto)	NotificationResponseDto	REQUIRED	Internal + Event	알림 허용 여부 검증 후 Notification 저장, Redis 카운트 증가 및 푸시 이벤트 발행
createNotificationsBatch(List<NotificationCreateRequestDto>)	List<NotificationResponseDto>	REQUIRED	Internal + Event	다수 사용자 일괄 발송 (그룹 채팅, 시스템 공지 등)
getMyNotifications(accountId, Pageable)	Page<NotificationResponseDto>	readOnly	-	내 알림 목록 (최신순)
getUnreadCount(accountId)	UnreadCountResponseDto	readOnly	RedisCache	안 읽은 알림 수 (Redis 조회 → 캐시 미스 시 Db 호출 후 Redis 복구)
markAsRead(accountId, notificationId)	void	REQUIRED	Ownership + CacheEvict	읽음 처리
markAllAsRead(accountId)	void	REQUIRED	CacheEvict	모두 읽음 처리
deleteNotification(accountId, notificationId)	void	REQUIRED	Ownership + CacheEvict	삭제
deleteAllByAccountId(accountId)	void	REQUIRED	CacheEvict	전체 삭제 (회원 탈퇴 CASCADE)
deleteAllByRefTypeAndRefId(refType, refId)	void	REQUIRED	Internal	대상 도메인 삭제 시 관련 알림 일괄 삭제

«Service» NotificationSettingsService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getMySettings(accountId)	NotificationSettingsResponseDto	readOnly	-	내 알림 설정 조회
updateSettings(accountId, NotificationSettingsUpdateRequestDto)	NotificationSettingsResponseDto	REQUIRED	-	카테고리별 ON/OFF 변경
isAllowedForAccount(accountId, NotificationType type)	boolean	readOnly	Internal	발송 전 수신 동의 여부 검증

«Service» AdminNotificationService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
sendSystemNotice(SystemNoticeRequestDto)	void	REQUIRED	Admin + Async	전체 또는 필터(지역/연령) 대상 시스템 공지
sendEventNotice(EventNoticeRequestDto)	void	REQUIRED	Admin + Async	이벤트 알림 (마케팅 수신 동의자 한정)
getNotificationHistory(NotificationSearchDto, Pageable)	Page<NotificationResponseDto>	readOnly	Admin	발송 이력 조회

«Helper» NotificationAccessHelper		
메서드명	반환타입	설명
verifyNotificationOwnership(accountId, notificationId)	Notification	본인 알림 검증
buildLinkUrl(refType, refId, extraParams)	String	refType별 딥링크 URL 자동 생성

«Service» InquiryService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
registerInquiry(accountId, InquiryCreateRequestDto)	InquiryResponseDto	REQUIRED	TypeValidation	문의 등록 (inquiryType별 허용 category 검증, targetId 존재 검증)
getMyInquiries(accountId, InquirySearchDto, Pageable)	Page<InquiryResponseDto>	readOnly	-	내 문의 목록 (status 필터)
getInquiryDetail(accountId, inquiryId)	InquiryDetailResponseDto	readOnly	Ownership	본인 문의 상세 (답변 + 이미지 포함)
updateInquiry(accountId, inquiryId, InquiryUpdateRequestDto)	InquiryResponseDto	REQUIRED	Ownership + StatusGuard(PENDING만 허용)	문의 수정 (PENDING 상태만)
deleteInquiry(accountId, inquiryId)	void	REQUIRED	Ownership + StatusGuard(PENDING만 허용)	문의 삭제 (PENDING 상태만, Hard Delete + CASCADE)
closeInquiry(accountId, inquiryId)	void	REQUIRED	Ownership + StatusGuard(ANSWERED만 허용)	ANSWERED → CLOSED 전이
addInquiryImage(accountId, inquiryId, imageUrl)	InquiryImageResponseDto	REQUIRED	Ownership + ImageService	이미지 추가
deleteInquiryImage(accountId, inquiryId, imageId)	void	REQUIRED	Ownership + ImageService	이미지 삭제

«Service» OwnerInquiryService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getStoreInquiries(accountId, storeId, InquirySearchDto, Pageable)	Page<InquiryResponseDto>	readOnly	StoreOwnership	본인 매장 문의 목록
getStoreInquiryDetail(accountId, inquiryId)	InquiryDetailResponseDto	readOnly	StoreOwnership	매장 문의 상세
answerInquiry(accountId, inquiryId, InquiryReplyCreateRequestDto)	InquiryReplyResponseDto	REQUIRED	StoreOwnership + StatusGuard + Event	답변 작성 (PENDING → ANSWERED, 알림 발송)
updateAnswer(accountId, inquiryId, InquiryReplyUpdateRequestDto)	InquiryReplyResponseDto	REQUIRED	ReplyOwnership	답변 수정

«Service» AdminInquiryService				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
getAllInquiries(InquirySearchDto, Pageable)	Page<InquiryResponseDto>	readOnly	Admin	전체 문의 조회
getSystemInquiries(InquirySearchDto, Pageable)	Page<InquiryResponseDto>	readOnly	Admin	SYSTEM 문의 (관리자 답변 대상)
answerSystemInquiry(adminId, inquiryId, InquiryReplyCreateRequestDto)	InquiryReplyResponseDto	REQUIRED	Admin + StatusGuard + Event	SYSTEM 답변 작성
updateAnswer(adminId, inquiryId, InquiryReplyUpdateRequestDto)	InquiryReplyResponseDto	REQUIRED	Admin	답변 수정
forceDeleteInquiry(inquiryId)	void	REQUIRED	Admin	문의 강제 삭제
forceCloseInquiry(inquiryId)	void	REQUIRED	Admin	문의 강제 종료
autoCloseExpiredInquiries()	void	-	@Scheduled(cron = "0 0 1 * * *")	ANSWERED 후 7일 경과 문의 자동 CLOSED 처리 (배치)

«Helper» **InquiryAccessHelper**

메서드명	반환타입	설명
verifyInquiryOwnership(accountId, inquiryId)	Inquiry	본인 문의 검증
verifyStoreInquiryAccess(accountId, inquiryId)	Inquiry	사장이 본인 매장 문의 접근 검증 (Inquiry.targetId == owner.storeId)
verifyReplyOwnership(accountId, inquiryReplyId)	InquiryReply	답변자 본인 검증
validateCategoryByType(InquiryType, InquiryCategory)	void	inquiryType별 허용 category 검증
validateTarget(InquiryType, Long targetId)	void	targetId 존재 검증 (STORE면 StoreRepository.existsById)
guardEditableStatus(Inquiry, allowed: List<InquiryStatus>)	void	상태 전이 검증 (예: PENDING만 수정 가능)

- Interface

«interface» PushAdapter		
메서드명	반환타입	설명
send(PushMessage message)	PushResult	단건 푸시 발송
sendBatch(List<PushMessage> messages)	List<PushResult>	배치 푸시 발송 (시스템 공지용, 최대 500건/요청)

- **Component**

«Component» NotificationCleanupScheduler

메서드명	반환타입	트랜잭션	정책 / AOP	설명
cleanupOldNotifications()	void	-	@Scheduled(cron = "0 0 3 * * *")	매일 새벽 3시 — 6개월 이전 알림 일괄 삭제(NotificationService.deleteOldNotifications 호출)
recalculateUnreadCounts()	void	-	@Scheduled(fixedRate = 300000)	5분마다 — Redis unread 캐시 ↔ DB 정합성 보정

«Component» InquiryAutoCloseScheduler

메서드명	반환타입	트랜잭션	정책 / AOP	설명
autoCloseExpired()	void	-	@Scheduled(cron = "0 0 1 * * *")	매일 새벽 1시 — AdminInquiryService.autoCloseExpiredInquiries() 호출 (ANSWERED 상태 + 7일 경과 → CLOSED 자동 전이)

«Component» ChatNotificationEventListener

메서드명	반환타입	트랜잭션	정책 / AOP	설명
onMessageSent(ChatMessageSentEvent)	void	REQUIRED	@TransactionalEventListener(AFTER_COMMIT) + @Async	ACTIVE 참여자(발신자 제외)의 unread:account:{accountId} INCR + 방별 INCR + 비활성 참여자 푸시 발송 이벤트 발행
onParticipantInvited(ParticipantInvitedEvent)	void	REQUIRED	@TransactionalEventListener(AFTER_COMMIT) + @Async	GROUP 채팅방 입장 SYSTEM 메시지 발행
onParticipantLeft(ParticipantLeftEvent)	void	REQUIRED	@TransactionalEventListener(AFTER_COMMIT) + @Async	퇴장 SYSTEM 메시지 발행 + GROUP 전체 퇴장 시 isActive=false

«Component» ChatUnreadReconcileScheduler				
메서드명	반환타입	트랜잭션	정책 / AOP	설명
recalculateChatUnreadCounts()	void	-	@Scheduled(fixedRate = 300000)	5분마다 — ChatMessageRepositoryCustom.countUnreadByAccountIdFromDb 호출하여 Redis 캐시 ↔ DB 비교, 불일치 시 보정

«Component» FcmPushAdapter (구현체)		
메서드명	반환타입	설명
send(PushMessage)	PushResult	FCM API 호출 (Android/iOS/Web 통합)
sendBatch(List<PushMessage>)	List<PushResult>	FCM 멀티캐스트 API 호출
handleInvalidToken(String fcmToken)	void	만료/무효 토큰 처리 (Account.fcmToken 무효화)

- Repository

«Repository» ImageRepository (도메인별 Image Repository에 공통 쿼리 패턴)		
메서드명	반환타입	설명
findById(Long imageId)	Optional<XxxImage>	이미지 조회
findAllByXxxId(Long parentId)	List<XxxImage>	부모 엔티티 이미지 목록 (displayOrder 정렬)
findThumbnailByXxxId(Long parentId)	Optional<XxxImage>	대표 이미지 조회 (목록 화면용)
countByXxxId(Long parentId)	Long	이미지 개수 (업로드 제한 검증)
findFirstByXxxIdOrderByDisplayOrderAsc(Long parentId)	Optional<XxxImage>	대표 자동 승격용 (대표 삭제 시)
deleteAllByXxxId(Long parentId)	void	부모 삭제 시 CASCADE

«Repository» CategoryRepository		
메서드명	반환타입	설명
findById(Long categoryId)	Optional<Category>	카테고리 조회
findAllByTypeAndIsActiveTrue(CategoryType type)	List<Category>	타입별 활성 카테고리 (트리 구성용)
findAllByType(CategoryType type)	List<Category>	타입별 전체 (관리자)
findAllByTypeAndParentIdsIsNullAndIsActiveTrue(CategoryType type)	List<Category>	타입별 최상위 활성 @Cacheable(value = "categoryTree", key = "#type")
findAllByParentIdAndIsActiveTrue(Long parentId)	List<Category>	활성 자식
existsByParentId(Long categoryId)	boolean	자식 존재 여부 (Hard Delete 검증)
existsByTypeAndParentIdAndName(CategoryType type, Long parentId, String name)	boolean	중복 이름 검증
countByType(CategoryType type)	Long	타입별 카테고리 수

«Repository» FavoriteRepository		
메서드명	반환타입	설명
findById(Long favoriteId)	Optional<Favorite>	찜 조회
findByIdAndAccountId(Long favoriteId, Long accountId)	Optional<Favorite>	본인 찜 검증용
findByAccountIdAndRefTypeAndRefId(Long accountId, FavoriteRefType refType, Long refId)	Optional<Favorite>	토글 시 기존 찜 조회
existsByAccountIdAndRefTypeAndRefId(Long accountId, FavoriteRefType refType, Long refId)	boolean	찜 여부 확인
findAllByAccountId(Long accountId, Pageable pageable)	Page<Favorite>	내 찜 전체 (refType 무관)
findAllByAccountIdAndRefType(Long accountId, FavoriteRefType refType, Pageable pageable)	Page<Favorite>	타입별 내 찜
findAllByAccountIdAndRefType(Long accountId, FavoriteRefType refType, Pageable pageable)	Long	대상별 찜 수 (상세 화면 표시용)
deleteByAccountIdAndRefTypeAndRefId(Long accountId, FavoriteRefType refType, Long refId)	void	토글 해제
deleteAllByRefTypeAndRefId(FavoriteRefType refType, Long refId)	void	대상 도메인 삭제 시 CASCADE
deleteAllByAccountId(Long accountId)	void	회원 탈퇴 시 CASCADE

«Repository» FavoriteRepositoryCustom (QueryDSL)		
메서드명	반환타입	설명
findFavoriteRefIdsByAccountIdAndType(Long accountId, FavoriteRefType refType, List<Long> refIds)	Set<Long>	다수 대상 중 사용자가 찜한 refId Set (목록 화면 배치 조회용)
findFavoriteStats(FavoriteRefType refType, LocalDateTime from, LocalDateTime to, int limit)	List<FavoriteStatProjection>	인기 항목 통계 (Group By refId + 기간 필터)

«Repository» NotificationRepository		
메서드명	반환타입	설명
findById(Long notificationId)	Optional<Notification>	알림 조회
findByIdAndAccountId(Long notificationId, Long accountId)	Optional<Notification>	본인 알림 검증용
findAllByAccountIdOrderByCreatedAtDesc(Long accountId, Pageable pageable)	Page<Notification>	내 알림 목록 (최신순)
countByAccountIdAndIsReadFalse(Long accountId)	Long	안 읽은 알림 수 (캐시 미스 시)
markAllAsReadByAccountId(@Param("accountId") Long accountId, @Param("now") LocalDateTime now)	int	@Modifying / 사용자의 모든 알림 일괄 읽음 처리 (isRead=true, readAt=now)
deleteAllByAccountId(Long accountId)	void	사용자 알림 일괄 삭제 (회원 탈퇴 CASCADE)
deleteAllByRefTypeAndRefId(NotificationRefType refType, Long refId)	void	대상 도메인 삭제 시 CASCADE
deleteOldNotifications(LocalDateTime threshold)	int	오래된 알림 정리 배치용 (예: 6개월 이전)

«Repository» NotificationRepositoryCustom (QueryDSL)		
메서드명	반환타입	설명
searchAdminNotifications(NotificationSearchDto, Pageable)	Page<Notification>	관리자용 발송 이력 검색 (type/기간/대상자 필터)
countSentByTypeAndPeriod(NotificationType type, LocalDateTime from, LocalDateTime to)	Long	통계용 — 기간별 type 발송 수
findEnabledNotificationsByAccountId(Long accountId, Pageable pageable)	Page<Notification>	NotificationSettings JOIN으로 수신 거부 카테고리 제외 조회

«Repository» NotificationSettingsRepository		
메서드명	반환타입	설명
findByAccountId(Long accountId)	Optional<NotificationSettings>	사용자 설정 조회 (없으면 기본값 사용)
existsByAccountId(Long accountId)	boolean	설정 존재 여부

«Repository» InquiryRepository		
메서드명	반환타입	설명
findById(Long inquiryId)	Optional<Inquiry>	문의 조회
findByIdAndAccountId(Long inquiryId, Long accountId)	Optional<Inquiry>	본인 문의 검증용
findAllByAccountId(Long accountId, Pageable pageable)	Page<Inquiry>	내 문의 목록
findAllByAccountIdAndStatus(Long accountId, InquiryStatus status, Pageable pageable)	Page<Inquiry>	상태별 내 문의
findAllByInquiryTypeAndTargetId(InquiryType type, Long targetId, Pageable pageable)	Page<Inquiry>	매장별 문의 (STORE)
findAllByInquiryTypeAndTargetIdAndStatus(InquiryType type, Long targetId, InquiryStatus status, Pageable pageable)	Page<Inquiry>	매장별 + 상태별 문의
findAllByInquiryType(InquiryType type, Pageable pageable)	Page<Inquiry>	시스템 문의 목록 (관리자용)
countByInquiryTypeAndTargetIdAndStatus(InquiryType type, Long targetId, InquiryStatus status)	Long	매장 미답변 문의 수 (사장 대시보드)

«Repository» InquiryRepositoryCustom (QueryDSL)		
메서드명	반환타입	설명
searchInquiries(InquirySearchDto, Pageable)	Page<Inquiry>	복합 검색 (type/category/status/기간/키워드)
findExpiredAnswered(LocalDateTime threshold)	List<Inquiry>	자동 종료 배치용 — ANSWERED 후 N일 경과 문의

«Repository» InquiryReplyRepository		
메서드명	반환타입	설명
findById(Long inquiryreplyId)	Optional<InquiryReply>	답변 조회
findByInquiryId(Long inquiryId)	Optional<InquiryReply>	문의의 답변 조회(1:1)
existsByInquiryId(Long inquiryId)	boolean	답변 존재 여부(중복 답변 방지)
deleteByInquiryId(Long inquiryId)	void	문의 삭제 시CASCADE

«Repository» ReportRepository		
메서드명	반환타입	설명
findById(Long reportId)	Optional<Report>	신고 조회
findByIdAndAccountId(Long reportId, Long accountId)	Page<Report>	본인 신고 검증용
findAllByAccountId(Long accountId, Pageable pageable)	List<Report>	내 신고 목록
findAllByRefTypeAndRefId(ReportRefType refType, Long refId)	List<Report>	대상별 신고 목록
findAllByStatus(ReportStatus status, Pageable pageable)	Page<Report>	상태별 신고 목록(관리자용)
countByRefTypeAndRefIdAndStatus(ReportRefType refType, Long refId, ReportStatus status)	Long	중복/누적 신고 검증
existsByAccountIdAndRefTypeAndRefId(Long accountId, ReportRefType refType, Long refId)	boolean	동일 대상 중복 신고 방지
deleteAllByRefTypeAndRefId(ReportRefType refType, Long refId)	void	대상 도메인 삭제 시CASCADE

«Repository» ReportRepositoryCustom (QueryDSL)		
메서드명	반환타입	설명
searchReports(ReportSearchDto condition, Pageable pageable)	Page<Report>	관리자 복합 필터(refType/status/기간/키워드)
findTopReportedTargets(ReportRefType refType, LocalDateTime from, int limit)	List<ReportStatProjection>	신고 누적 통계(관리자 대시보드)

5.8 ENUM

- Account

«Enum» AccountRole		«Enum» AccountStatus		«Enum» OAuthProvider		«Enum» ApprovalStatus	
ENUM 값	설명	ENUM 값	설명	ENUM 값	설명	ENUM 값	설명
ROLE_USER	일반 사용자	ACTIVE	정상 상태	LOCAL	일반 로그인	PENDING	승인 대기
ROLE_OWNER	사장 회원	ROLE_OWNER	이용 정지	KAKAO	카카오 로그인	APPROVED	승인 완료
ROLE_ADMIN	관리자	SUSPENDED	탈퇴 (30일 유예 상태)	NAVER	네이버 로그인	REJECTED	승인 거절

«Enum» AuditResult	
ENUM 값	설명
SUCCESS	작업 성공
FAILURE	작업 실패

«Enum» AuditActionType		«Enum» AuditActionType		«Enum» AuditActionType	
ENUM 값	설명	ENUM 값	설명	ENUM 값	설명
ACCOUNT_SUSPEND	계정 정지	CATEGORY_CREATE	카테고리 생성	INQUIRY_FORCE_DELETE	문의 삭제
ACCOUNT_ACTIVATE	계정 활성화	CATEGORY_UPDATE	카테고리 수정	INQUIRY_FORCE_CLOSE	문의 강제 종료
ACCOUNT_FORCE_DELETE	계정 강제 삭제	CATEGORY_DEACTIVATE	카테고리 비활성화	NOTIFICATION_SYSTEM_SEND	시스템 알림 발송
ACCOUNT_WITHDRAWAL_CANCEL	탈퇴 취소	CATEGORY_HARD_DELETE	카테고리 완전 삭제	NOTIFICATION_EVENT_SEND	이벤트 알림 발송

«Enum» AuditActionType	
ENUM 값	설명
REVIEW_FORCE_DELETE	리뷰 강제 삭제
USED_PRODUCT_FORCE_DELETE	중고상품 삭제
USED_REVIEW_FORCE_DELETE	중고 리뷰 삭제
POST_FORCE_DELETE	게시글 삭제
COMMENT_FORCE_DELETE	댓글 삭제
REPLY_FORCE_DELETE	대댓글 삭제

«Enum» AuditActionType		«Enum» AuditActionType	
ENUM 값	설명	ENUM 값	설명
CHAT_MESSAGE_FORCE_DELETE	채팅 메시지 삭제	STORE_SUSPEND	상점 정지
CHAT_ROOM_FORCE_DEACTIVATE	채팅방 비활성화	STORE_ACTIVATE	상점 활성화

• Store

«Enum» StoreStatus		«Enum» NoticeType	
ENUM 값	설명	ENUM 값	설명
OPEN	영업중	GENERAL	일반 공지
CLOSED	영업 종료	HOLIDAY	휴무 공지
TEMP_CLOSED	일시 휴무	EVENT	이벤트 공지

«Enum» ProductStatus		«Enum» EventStatus		«Enum» OptionSelectionType	
ENUM 값	설명	ENUM 값	설명	ENUM 값	설명
ACTIVE	판매중	ACTIVE	진행중	SINGLE	단일 선택
SOLD_OUT	품절	ENDED	이벤트 종료	MULTIPLE	다중 선택

• Order / Payment

«Enum» OrderStatus		«Enum» PaymentStatus	
ENUM 값	설명	ENUM 값	설명
PENDING	결제 대기	PENDING	결제 대기
PAID	결제 완료	PAID	결제 완료
FAILED	결제 실패	FAILED	결제 실패
CANCELLED	주문 취소	CANCELLED	결제 취소
REFUNDED	환불 완료	REFUNDED	환불 완료

«Enum» PaymentStatus		«Enum» ReservationStatus	
ENUM 값	설명	ENUM 값	설명
CARD	신용/체크카드	PENDING	예약 대기
KAKAOPAY	카카오페이	CONFIRMED	예약 확정
NAVERPAY	네이버페이	EXPIRED	예약 만료
TOSS	토스	CANCELLED	예약 취소

«Enum» ReservationType	
ENUM 값	설명
VISIT	매장 방문 예약
PRODUCT	상품 예약

• UsedProduct

«Enum» UsedProductStatus	
ENUM 값	설명
ON_SALE	판매중
RESERVED	예약됨
SOLD	판매 완료

• Chat

«Enum» ChatRoomRefType		«Enum» ChatRoomType		«Enum» MessageType		«Enum» ParticipantStatus	
ENUM 값	설명	ENUM 값	설명	ENUM 값	설명	ENUM 값	설명
TRADE	중고거래	PRIVATE	1:1 채팅	TEXT	텍스트 메시지	ACTIVE	참여중
STORE	상점 문의	GROUP	일반 그룹	IMAGE	이미지 메시지	LEFT	채팅방 퇴장
COMMUNITY	커뮤니티	GROUP_STREET	동네 단체 채팅	SYSTEM	시스템 메시지		
NONE	일반 채팅						

• 공통 도메인

«Enum» NotificationType	
ENUM 값	설명
ORDER_CREATED	주문 생성
ORDER_CONFIRMED	주문 확정
ORDER_CANCELLED	주문 취소
ORDER_REFUNDED	환불 완료
PAYMENT_COMPLETED	결제 완료
PAYMENT_FAILED	결제 실패

«Enum» NotificationType	
ENUM 값	설명
RESERVATION_REQUESTED	예약 요청
RESERVATION_CONFIRMED	예약 확정
RESERVATION_REJECTED	예약 거절
RESERVATION_CANCELLED	예약 취소
RESERVATION_EXPIRED	예약 만료

«Enum» CategoryType	
ENUM 값	설명
STORE	상점 카테고리
USED	중고거래 카테고리
COMMUNITY	커뮤니티 카테고리

«Enum» FavoriteRefType		«Enum» CategoryType		«Enum» CategoryType		«Enum» CategoryType	
ENUM 값	설명	ENUM 값	설명	ENUM 값	설명	ENUM 값	설명
STORE	상점 찜	CHAT_NEW_MESSAGE	새 메시지	OWNER_APPROVED	사장 승인 완료	SYSTEM_NOTICE	시스템 공지
USED_PRODUCT	중고상품 찜	CHAT_INVITED	채팅 초대	OWNER_REJECTED	사장 승인 거절	SYSTEM_ANNOUNCEMENT	시스템 안내

«Enum» CategoryType	
ENUM 값	설명
MARKETING_EVENT	이벤트 알림
MARKETING_PROMOTION	프로모션 알림

«Enum» CategoryType		«Enum» CategoryType		«Enum» CategoryType	
ENUM 값	설명	ENUM 값	설명	ENUM 값	설명
STORE_REVIEW_NEW	리뷰 등록	USED_PRODUCT_RESERVED	상품 예약됨	COMMUNITY_COMMENT_NEW	댓글 등록
STORE_REVIEW_REPLY	리뷰 답글	USED_PRODUCT_SOLD	상품 판매 완료	COMMUNITY_REPLY_NEW	대댓글 등록
STORE_NOTICE_NEW	공지 등록	USED_PRODUCT_REVIEW_NEW	거래 리뷰 등록	COMMUNITY_POST_LIKE	게시글 좋아요

- 기타 공통

«Enum» NotificationRefType	
ENUM 값	설명
ORDER	주문
PAYMENT	결제
RESERVATION	예약
CHAT_ROOM	채팅방
STORE	상점
STORE_REVIEW	상점 리뷰
USED_PRODUCT	중고상품
USED_PRODUCT_REVIEW	중고 리뷰
COMMUNITY_POST	게시글
COMMUNITY_COMMENT	댓글
INQUIRY	문의
OWNER_INFO	사장 정보
SYSTEM	시스템

«Enum» InquiryCategory		«Enum» ReportReason	
ENUM 값	설명	ENUM 값	설명
PRODUCT_QUESTION	상품 문의	SPAM	스팸
ORDER_ISSUE	주문 문제	INAPPROPRIATE_CONTENT	부적절한 내용
DELIVERY_ISSUE	배송 문제	HARASSMENT	괴롭힘
REFUND_REQUEST	환불 요청	FRAUD	사기
ACCOUNT_ISSUE	계정 문제	FAKE_PRODUCT	가짜 상품
PAYMENT_ISSUE	결제 문제	SEXUAL_CONTENT	음란물
BUG_REPORT	버그 신고	VIOLENCE	폭력
FEATURE_REQUEST	기능 요청	PRIVACY_VIOLATION	개인정보 침해
REPORT_ABUSE	신고	COPYRIGHT	저작권 침해
ETC	기타	ETC	기타

«Enum» ReportRefType	
ENUM 값	설명
USER	사용자
STORE	상점
STORE_REVIEW	상점 리뷰
USED_PRODUCT	중고상품
USED_PRODUCT_REVIEW	중고 리뷰
COMMUNITY_POST	게시글
COMMUNITY_COMMENT	댓글
COMMUNITY_REPLY	대댓글
CHAT_MESSAGE	채팅 메시지

«Enum» ImageType	
ENUM 값	설명
STORE	상점
PRODUCT	상품
STORE_REVIEW	상점 리뷰
USED_PRODUCT	중고상품
COMMUNITY	커뮤니티
INQUIRY	문의

«Enum» InquiryStatus	
ENUM 값	설명
PENDING	답변 대기
ANSWERED	답변 완료
CLOSED	문의 종료

«Enum» ReportStatus		«Enum» PushResult	
ENUM 값	설명	ENUM 값	설명
PENDING	처리 대기	SUCCESS	전송 성공
REVIEWING	검토중	FAILED	전송 실패
PROCESSED	처리 완료	INVALID_TOKEN	잘못된 토큰
DISMISSED	기각	RATE_LIMITED	전송 제한

«Enum» InquiryType	
ENUM 값	설명
STORE	매장 문의
SYSTEM	시스템 문의

6. 공통 설계 패턴

6.1 동시성 제어 전략 (Concurrency Control Strategy)

이름 시스템은 다수의 사용자가 동시에 동일 자원(재고, 주문, 예약 등)에 접근하는 구조를 가진다.

특히 재고 차감, 결제 처리, 검증 등과 같이 데이터 정합성이 중요한 영역에서는 오류 발생 시 비즈니스에 직접적인 영향을 미친다.

이에 따라 본 시스템은 상황에 맞는 최적의 동시성 제어를 적용하기 위해 다층적 동시성 제어 전략(Multi-layer Concurrency Control)을 도입한다.

6.1.1 동시성 제어 4계층

계층	메커니즘	사용 시나리오
1	Redis 분산 락	분산 환경 동시 진입 차단 (예약 capacity, 결제 멍등성)
2	비관적 락 @Lock(PESSIMISTIC_WRITE)	재고 차감, 결제 상태 변경, 거래 상태 전이
3	낙관적 락 @Version	평점 갱신, 카운트 갱신
4	DB 원자 UPDATE	favoriteCount, viewCount 등 단순 카운트

6.1.2 Redis 분산 락 명명 규칙

분산 환경에서 동일 자원에 대한 동시 접근을 제어하기 위해 Redis 기반 분산 락을 사용하며, 락 키는 도메인 및 목적에 따라 일관된 규칙으로 정의한다.

Key Pattern	TTL	설명
order:product:{productId}	3s	주문 시 재고 차감 (단일 상품)
order:products:{1-3-5}	3s	다중 상품 (productIds 오름차순 정렬 후 결합)
payment:order:{orderId}	10s	결제 처리 시 중복 방지
payment:webhook:{portOnePaymentId}	30s	Webhook 중복 수신 방지
reservation:{storeId}:{yyyy-MM-dd-HH-mm}	3s	매장 방문 예약 (분 단위)
chat:private:{refType}:{refId}	5s	1:1 채팅방 멍등 생성
used-product:trade:{usedproductId}	5s	중고거래 상태 변경

- 데드락 방지 전략

다중 상품 주문 시, 여러 상품에 대한 락 획득 순서가 달라질 경우 데드락이 발생 가능하다. 이를 방지하기 위해 productId를 오름차순으로 정렬한 후 단일 키로 결합하여 사용한다.

예시: [3, 1, 5] → 1-3-5

6.1.3 비관적 락 적용 위치

데이터 충돌 가능성이 높고, 반드시 순차 처리가 필요한 경우 비관적 락을 적용한다.

Repository	메서드	용도
ProductRepository	findByIdForUpdate(productId)	재고 차감
EventProductRepository	findActiveByProductIdForUpdate	예약 확정 및 취소
OrderRepository	findByIdForUpdate(orderId)	결제 처리 및 취소
ReservationRepository	findByIdForUpdate(reservationId)	예약 확정 및 취소
UsedProductRepository	findByIdForUpdate(usedproductId)	거래 상태 변경

6.1.4 낙관적 락 적용

충돌 가능성이 상대적으로 낮고, 성능이 우선인 경우 낙관적 락을 적용한다.

- 적용 대상 Entity
 - Store
 - Product
 - EventProduct
 - Orders
 - Reservation
 - UsedProduct

각 엔티티는 JPA의 @Version 컬럼을 통해 버전 기반 충돌 검증을 수행한다.

- 재시도 처리 전략

낙관적 락 충돌 발생 시, 사용자 경험 저하를 최소화하기 위해 자동 재시도 정책을 적용한다.

- 최대 재시도 횟수: 3회
- 초기 대기 시간: 100ms
- 백오프 전략: 지수 증가 (multiplier = 2)

- 적용 방식

```
@Retryable(
    value = ObjectOptimisticLockingFailureException.class,
    maxAttempts = 3,
    backoff = @Backoff(delay = 100, multiplier = 2)
)
@Transactional
public void updateRating(Long storeId, double newRating) {
    ...
}
```

6.1.5 DB 원자 연산 (Atomic UPDATE)

조회수, 찜 수 등 단순 카운트 데이터는 락을 사용하지 않고 DB의 원자적 UPDATE 연산을 통해 처리한다.

- 적용 목적
 - 락 사용 없이 성능 확보
 - 동시성 문제 최소화

- 트랜잭션 비용 감소

- 구현 방식

```
@Modifying
@Query("UPDATE Store s SET s.favoriteCount = s.favoriteCount + 1
WHERE s.storeId = :storeId")
int incrementFavoriteCount(@Param("storeId") Long storeId);
```

- 음수 방지 처리

```
@Modifying
@Query("""
UPDATE Store s
SET s.favoriteCount = s.favoriteCount - 1
WHERE s.storeId = :storeId AND s.favoriteCount > 0
""")
int decrementFavoriteCount(@Param("storeId") Long storeId);
```

- 설계 의도

- 단순 카운트는 DB 레벨에서 직접 처리하여 성능 최적화
- 조건절을 통해 데이터 무결성 보장 (음수 방지)

6.2 이벤트 기반 아키텍처 (Event-Driven Architecture)

도메인 간 결합도를 낮추고, 알림·통계·캐시 갱신과 같은 부가 기능과 핵심 비즈니스 로직의 분리를 위해 Spring의 ApplicationEventPublisher를 활용한 이벤트 기반 아키텍처(Event-Driven Architecture)를 적용한다.
이를 통해 서비스 계층은 핵심 비즈니스 로직에 집중하고, 부가 기능은 이벤트 리스너에서 독립적으로 처리한다

6.2.1 이벤트 발행 원칙

발행 위치: 도메인 이벤트는 Service 계층에서 발행한다.

제한 사항: Entity 내부에서는 이벤트를 발행하지 않는다.

6.2.2 이벤트 명명 규칙

이벤트는 발생 시점과 의미를 명확히 표현하기 위해 다음과 같은 네이밍 규칙을 따른다.

패턴	예시	발행 시점
XxxCreatedEvent	OrderCreatedEvent	엔티티 생성 직후
XxxConfirmedEvent	ReservationConfirmedEvent	상태 전이 직후
XxxCancelledEvent	OrderCancelledEvent	취소 처리 후
XxxCompletedEvent	PaymentCompletedEvent	완료 처리 후

6.2.3 트랜잭션 동기화 전략

이벤트 리스너는 트랜잭션의 상태에 따라 실행 시점을 제어한다.

Phase	사용 사례	특징
AFTER_COMMIT	알림, 푸시, 이메일 발송	트랜잭션 커밋 후 실행 → 롤백 시 실행되지 않음
BEFORE_COMMIT	감사 로그 기록	REQUIRES_NEW와 함께 사용하여 실패 시에도 기록 유지
AFTER_COMPLETION	락 해제, 리소스 정리	성공/실패 여부와 관계없이 실행

- 필수 원칙

모든 알림, 푸시, 이메일 관련 이벤트 리스너는 반드시

@TransactionalEventListener(phase = AFTER_COMMIT)을 사용한다.

트랜잭션이 롤백된 경우, 사용자에게 잘못된 알림이 발송되는 것을 방지

6.2.4 비동기 이벤트 처리

이벤트 리스너는 @Async를 활용하여 비동기적으로 처리한다.

이를 통해 메인 트랜잭션의 응답 시간을 최소화한다.

- 적용 예시

@Async

@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)

```
public void onMessageSent(ChatMessageSentEvent event) {  
    // 1. ACTIVE 참여자 조회  
    // 2. unread:account:{accountId} INCR  
    // 3. 비활성 참여자에게 FCM 푸시  
}
```

- 설계 의도

- 이벤트 처리 로직을 비동기화하여 응답 속도 개선
- 알림, 카운트 증가 등 부가 기능을 메인 로직과 분리
- 시스템 확장 시 이벤트 소비자만 추가하면 되도록 구조 설계

6.2.5 주요 이벤트 카탈로그

시스템에서 발생하는 주요 이벤트와 처리 내용을 다음과 같이 정의한다.

이벤트	발행 위치	처리 내용
OrderCreatedEvent	OrderService.createOrder	사장에게 신규 주문 알림
OrderConfirmedEvent	OwnerOrderService.confirmOrder	사용자에게 주문 확정 알림
OrderCancelledEvent	OrderService.cancelOrder	결제 취소 및 재고 복구
PaymentCompletedEvent	PaymentService.handleWebhook	주문 상태를 PAID로 전이
ChatMessageSentEvent	ChatMessageService.sendMessage	참여자 unread 증가 및 푸시 발송
ReviewCreatedEvent	StoreReviewService.registerReview	평점 재계산 및 사장 알림
OwnerApprovedEvent	AdminAccountService.approveOwner	사장 승인 알림 및 권한 변경
PostLikedEvent	CommunityPostService.togglePostLike	게시글 작성자 알림
InquiryAnsweredEvent	OwnerInquiryService.answerInquiry	문의 답변 알림

6.3 CQRS 패턴 적용 (Command Query Responsibility Segregation)

완전한 CQRS(별도 DB 분리)는 적용하지 않지만, Service 계층에서 Command(쓰기)와 Query(읽기)를 명확히 분리한 경량 CQRS 패턴을 적용한다.

6.3.1 Command / Query 분리 원칙

구분	Command (쓰기)	Query (읽기)
메서드 Prefix	create / update / delete / register / confirm	get / find / search / list
트랜잭션	@Transactional (REQUIRED)	@Transactional(readOnly = true)
반환 타입	Entity → ResponseDto 변환	QueryDSL Projection DTO
부수 효과	상태 변경, 이벤트 발행	없음
캐싱 전략	@CacheEvict	@Cacheable
DB 라우팅	Master DB	Slave DB (Replication 시)

6.3.2 readOnly 트랜잭션의 이점

읽기 전용 트랜잭션은 다음과 같은 성능 최적화 효과를 제공한다.

- Hibernate Dirty Checking 비활성화
 - 영속성 컨텍스트 스냅샷 비교 생략 → 성능 향상
- Flush Mode MANUAL 적용
 - 자동 flush 방지 → 불필요한 UPDATE 차단
- DB Connection 라우팅 가능
 - ReplicationRoutingDataSource를 통해 Slave DB로 분산 처리
문서화 효과 → 메서드 시그니처만으로 읽기/쓰기 구분 가능

6.3.3 QueryDSL Projection 활용

목록 조회 시 Entity 전체를 조회하지 않고, 필요한 필드만 DTO로 직접 Projection하여 조회한다.

- 적용 예시

```
List<StoreListResponseDto> content = queryFactory
    .select(Projections.fields(StoreListResponseDto.class,
        s.storeId, s.name, s.rating, s.favoriteCount, s.status,
        si.imageUrl.as("thumbnailUrl")))
    .from(s)
    .leftJoin(si).on(si.store.eq(s).and(si.isThumbnail.isTrue()))
    .where(
        regionEq(cond.getRegionId()),
        categoryEq(cond.getCategoryId()),
        keywordContains(cond.getKeyword()),
        s.status.eq(StoreStatus.OPEN)
    )
    .orderBy(orderSpec(cond.getSort()))
    .offset(pageable.getOffset())
    .limit(pageable.getPageSize())
    .fetch();
```

- 설계 의도
 - 필요한 데이터만 조회하여 성능 최적화
 - Entity 로딩 비용 감소
 - API 응답 속도 개선

- N+1 문제 해결

목록 조회 시 대표 이미지를 가져오는 과정에서 발생하는

OneToMany Lazy Loading 기반 N+1 문제를 다음과 같이 해결한다.

- Projection + LEFT JOIN을 사용하여 단일 쿼리로 처리
- 추가 쿼리 발생 방지

6.4 캐싱 전략 (Caching Strategy)

응답 시간 단축, DB 부하 감소, 동시성 제어를 위해 Redis를 활용한 다목적 캐싱 전략을 적용한다.

6.4.1 캐싱 카탈로그

용도	Key 패턴	TTL	갱신 시점
카테고리 트리	category:tree:{type}	1h	관리자 변경 시 CacheEvict
상점 상세	store:detail:{storeId}	10m	상점 정보 수정 / 평점 변경
사용자 unread 알림	unread:account:{accountId}	영구	알림 생성/읽음 시 INCR/DECR
전체 채팅 unread	unread:chat:{accountId}	영구	메시지 송수신 시
방별 채팅 unread	unread:chat:{accountId}:room:{roomId}	영구	동일
게시글 조회수	view:post:{postId}	5m	Redis 누적 후 배치 동기화
상품 조회수	view:product:{productId}	5m	동일
중고상품 조회수	view:used:{usedProductId}	5m	동일
사장 대시보드	dashboard:store:{storeId}	1m	주문/리뷰 발생 시 evict

6.4.2 Cache-Aside 패턴

조회 시 캐시를 먼저 확인하고, 없을 경우 DB 조회 후 캐시에 저장한다.

- 적용 예시

```
@Cacheable(value = "categoryTree", key = "#type", unless = "#result == null")
@Transactional(readOnly = true)
public List<CategoryTreeResponseDto> getCategoryTree(CategoryType type) {
    List<Category> categories = categoryRepository.findAllByTypeAndIsActiveTrue(type);
    return buildTree(categories);
}
@CacheEvict(value = "categoryTree", key = "#dto.type")
@Transactional
public CategoryResponseDto createCategory(CategoryCreateRequestDto dto) {
    ...
}
```

6.4.3 Write-Through 패턴 (카운터 동기화)

조회수와 같이 쓰기 빈도가 높고, 약한 정합성을 허용하는 데이터는 Redis에서 먼저 처리한 후 배치로 DB에 반영한다.

- 조회 시 (Redis INCR)

```
public void incrementViewCount(Long postId) {
    redisTemplate.opsForValue().increment("view:post:" + postId);
}
```

- 배치 동기화 (Redis → DB)

```
@Scheduled(fixedRate = 300_000)
public void syncViewCounts() {
    Set<String> keys = redisTemplate.keys("view:post:*");

    for (String key : keys) {
        Long postId = extractId(key);
        Integer increment = redisTemplate.opsForValue().get(key);

        if (increment > 0) {
            communityPostRepository.addViewCount(postId, increment);
            redisTemplate.delete(key);
        }
    }
}
```

6.4.4 캐시 정합성 보정

unread 카운트는 Redis 기반으로 관리되며, 주기적으로 DB와 비교하여 정합성을 보정한다.

- 실행 주기: 5분
- 처리 컴포넌트: ChatUnreadReconcileScheduler
- 처리 방식: DB 기준 재계산 후 Redis 갱신

6.4.5 캐시 무효화 규칙

전략	설명
즉시 무효화	카테고리, 상점 정보 변경 시 즉시 CacheEvict
TTL 기반 만료	통계, 대시보드 등 일부 stale 허용
부분 무효화	특정 사용자만 캐시 제거 (예: unread:account:{accountId})
전체 무효화 금지	@CacheEvict(allEntries = true) 운영 환경 사용 금지

6.5 멍등성 보장 (Idempotency)

동일 요청이 여러 번 전달되더라도 동일한 결과를 보장하기 위해

멍등성(Idempotency) 설계를 적용한다.

이를 통해 네트워크 재시도, Webhook 재전송, 사용자 중복 클릭 등으로 인한 데이터 중복 생성 및 정합성 오류를 방지한다.

6.5.1 멱등성 적용 시나리오

시나리오	발생 원인	보장 방식
결제 요청	중복 클릭, 네트워크 재시도	Idempotency-Key + Redis
PortOne Webhook	PG사 재전송 정책	portOnePaymentId UNIQUE 제약
1:1 채팅방 생성	동일 거래에서 중복 채팅 시작	(refType, refId, type) UNIQUE
좋아요 토글	빠른 중복 탭	(accountId, postId) UNIQUE + 토글
주문 생성	중복 제출	주문번호 + Idempotency-Key

6.5.2 Idempotency-Key 패턴

클라이언트에서 생성한 고유 키를 기반으로 서버에서 중복 요청을 제어한다.

- 처리 흐름
 - Client → UUID 생성 후 HTTP Header에 포함
POST /payments
Idempotency-Key: 7f8e9d2a-...
 - Server → Redis 조회
GET idempotency:payment:{key}
 - Cache HIT → 기존 응답 반환
 - Cache MISS → 정상 처리 진행
 - 처리 완료 후 Redis 저장 (TTL 24h)
 - 동일 요청 재시도 시 캐시된 응답 반환
- 설계 의도
 - 네트워크 재시도 상황에서도 동일 결과 보장
 - 결제 중복 처리 방지
 - 클라이언트-서버 간 멱등성 협력 구조 구현

6.5.3 DB UNIQUE 제약 기반 멱등성

Webhook과 같이 클라이언트가 멱등 키를 제어할 수 없는 경우, 비즈니스 키에 UNIQUE 제약을 적용하여 중복 처리를 방지한다.

- 적용 예시

```
@Entity
@Table(uniqueConstraints = {
    @UniqueConstraint(columnNames = "portOnePaymentId"),
    @UniqueConstraint(columnNames = "idempotencyKey")
})
public class Payment { ... }

@Transactional
public void handleWebhook(WebhookRequestDto dto) {
    if (paymentRepository.existsByPortOnePaymentId(dto.getPortOnePaymentId())) {
        log.info("이미 처리된 Webhook: {}", dto.getPortOnePaymentId());
        return;
    }
    // 정상 처리...
}
```

- 설계 의도
 - 외부 시스템(Webhook) 재전송에 대한 방어
 - DB 레벨에서 최종 중복 방지
 - 애플리케이션 로직과 DB 제약의 이중 안전장치

6.5.4 토글 작업 멱등성

좋아요/싫어요 같은 토글 기능은 "현재 상태와 관계없이 결과 상태가 결정"되도록 설계한다.

- 적용 예시

```
public FavoriteToggleResponseDto toggleFavorite(Long accountId,
FavoriteToggleRequestDto dto) {
    Optional<Favorite> existing = favoriteRepository
        .findByAccountIdAndRefTypeAndRefId(accountId, dto.getRefType(), dto.getRefId());

    if (existing.isPresent()) {
        favoriteRepository.delete(existing.get());
        return new FavoriteToggleResponseDto(false);
    } else {
        favoriteRepository.save(new Favorite(accountId, dto.getRefType(), dto.getRefId()));
        return new FavoriteToggleResponseDto(true);
    }
}
```

- 설계 의도
 - 중복 클릭 시에도 상태 일관성 유지
 - UNIQUE 제약과 결합하여 데이터 무결성 보장

6.6 삭제 정책 (Deletion Policy)

도메인 특성에 따라 Soft Delete와 Hard Delete를 구분하여 적용한다.
무분별한 Soft Delete 사용은 쿼리 복잡도 증가 및 저장 공간 낭비를 초래할 수 있다.

6.6.1 삭제 정책 결정 기준

판단 기준	Soft Delete	Hard Delete
복구 필요성	복구 필요	복구 불필요
법적 보존	필요	불필요
참조 무결성	참조 많음	독립적
통계 목적	유지 필요	영향 없음

6.6.2 도메인별 삭제 정책

도메인	정책	적용 이유
Account	Soft Delete (30일 유예)	복구 + 법적 보존 + 다수 참조
ChatMessage	Soft Delete (isDeleted)	대화 맥락 유지
Category	Soft Delete (isActive)	참조 데이터 보호
ProductCategory	Soft + 조건부 Hard Delete	미사용 시 물리 삭제
기타 도메인	Hard Delete + CASCADE	단순 데이터

6.6.3 Account Soft Delete 흐름

- 1. 사용자 탈퇴 요청
 - ↓ Re-auth 검증
 - ↓ status = WITHDRAWN, deletedAt 설정
 - ↓ Refresh Token 삭제
- 2. 30일 이내 복구 가능
 - status = ACTIVE
- 3. 30일 이후 자동 삭제
 - Scheduler 수행
 - CASCADE 정리

6.6.4 ChatMessage Soft Delete

메시지 삭제 시 실제 데이터는 유지하고 상태만 변경한다.

- 적용 예시

@Entity

```
public class ChatMessage {
    private boolean isDeleted = false;

    public void markDeleted() {
        this.isDeleted = true;
    }
}

if (msg.isDeleted()) {
    return new ChatMessageResponseDto(
        msg.getChatmessageId(),
        "삭제된 메시지입니다",
        null,
        msg.getMessageType(),
        msg.getSentAt()
    );
}
```

- 설계 의도

- 대화 흐름 유지
- 신고 및 감사 로그 대응 가능

6.6.5 Category 삭제 전략

기본적으로 Soft Delete를 적용하며, 참조 데이터가 없는 경우에만 Hard Delete를 허용한다.

- 적용 예시

```
public void deactivateCategory(Long categoryId) {
    category.deactivate();
}

public void hardDeleteCategory(Long categoryId) {
    if (참조 존재) throw Exception;
    categoryRepository.deleteById(categoryId);
}
```

- 설계 의도

- 참조 무결성 유지
- 안전한 삭제 정책 적용

6.6.6 Hard Delete + CASCADE 정책

트리거	대상 도메인	처리 방식
회원 삭제	Favorite, Notification, Cart	deleteAllByAccountId
상점 삭제	Product, Image	deleteAllByStoreId
상품 삭제	Option, Image	deleteAllByProductId
리뷰 삭제	Reply, Image	deleteAllByReviewId
게시글 삭제	Comment, Like	deleteAllByPostId
댓글 삭제	Reply	deleteAllByCommentId
채팅방 삭제	Message, Participant	deleteAllByRoomId
공통 삭제	Favorite, Report	deleteAllByRefTypeAndRefId
기타 도메인	Hard Delete + CASCADE	단순 데이터

6.7 AOP 기반 횡단 관심사 분리

로그, 권한 검증, 감사 기록, 캐시 처리와 같은 비즈니스 로직과 직접 관련 없는 공통 기능을 AOP(Aspect-Oriented Programming)로 분리한다.+

이를 통해 핵심 비즈니스 로직의 복잡도를 낮추고, 공통 기능을 중앙에서 일관되게 관리할 수 있도록 한다.

6.7.1 AOP 적용 영역

관심사	어노테이션	구현 클래스	역할
감사 로그	@AdminAudit	AdminAuditAspect	관리자 작업 자동 기록
권한 검증	@OwnerOnly, @AdminOnly	AuthorizationAspect	사용자 권한 검증
분산 락	@DistributedLock	DistributedLockAspect	Redis 기반 락 제어
멱등성	@Idempotent	IdempotencyAspect	중복 요청 방지
재시도	@Retryable	Spring Retry	외부 API 실패 및 낙관적 락 재시도
성능 측정	@PerformanceLog	PerformanceLogAspect	실행 시간 로깅

6.7.2 @AdminAudit — 감사 로그 자동화

- 어노테이션 정의

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface AdminAudit {
    AuditActionType value();
    String targetType() default "";
    String targetIdParam() default "";
}
```

- Aspect 구현

```
@Aspect
@Component
@RequiredArgsConstructor
public class AdminAuditAspect {

    private final AdminAuditService auditService;

    @Around("@annotation(adminAudit)")
    public Object log(ProceedingJoinPoint joinPoint, AdminAudit adminAudit) throws
    Throwable {

        Long adminId = SecurityUtil.getCurrentAccountId();
        String parameters = serializeArgs(joinPoint.getArgs());
        Long targetId = extractTargetId(joinPoint, adminAudit.targetIdParam());

        try {
            Object result = joinPoint.proceed();

            auditService.recordAction(AuditActionDto.success(
                adminId,
                adminAudit.value(),
                adminAudit.targetType(),
                targetId,
                parameters
            ));

            return result;

        } catch (Throwable e) {

            auditService.recordAction(AuditActionDto.failure(
                adminId,
                adminAudit.value(),
                adminAudit.targetType(),
                targetId,
                parameters,
                e
            ));

            throw e;
        }
    }
}
```

```

        e.getMessage()
    ));

    throw e;
}
}
}.

```

- 사용 예시

```

@AdminAudit(
    value = AuditActionType.ACCOUNT_SUSPEND,
    targetType = "ACCOUNT",
    targetIdParam = "accountId"
)
@Transactional
public void suspendAccount(Long accountId) {
    Account account = accountRepository.findById(accountId).orElseThrow();
    account.suspend();
}

```

6.7.3 @DistributedLock — 분산 락 처리

Redis 기반 분산 락을 AOP로 구현해 동시성 제어를 공통 로직으로 분리한다

- 어노테이션 정의

```

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface DistributedLock {
    String key();
    long waitTime() default 3000;
    long leaseTime() default 3000;
}

```

- Aspect 구현

@Aspect

@Component

@Order(1) // @Transactional보다 먼저 실행

public class DistributedLockAspect {

private final RedissonClient redissonClient;

@Around("@annotation(distributedLock)")

public Object lock(ProceedingJoinPoint joinPoint, DistributedLock distributedLock)

throws Throwable {

String key = parseSpEL(distributedLock.key(), joinPoint);

RLock lock = redissonClient.getLock(key);

boolean acquired = lock.tryLock(
distributedLock.waitTime(),
distributedLock.leaseTime(),
MILLISECONDS
);

if (!acquired) {
throw new BusinessException("동시 접근 실패");
}

try {
return joinPoint.proceed();
} finally {
if (lock.isHeldByCurrentThread()) {
lock.unlock();
}
}

}

- 설계 원칙

- 분산 락은 반드시 트랜잭션 시작 전에 획득
- @Order(1)을 통해 @Transactional보다 먼저 실행
- 트랜잭션 내부에서 락을 획득하면 커밋 전에 해제되어 무력화될 수 있음

6.7.4 AOP 실행 순서

AOP는 실행 순서에 따라 동작이 달라지므로 명확한 우선순위를 정의한다.

순서	Aspect	이유
1	DistributedLockAspect	트랜잭션 이전에 락 획득
2	IdempotencyAspect	중복 요청 사전 차단
3	AdminAuditAspect	트랜잭션 외부에서 기록
4	@Transactional	Spring 기본 트랜잭션 처리
5	AuthorizationAspect	트랜잭션 내부 권한 검증
6	PerformanceLogAspect	실제 실행 시간 측정

6.8 트랜잭션 전파 전략

이름 시스템은 Spring의 @Transactional 전파 속성을 명시적으로 제어하여 트랜잭션 경계를 명확하게 관리한다.

6.8.1 전파 속성 사용 원칙

전파 속성	사용 사례	예시
REQUIRED (기본)	일반 비즈니스 로직	createOrder, updateProduct
REQUIRES_NEW	별도 트랜잭션 필요	감사 로그 기록
NESTED	부분 롤백 처리	배치 처리
MANDATORY	반드시 트랜잭션 필요	내부 헬퍼 메서드

6.8.2 REQUIRES_NEW 적용

감사 로그와 같이 본 작업과 독립적으로 처리해야 하는 경우 별도의 트랜잭션으로 실행한다.

- 적용 예시

```
@Service
```

```
public class AdminAccountService {
```

```
    @Transactional
```

```
    public void suspendAccount(Long accountId) {
```

```
        Account account = accountRepository.findById(accountId).orElseThrow();
```

```
        account.suspend();
```

```
    }
```

```
}
```

```
@Service
```

```
public class AdminAuditService {
```

```
    @Transactional(propagation = Propagation.REQUIRES_NEW)
```

```
    public void recordAction(AuditActionDto dto) {
```

```
        adminAuditLogRepository.save(AdminAuditLog.from(dto));
```

```
    }
```

```
}
```

- 설계 의도

- 본 작업 실패 여부와 관계없이 감사 로그 유지
- 트랜잭션 분리를 통한 안정성 확보

6.8.3 readOnly 트랜잭션 활용

읽기 전용 로직에는 readOnly 옵션을 적용하여 성능을 최적화한다.

- 적용 예시

@Service

```
public class StoreService {
```

```
    @Transactional(readOnly = true)
```

```
    public Page<StoreResponseDto> getStoreList(...) { ... }
```

```
    @Transactional(readOnly = true)
```

```
    public StoreDetailResponseDto getStoreDetail(...) { ... }
```

```
    @Transactional
```

```
    public void updateRating(...) { ... }
```

```
}
```

6.8.4 트랜잭션 설계 가이드라인

- Service 계층을 트랜잭션 경계로 설정 (Controller / Repository는 제외)
- Self-invocation 주의 (내부 호출 시 트랜잭션 적용 안 됨)
- 외부 API 호출은 트랜잭션 외부에서 처리 (이벤트 + @Async 활용)
- 롤백 대상 명시 (rollbackFor = Exception.class)
- 장시간 처리 로직은 timeout 설정 권장 (@Transactional(timeout = 30))

6.9 외부 시스템 연동 패턴

PortOne, FCM, AWS S3, OAuth, 사업자번호 검증 API 등 다수의 외부 시스템과 연동된다. 외부 시스템 장애가 내부 시스템으로 전파되지 않도록 격리(Isolation) 및 안정성 확보 전략을 적용한다.

6.9.1 외부 시스템 격리 전략

외부 시스템	용도	격리 전략
PortOne	결제, 환불, Webhook	Webhook 서명 검증 + 멍등성 키
FCM	푸시 알림	PushAdapter 인터페이스 + @Async
AWS S3	이미지 저장	Presigned URL 방식
Kakao/Naver OAuth	소셜 로그인	TokenService 캡슐화
사업자번호 API	사업자 검증	@Retryable + Circuit Breaker
KakaoMap API	지도/좌표 변환	클라이언트 직접 호출

6.9.2 PortOne Webhook 보안 처리

외부 결제 시스템(Webhook)은 신뢰할 수 없는 요청이므로 보안 검증 및 맥등성 처리를 반드시 수행한다.

- 처리 흐름
 1. PortOne → POST /payments/webhook
Headers: x-portone-signature, x-portone-timestamp
 2. WebhookSignatureFilter
 - timestamp ±5분 검증 (리플레이 공격 방지)
 - HMAC-SHA256 서명 검증
 - 실패 시 401 반환
 3. SecurityConfig
 - /payments/webhook → JWT 인증 제외
 4. PaymentService.handleWebhook()
 - portOnePaymentId UNIQUE 검증 (중복 방지)
 - 결제 금액 검증
 - 주문 상태 변경
 5. 200 OK 응답 (재전송 방지)

6.9.3 FCM 푸시 — Adapter 패턴

외부 API 의존성을 줄이기 위해 Adapter 패턴을 적용한다.

- 인터페이스

```
public interface PushAdapter {
    PushResult send(PushMessage message);
    List<PushResult> sendBatch(List<PushMessage> messages);
}
```

- 구현체

```
@Component
public class FcmPushAdapter implements PushAdapter {

    @Override
    public PushResult send(PushMessage message) {
        try {
            firebaseMessaging.send(buildMessage(message));
            return PushResult.SUCCESS;
        } catch (FirebaseMessagingException e) {
            if (isInvalidToken(e)) {
                handleInvalidToken(message.getFcmToken());
                return PushResult.INVALID_TOKEN;
            }
            return PushResult.FAILED;
        }
    }
}
```

```

    }
}
}

```

- 설계 의도
 - 외부 API 변경 시 영향 최소화
 - 테스트 및 Mocking 용이
 - 장애 대응 로직 캡슐화

6.9.4 S3 Presigned URL 패턴

이미지 업로드 시 서버를 거치지 않고 클라이언트가 S3에 직접 업로드하도록 설계한다.

- 처리 흐름
 1. Client → Server: Presigned URL 요청
 2. Server → URL 생성 (TTL 5분)
 3. Client → S3 직접 업로드
 4. Client → Server: imageUrl 전달
 5. Server → S3 객체 존재 검증 후 DB 저장

- 설계 의도
 - 서버 부하 감소
 - 업로드 성능 향상
 - 대용량 파일 처리 효율화

6.9.5 Circuit Breaker 패턴

외부 API 장애 시 시스템 전체 장애로 확산되는 것을 방지한다.

- 적용 예시


```

@CircuitBreaker(name = "businessApi", fallbackMethod = "fallback")
@Retryable(maxAttempts = 3, backoff = @Backoff(delay = 1000, multiplier = 2))
public boolean validate(String businessNumber) {
    return externalApi.checkBusinessNumber(businessNumber);
}

```

- Fallback 전략


```

public boolean fallback(String businessNumber, Throwable e) {
    log.warn("API 장애, 임시 허용");
    return true; // PENDING 상태로 처리
}

```

- 설계 의도
 - 외부 장애 전파 차단
 - 서비스 지속성 확보
 - 관리자 검증으로 후처리

6.10 보안 및 인가 패턴

사용자 권한을 기반으로 접근을 제어하며, 민감 작업에 대해서는 추가 검증을 수행한다.

6.10.1 권한 등급

Role	권한	URL
ROLE_USER	일반 사용자	기본 API
ROLE_OWNER	사장	/owner/**
ROLE_ADMIN	관리자	/admin/**

6.10.2 Spring Security 권한 설정

http

```
.authorizeHttpRequests(auth -> auth
    .antMatchers("/auth/**").permitAll()
    .antMatchers("/payments/webhook").permitAll()
    .antMatchers("/admin/**").hasRole("ADMIN")
    .antMatchers("/owner/**").hasRole("OWNER")
    .anyRequest().authenticated())
```

6.10.3 다단계 검증 패턴

단순 URL 권한 검증을 넘어 추가 검증을 수행한다.

- 검증 흐름
 1. ROLE 확인
 2. accountId 추출
 3. 소유권 검증
 4. 상태 검증
 5. 비즈니스 로직 실행

6.10.4 재인증 (Re-authentication)

민감 작업은 별도의 재인증 토큰을 요구한다.

- 처리 흐름
 1. 요청 → REAUTH_REQUIRED
 2. 비밀번호 검증
 3. Redis에 토큰 저장 (TTL 5분)
 4. 요청 재시도
 5. 1회 사용 후 삭제

6.10.5 민감 정보 마스킹

항목	원본	마스킹
이메일	hong@gmail.com	ho***@gmail.com
전화번호	010-1234-5678	010-****-5678
이름	홍길동	홍*동

6.10.6 JWT 토큰 정책

토큰	TTL	저장 위치	용도
Access Token	30분	클라이언트	인증
Refresh Token	14일	Redis	재발급
			Re-auth Token
이메일 인증	3분	Redis	회원가입
비밀번호 재설정	10분	Redis	비밀번호 재설정

6.11 페이징 및 정렬 표준

일관된 API 응답과 성능 최적화를 위해 페이징 및 정렬 기준을 표준화한다.

6.11.1 페이징 방식

타입	사용 사례	특징
Page<T>	관리자, 통계, 게시판	totalElements 포함, COUNT 쿼리 발생
Slice<T>	모바일 무한 스크롤	hasNext만 확인, COUNT 없음
커서 페이징	채팅 메시지	sentAt 기준, 안정적 페이징

6.11.2 기본 페이지 정책

- 모바일 앱: size = 20
- 관리자 화면: size = 50
- 채팅 메시지: size = 30 (커서 페이징)
- 최대 허용치: size = 100 (DoS 방지)
- 기본 정렬 기준
 - 일반 목록: createdAt DESC
 - 채팅: sentAt DESC
 - 상점 검색: 사용자 선택

6.11.3 정렬 필드 화이트리스트

허용되지 않은 정렬 필드를 차단하여 보안 및 성능 문제를 방지한다.

- 어노테이션

```
@Target({ElementType.METHOD, ElementType.PARAMETER})
```

```
@Retention(RetentionPolicy.RUNTIME)
```

```
public @interface SortableFields {
```

```
    String[] value();
```

```
}
```

- 검증 로직

```
public Sort validate(Sort sort, String[] allowedFields) {
```

```
    for (Sort.Order order : sort) {
```

```
        if (!allowed.contains(order.getProperty())) {
```

```
            throw new BadRequestException("허용되지 않은 정렬 필드");
```

```
        }
```

```
    }
```

```
    return sort;
```

```
}
```

- 사용 예시

```
@GetMapping("/stores")
```

```
@SortableFields({"createdAt", "rating", "favoriteCount"})
```

```
public ResponseEntity<Page<StoreResponseDto>> getStoreList(...) { ... }
```


6.11.4 커서 페이징

offset 기반 페이징의 성능 문제와 중복 노출 문제를 해결하기 위해 시간 기반 데이터는 커서 페이징을 사용한다.

- 적용 예시

```
List<ChatMessage> messages = repository
    .findAllByChatRoomIdAndSentAtBefore(roomId, cursor, pageable);
```

- 설계 의도
 - 대용량 데이터에서 성능 유지
 - 데이터 삽입 시 중복 노출 방지
 - 안정적인 페이징 보장

6.12 응답 표준화

모든 API 응답을 일관된 구조로 제공하여 클라이언트 개발 및 유지보수성을 향상시킨다.

6.12.1 공통 응답 구조

- 성공 응답

```
{
  "success": true,
  "data": {},
  "message": "요청이 성공했습니다",
  "timestamp": "...",
  "traceId": "..."
}
```

- 실패 응답

```
{
  "success": false,
  "error": {
    "code": "ORDER_001",
    "message": "재고 부족"
  },
  "timestamp": "...",
  "traceId": "..."
}
```

6.12.2 ApiResponse 클래스

```
public class ApiResponse<T> {  
    private boolean success;  
    private T data;  
    private ErrorDetail error;  
    private String message;  
    private LocalDateTime timestamp;  
    private String traceId;  
}
```

- 설계 의도
 - 모든 API 응답 형식 통일
 - traceId 기반 로그 추적
 - 클라이언트 파싱 단순화

6.12.3 에러 코드 체계

접두사	도메인	예시
AUTH_xxx	인증/인가	AUTH_001
ACCOUNT_xxx	회원	ACCOUNT_001
STORE_xxx	상점	STORE_002
ORDER_xxx	주문	ORDER_001
CHAT_xxx	채팅	CHAT_001
COMMUNITY_xxx	커뮤니티	COMMUNITY_001
COMMON_xxx	공통	COMMON_404
VALIDATION_xxx	입력값	VALIDATION_001

6.12.4 GlobalExceptionHandler

- 핵심 처리

@ExceptionHandler(BusinessException.class)

→ 비즈니스 오류 처리

@ExceptionHandler(ForbiddenException.class)

→ 권한 오류 처리

@ExceptionHandler(MethodArgumentNotValidException.class)

→ 입력값 검증 오류

@ExceptionHandler(Exception.class)

→ 서버 오류

- 설계 의도
 - 예외 처리 중앙 집중화
 - 에러 응답 일관성 확보
 - 로깅 용이

6.12.5 HTTP 상태 코드 표준

상태 코드	사용 사례
200 OK	조회, 수정, 삭제
201 Created	리소스 생성
204 No Content	삭제 성공
400 Bad Request	입력 오류
401 Unauthorized	인증 실패
403 Forbidden	권한 없음
404 Not Found	리소스 없음

6.13 로깅 전략

운영 안정성 확보 및 장애 대응을 위해 일관된 로깅 전략을 적용한다.
로그는 문제 분석, 사용자 행위 추적, 시스템 상태 모니터링을 위한 핵심 요소로 활용된다.

6.13.1 로그 레벨 가이드

로그는 중요도에 따라 다음과 같이 구분하여 사용한다.

레벨	사용처	예시
ERROR	시스템 오류	DB 커넥션 실패, NullPointerException
WARN	비즈니스 규칙 위반, 외부 API 실패	잔액 부족, 결제 API 재시도
INFO	주요 비즈니스 이벤트	주문 생성, 결제 완료, 회원가입
DEBUG	개발 환경 상세 흐름	메서드 진입/종료, 쿼리 파라미터
TRACE	최저 수준 상세	SQL 결과, HTTP 헤더 전체

- 운영 정책
 - 운영 환경: INFO 이상 로그만 출력
 - 개발 환경: DEBUG, TRACE 활용
 - DEBUG/TRACE는 임시 디버깅 시에만 사용

6.13.2 MDC (Mapped Diagnostic Context)

분산 환경에서 요청 단위 추적을 위해 MDC에 traceId를 저장한다.

- 적용 방식

@Component

@Order(Ordered.HIGHEST_PRECEDENCE)

```
public class MdcFilter extends OncePerRequestFilter {
```

```
    @Override
```

```
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain) throws ServletException,
```

```
    IOException {
```

```
        try {
```

```
            String traceId = request.getHeader("X-Trace-Id");
```

```
            if (traceId == null) {
```

```
                traceId = UUID.randomUUID().toString();
```

```
            }
```

```
            MDC.put("traceId", traceId);
```

```
            MDC.put("requestUri", request.getRequestURI());
```

```
            MDC.put("method", request.getMethod());
```

```
            chain.doFilter(request, response);
```

```
        } finally {
```

```
            MDC.clear(); // Thread Pool 재사용 시 메모리 누수 방지
```

```
        }
```

```
    }
```

```
}
```

- Logback 패턴

```
%d{yyyy-MM-dd HH:mm:ss.SSS} %-5level [%X{traceId}] [%X{accountId}] %logger{36} - %msg%n
```

- 로그 출력 예시

```
2026-04-29 12:34:56.789 INFO [7f8e9d2a-...] [123] OrderService - 주문 생성 완료: orderId=456
```

- 설계 의도

- 요청 단위 로그 추적 (traceId)
- 장애 발생 시 로그 연관성 확보
- 멀티 스레드 환경에서 안전한 컨텍스트 관리

6.13.3 비동기 로그 처리

로그 출력 성능 향상을 위해 비동기 로깅을 적용한다.

- Logback 설정

<configuration>

```
<appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>logs/eeum.log</file>
  <rollingPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
    <fileNamePattern>logs/eeum-%d{yyyy-MM-dd}.%i.log.gz</fileNamePattern>
    <maxFileSize>100MB</maxFileSize>
    <maxHistory>30</maxHistory>
    <totalSizeCap>10GB</totalSizeCap>
  </rollingPolicy>
</appender>
```

```
<appender name="ASYNC_FILE" class="ch.qos.logback.classic.AsyncAppender">
  <appender-ref ref="FILE"/>
  <queueSize>10000</queueSize>
  <discardingThreshold>0</discardingThreshold>
  <neverBlock>true</neverBlock>
</appender>
```

```
<root level="INFO">
  <appender-ref ref="ASYNC_FILE"/>
</root>
```

</configuration>

- 설계 의도
 - 로그 I/O로 인한 성능 저하 방지
 - 메인 스레드 블로킹 최소화
 - 대량 로그 처리 안정성 확보

6.13.4 민감 정보 마스킹

로그에 개인정보가 노출되지 않도록 마스킹 처리를 수행한다.

- 적용 예시

@Component

```
public class SensitiveDataMaskingConverter extends ClassicConverter {

    private static final Pattern EMAIL_PATTERN =
        Pattern.compile("([a-zA-Z0-9._-]{2})[a-zA-Z0-9._-]+(@[a-zA-Z0-9.-]+)");

    private static final Pattern PHONE_PATTERN =
        Pattern.compile("(\\W\\d{3})-(\\W\\d{4})-(\\W\\d{4})");

    private static final Pattern PASSWORD_PATTERN =
        Pattern.compile("(\\W\"password\\W\"\\W\\s*:\\W\\s*)\\W\"[^\\W\"]+\\W\"");

    @Override
    public String convert(ILoggingEvent event) {
        String message = event.getFormattedMessage();

        message = EMAIL_PATTERN.matcher(message).replaceAll("$1***$2");
        message = PHONE_PATTERN.matcher(message).replaceAll("$1-***-$3");
        message = PASSWORD_PATTERN.matcher(message).replaceAll("$1\\W***\\W");

        return message;
    }
}
```

- 설계 의도
 - 개인정보 보호
 - 로그 유출 시 보안 위험 최소화
 - 규제 대응 (개인정보 보호 기준)

6.13.5 비즈니스 이벤트 로깅

주요 비즈니스 이벤트는 INFO 레벨로 기록하여 서비스 상태 및 사용자 행동을 추적한다.

- 주요 로그 예시

이벤트	로그 메시지
회원가입	회원가입 완료: accountId=123, role=USER
로그인	로그인 성공: accountId=123, ip=192.168.0.1
주문 생성	주문 생성: orderId=456, totalPrice=25000
결제 완료	결제 완료: paymentId=789, amount=25000
사장 승인	사장 승인: ownerInId=42
관리자 작업	관리자 작업: adminId=1, action=ACCOUNT_SUSPEND

- 설계 의도
 - 사용자 행동 추적
 - 장애 분석 및 감사 로그 활용
 - 운영 모니터링 기반 데이터 확보

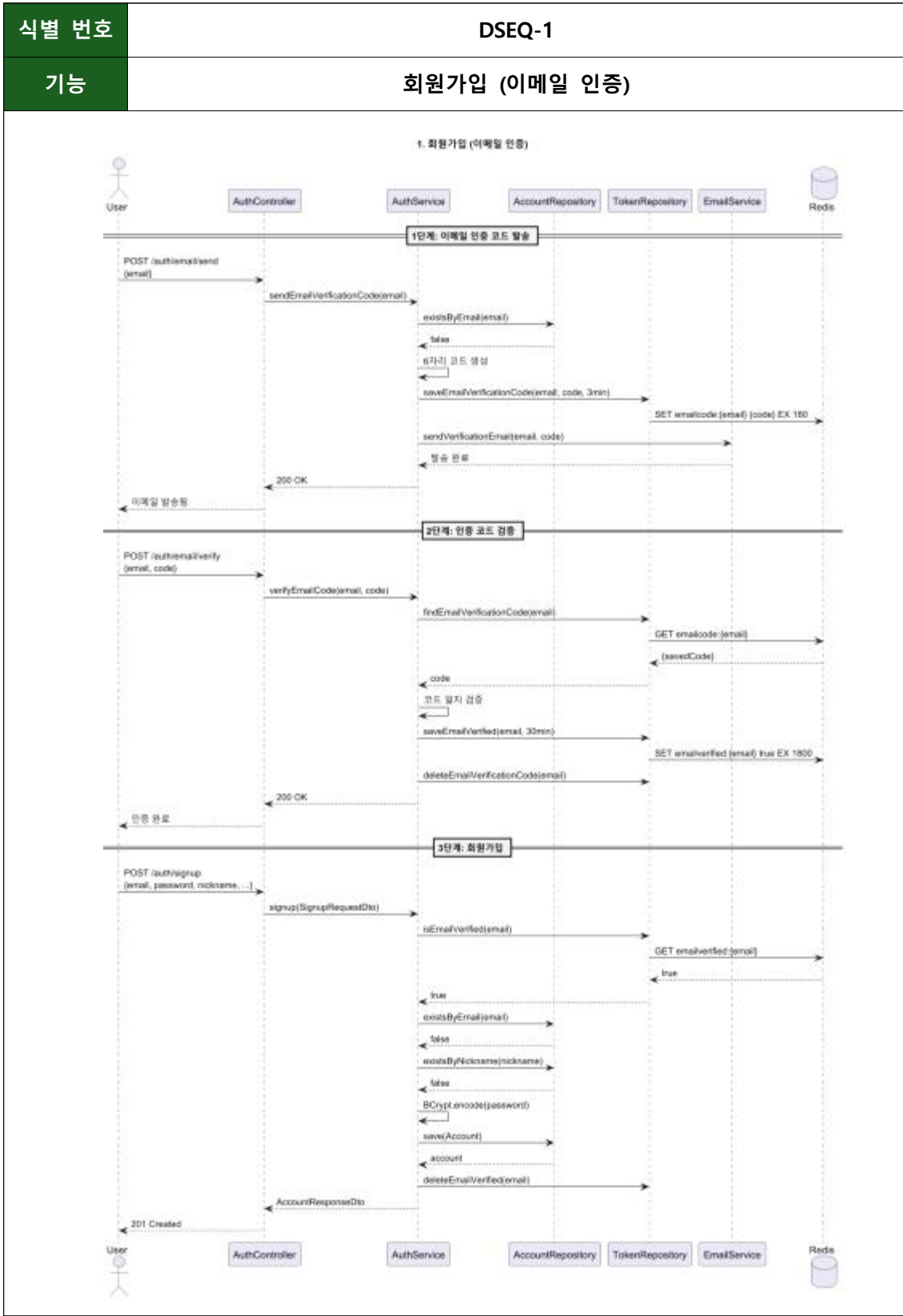
7. API 설계

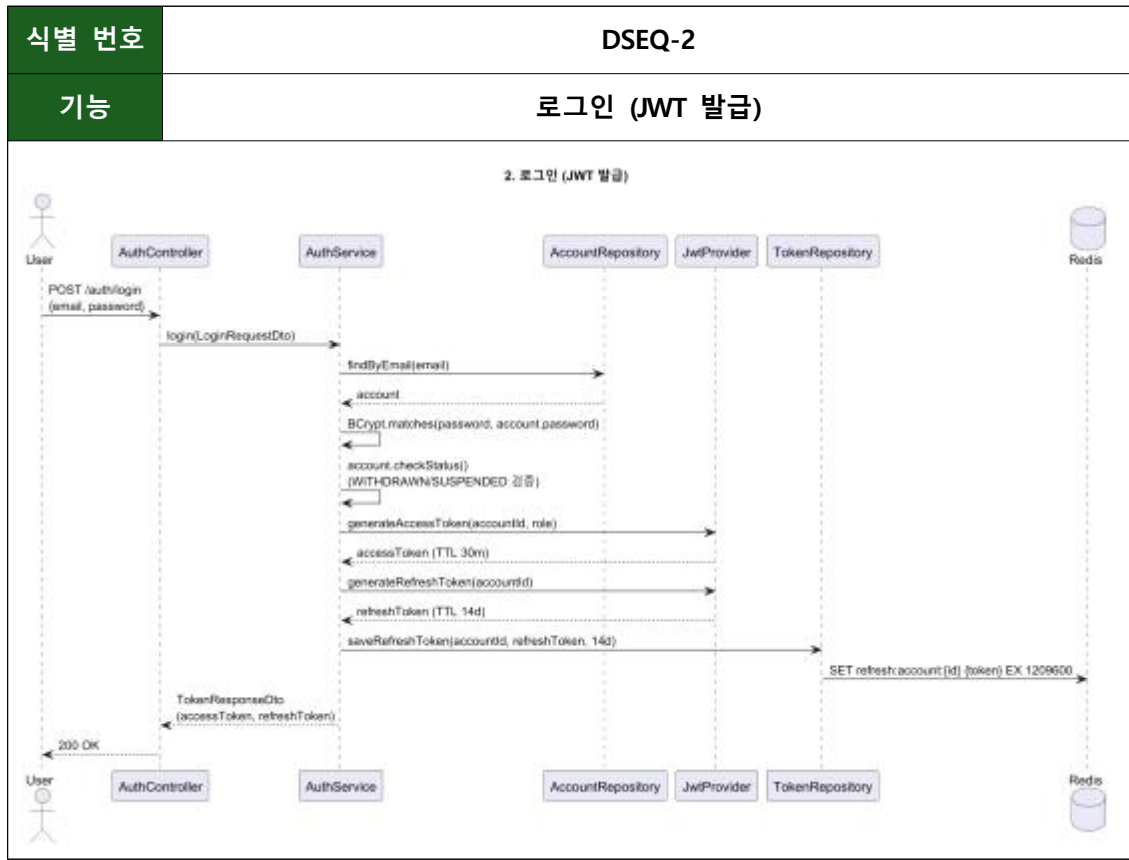
이름 시스템의 API는 RESTful 규칙을 따르며, Controller 계층을 기준으로 설계되었다. 각 도메인별 API는 다음과 같이 구성된다.

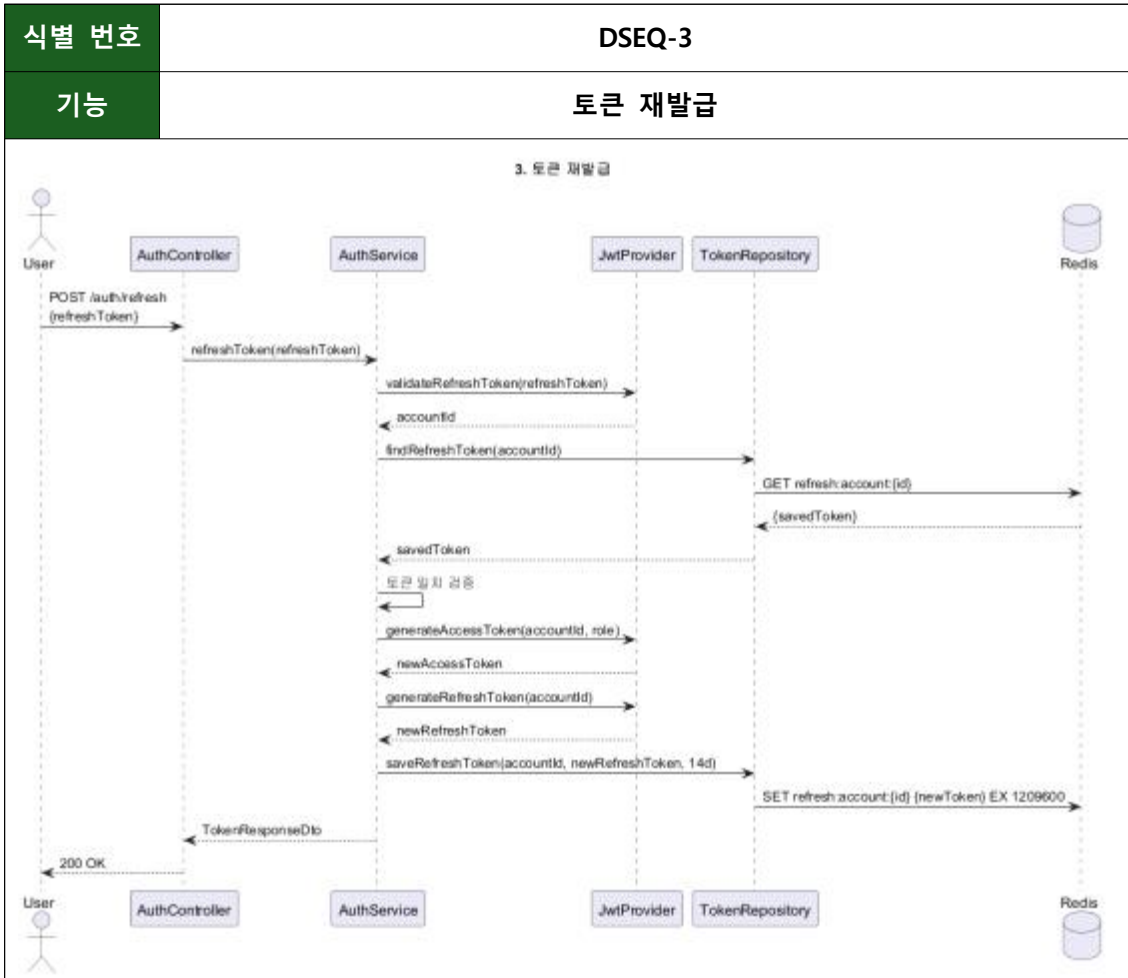
도메인	주요 기능
Auth	로그인, 회원가입, 토큰 재발급
Account	회원 정보 관리
Store	상점 조회 및 관리
Order / Payment	주문 및 결제 처리
UsedProduct	중고거래
Community	게시글 및 댓글
Chat	채팅 (REST + WebSocket)
Review	상점 리뷰, 중고거래 리뷰, 답글
Notification	알림 조회, 읽음 처리, 알림 설정, FCM 토큰 관리
Inquiry / Report	문의 작성 및 답변, 신고 접수 및 처리
Admin	관리자 기능

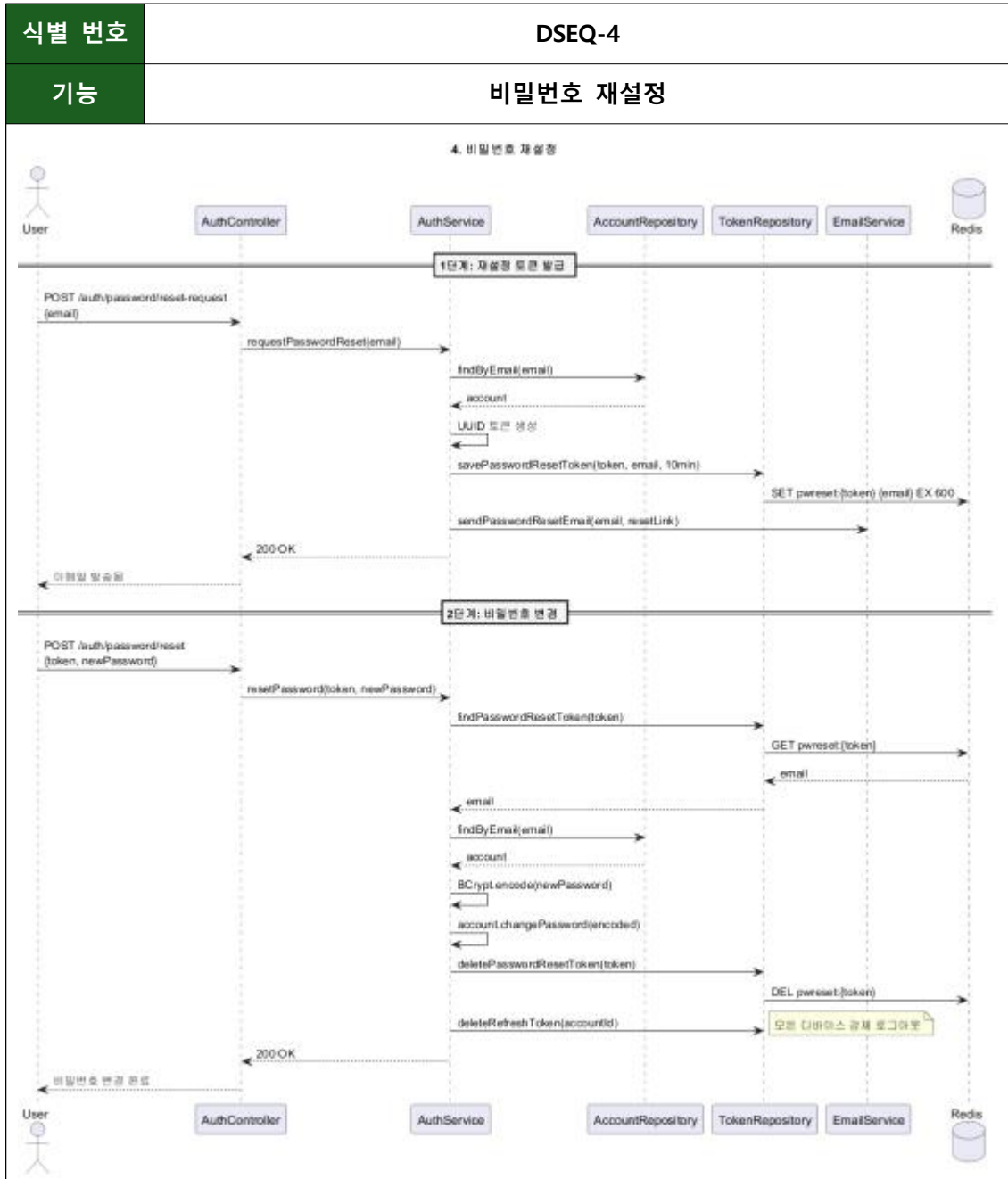
상세 API는 Controller 명세 기준으로 설계되었으며, 본 목차에서는 중복을 방지하기 위해 별도 기술하지 않는다.

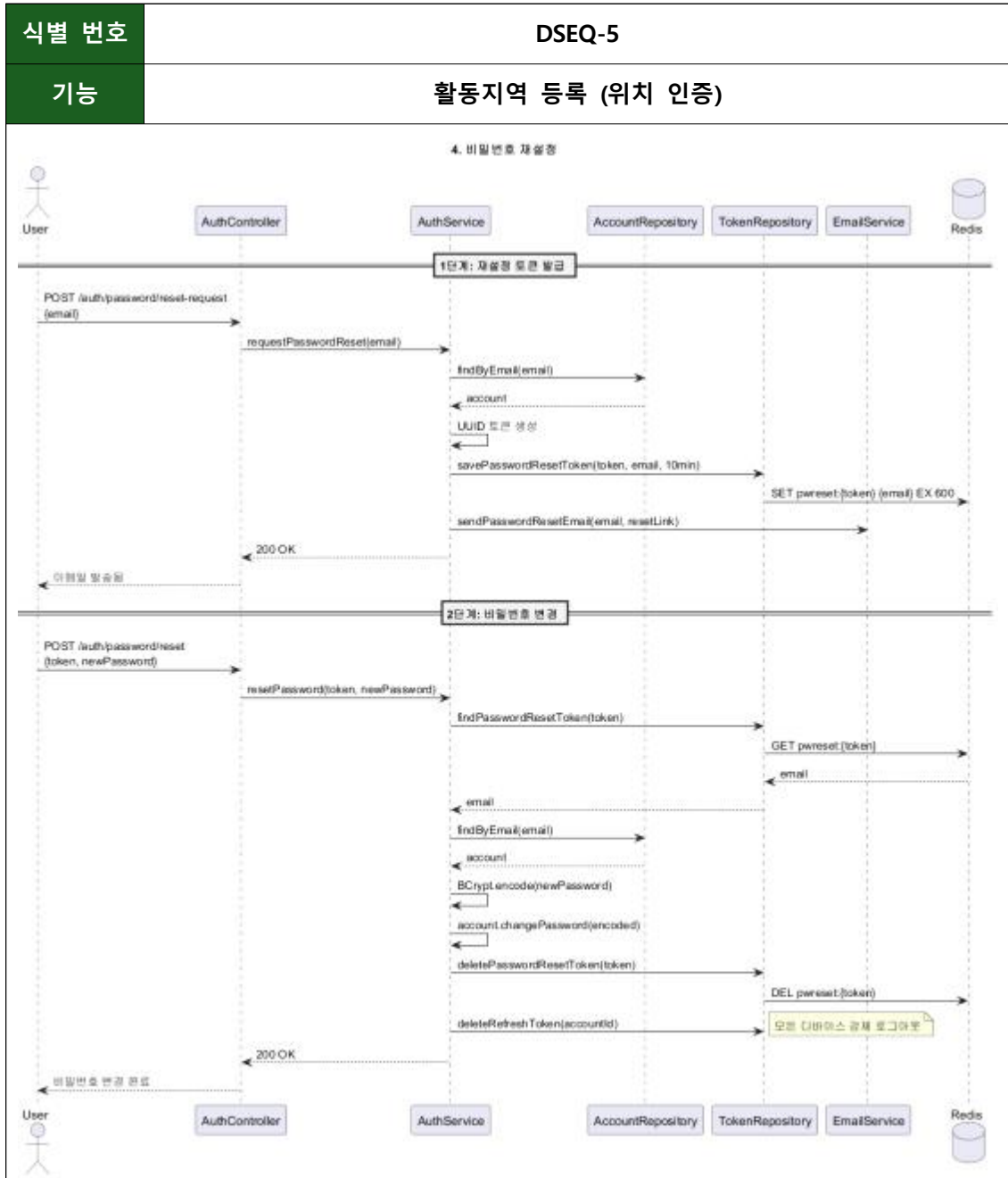
8. 설계 시퀀스 다이어그램









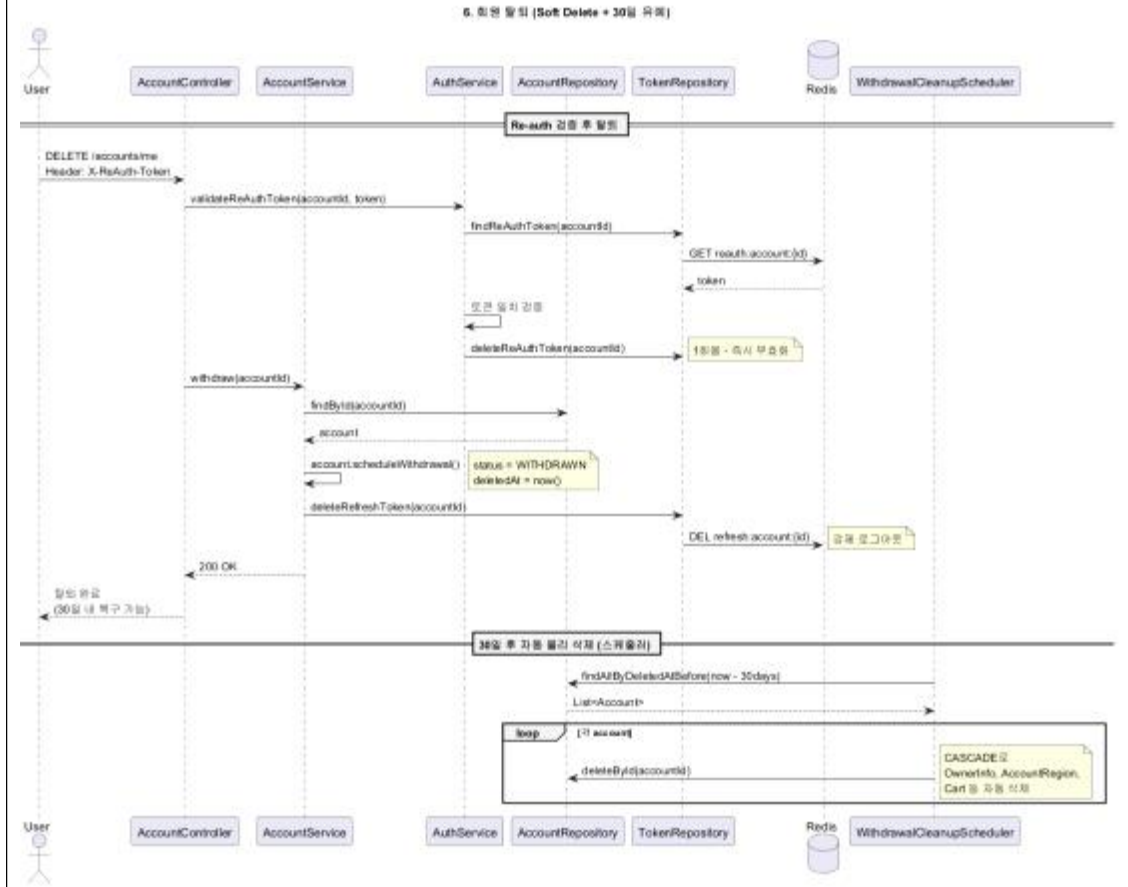


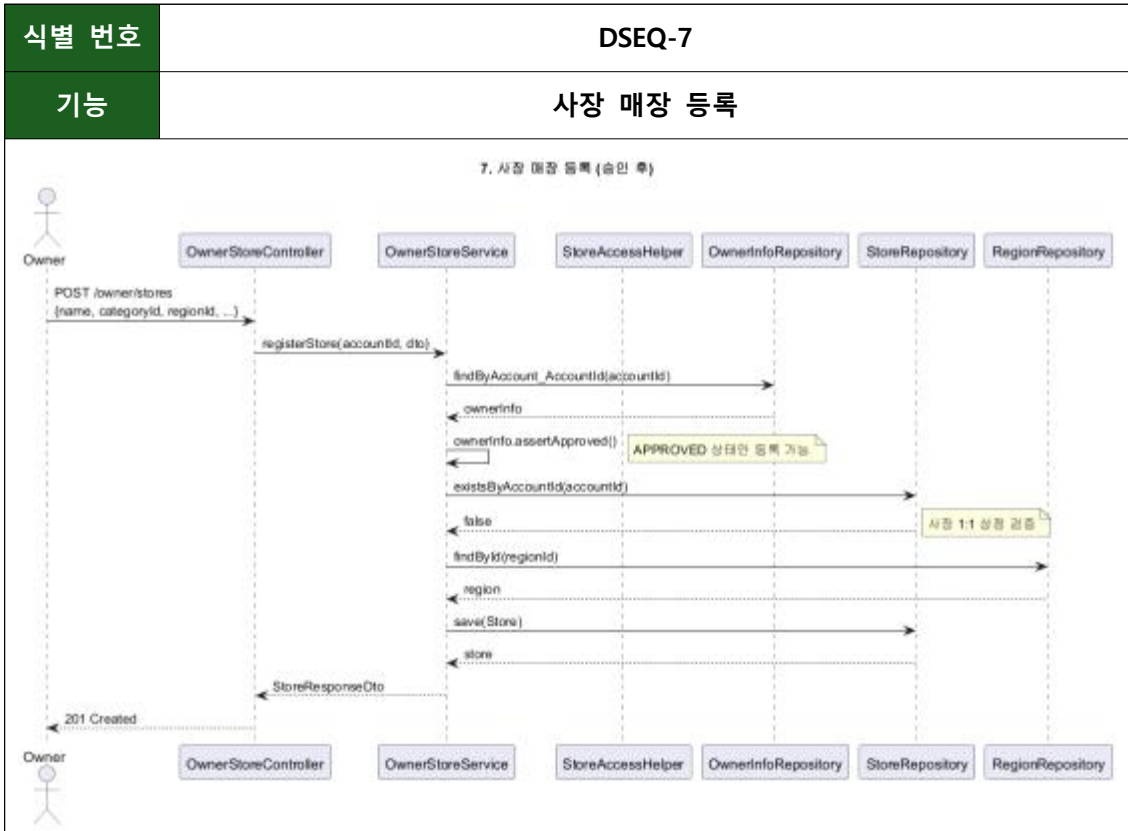
식별 번호

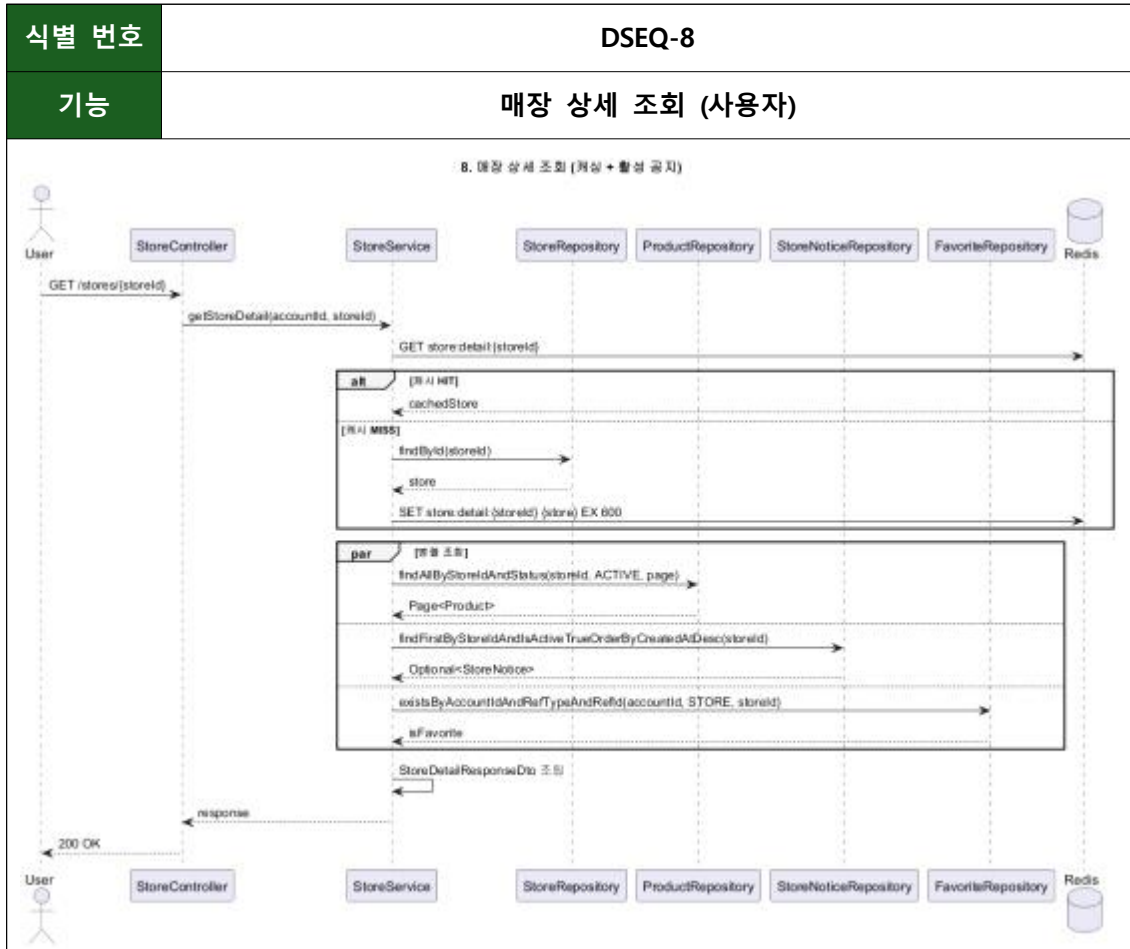
DSEQ-6

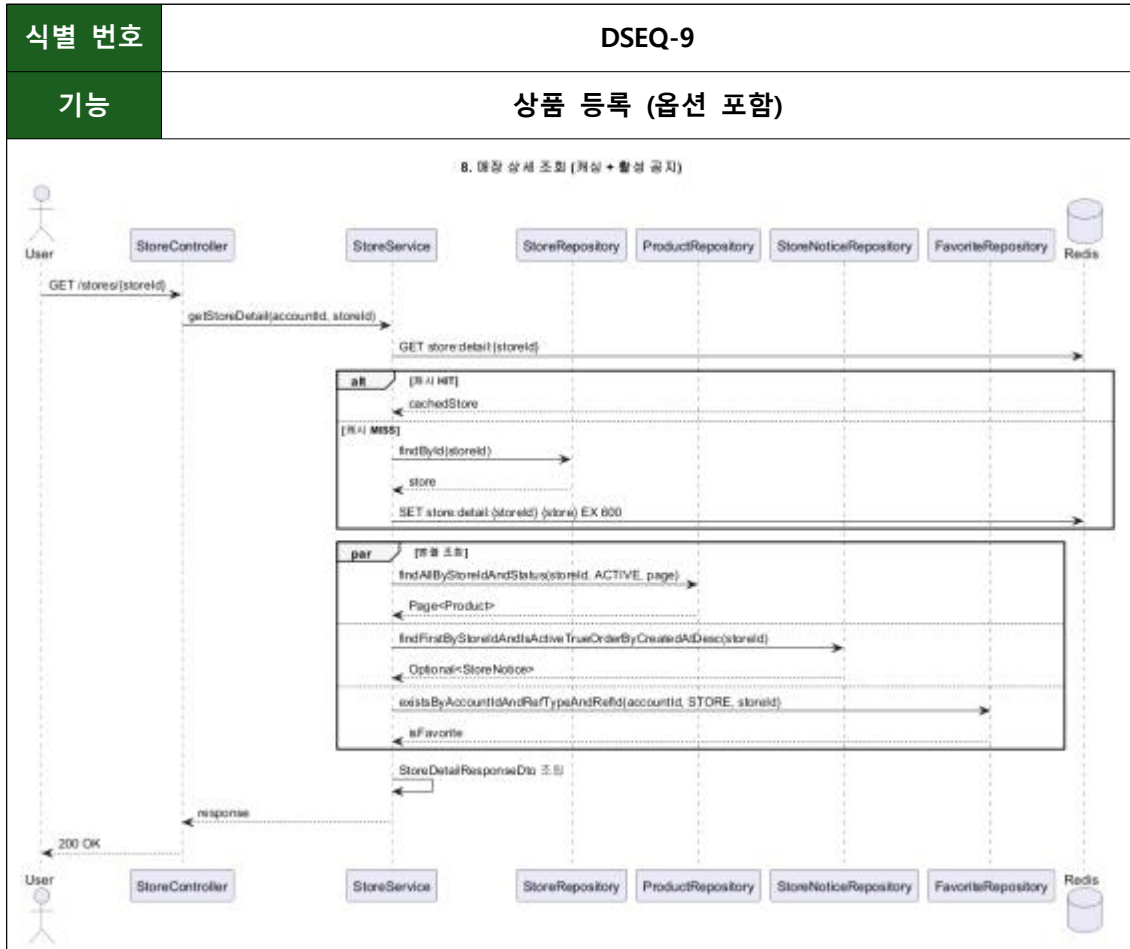
기능

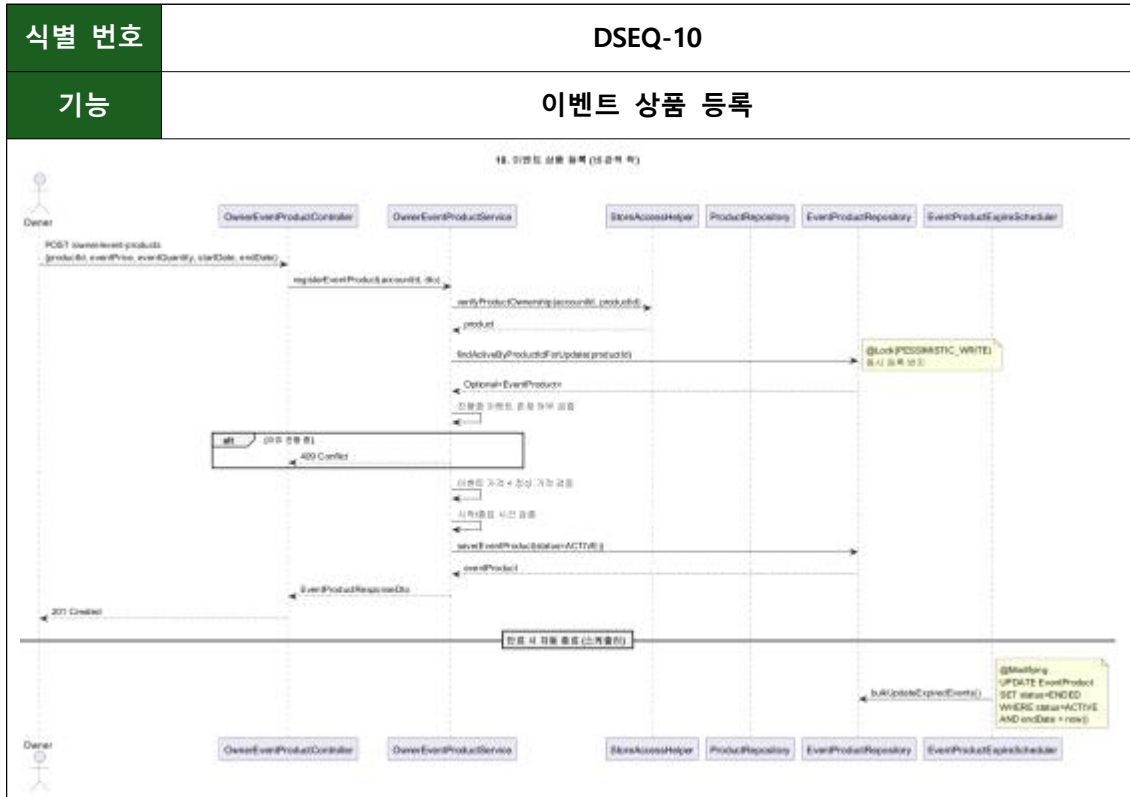
회원 탈퇴 (Soft Delete)

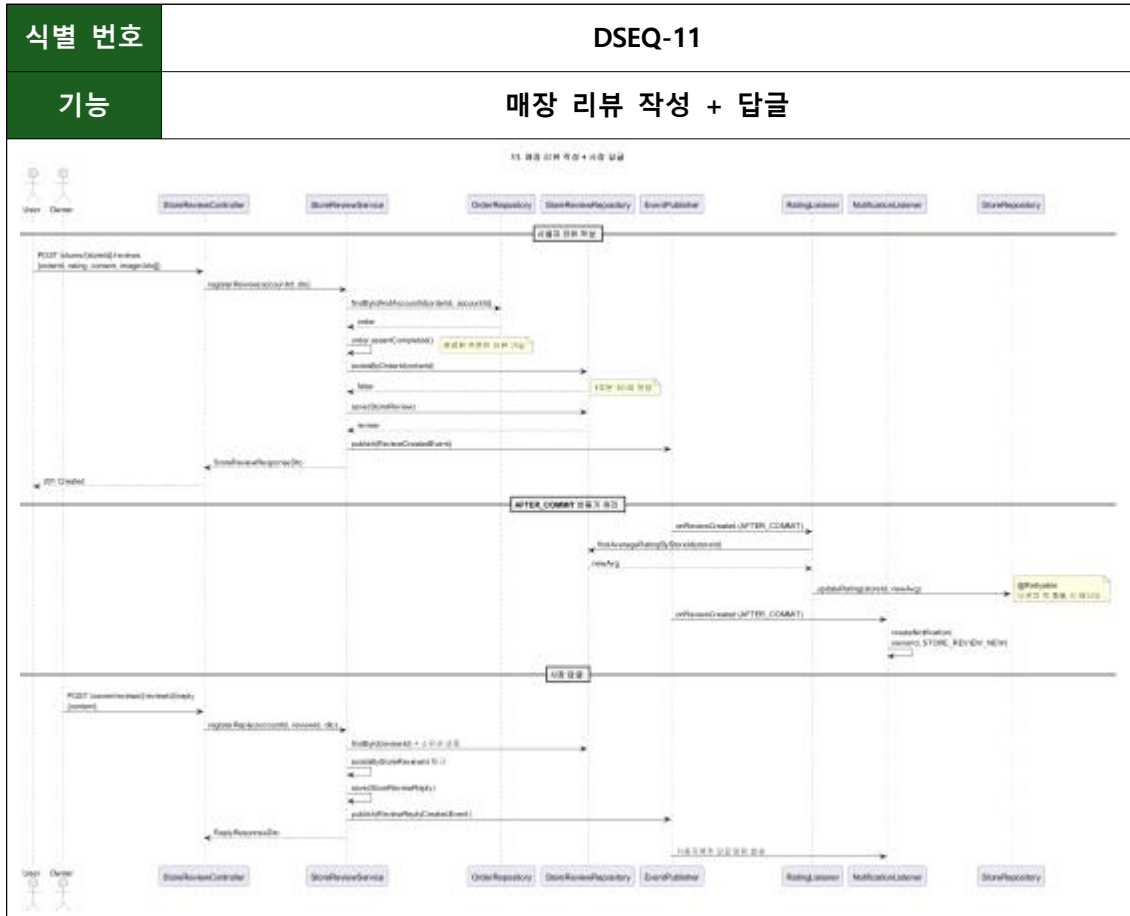










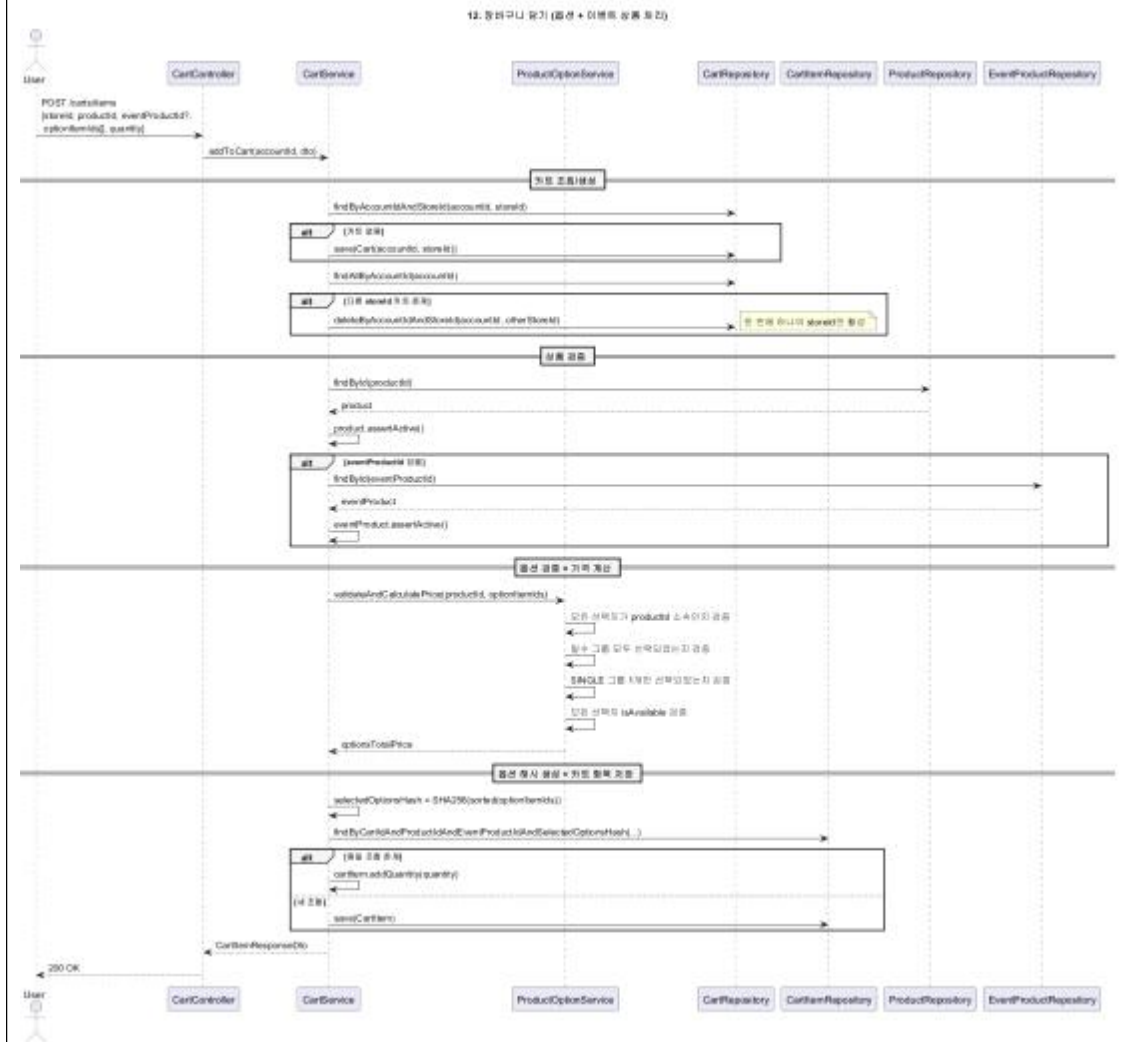


식별 번호

DSEQ-12

기능

장바구니 담기 (옵션)

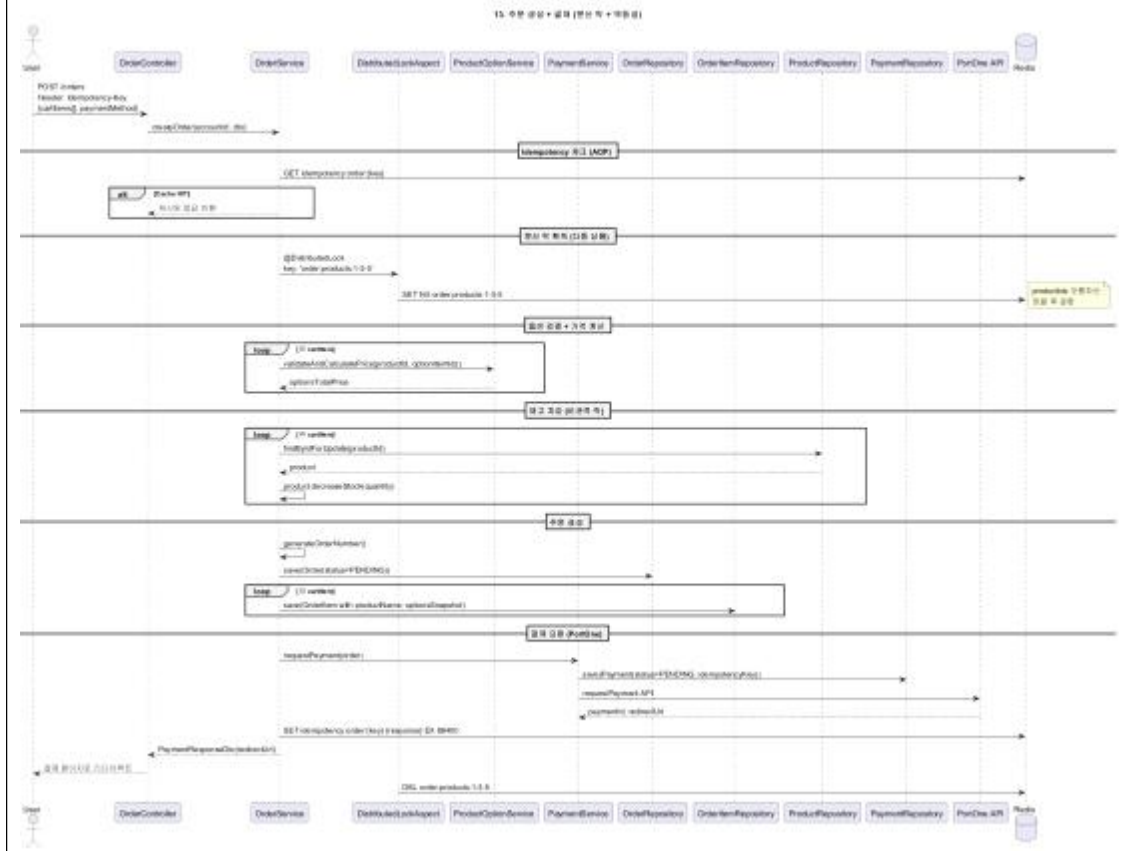


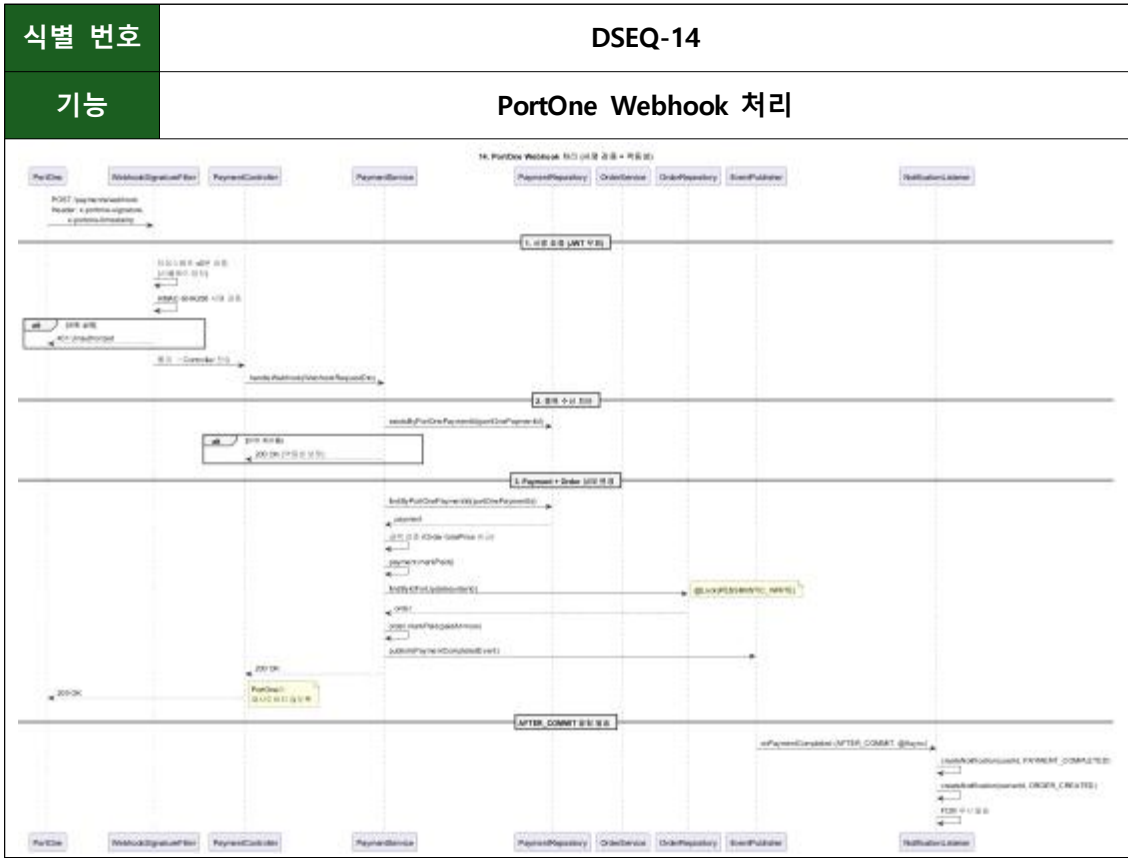
식별 번호

DSEQ-13

기능

주문 생성 + 결제 (PortOne)



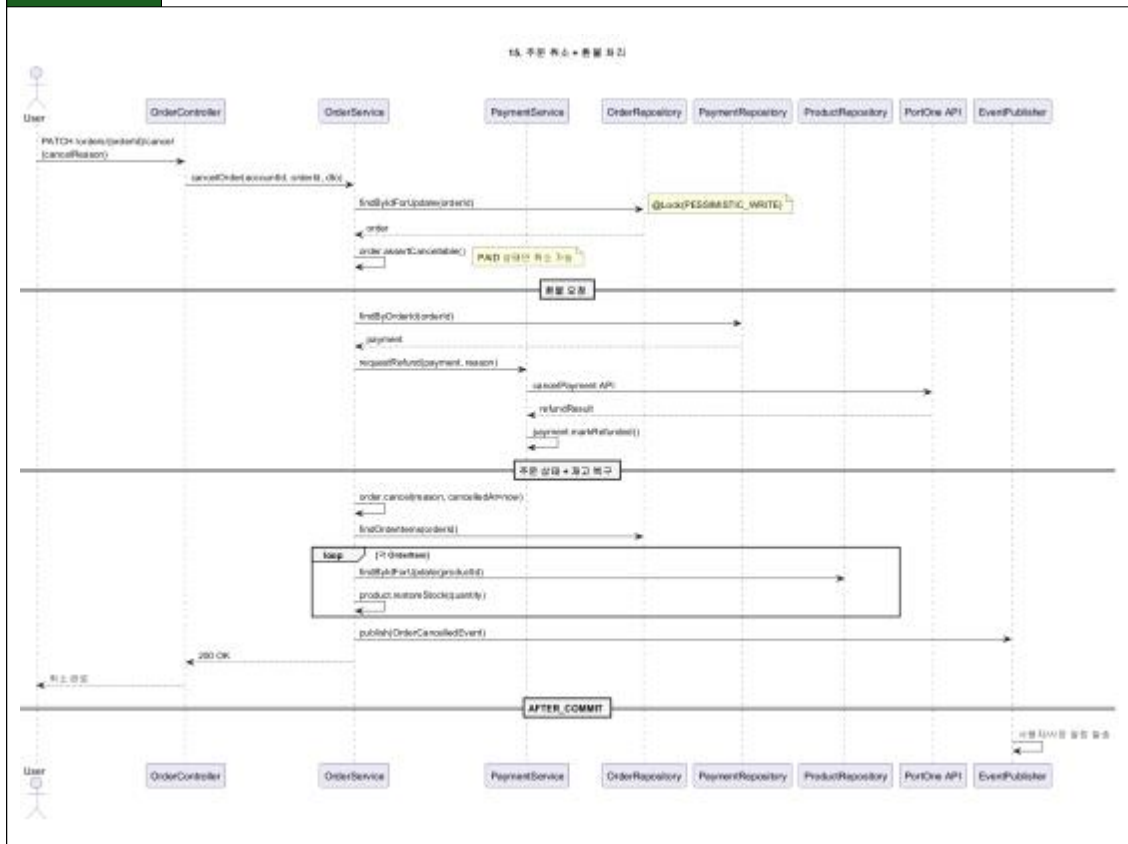


식별 번호

DSEQ-15

기능

주문 취소 + 환불

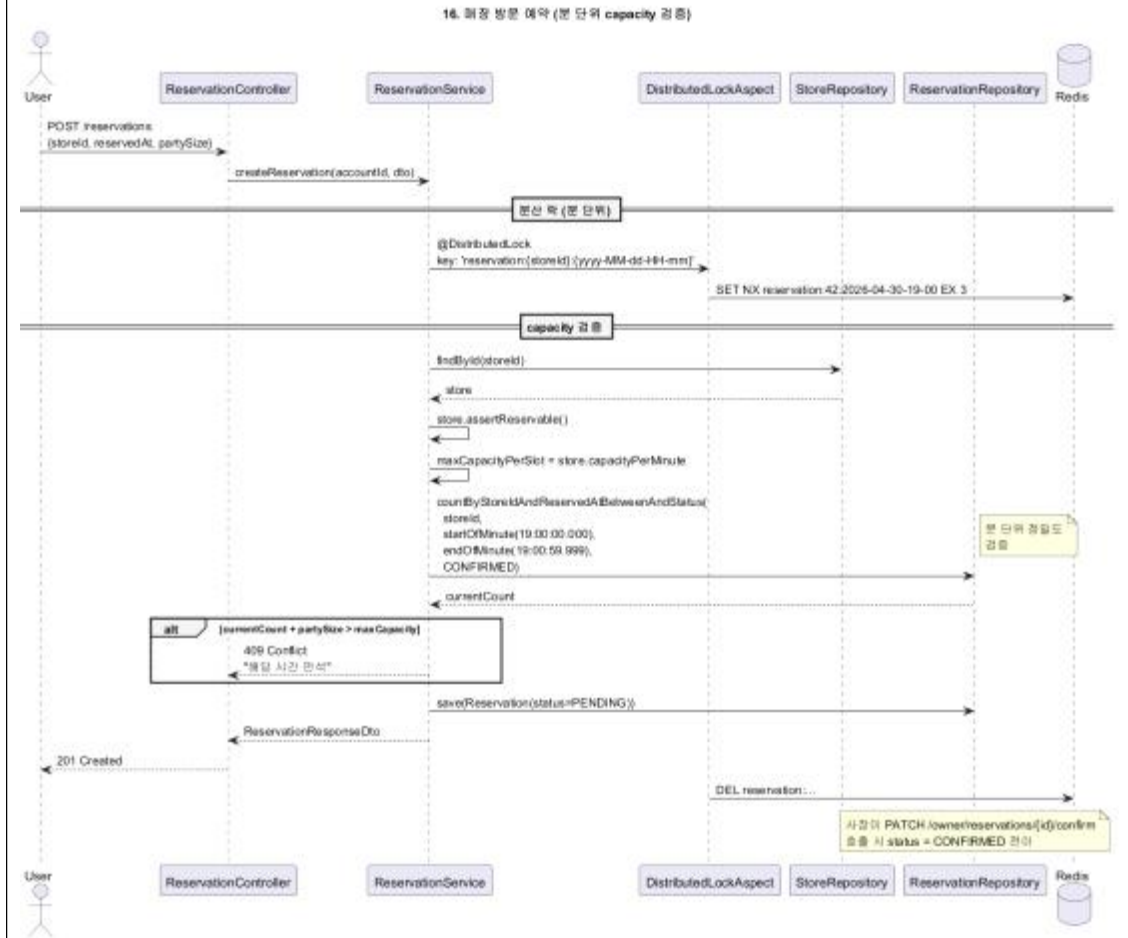


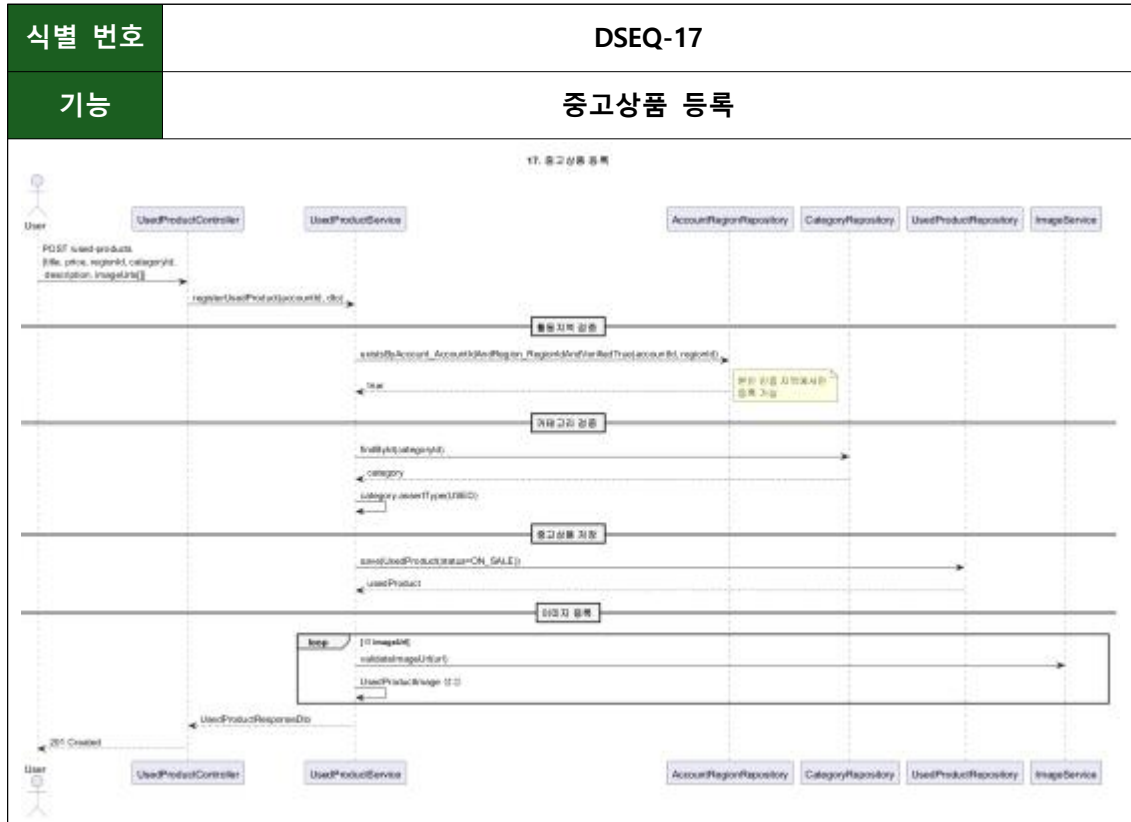
식별 번호

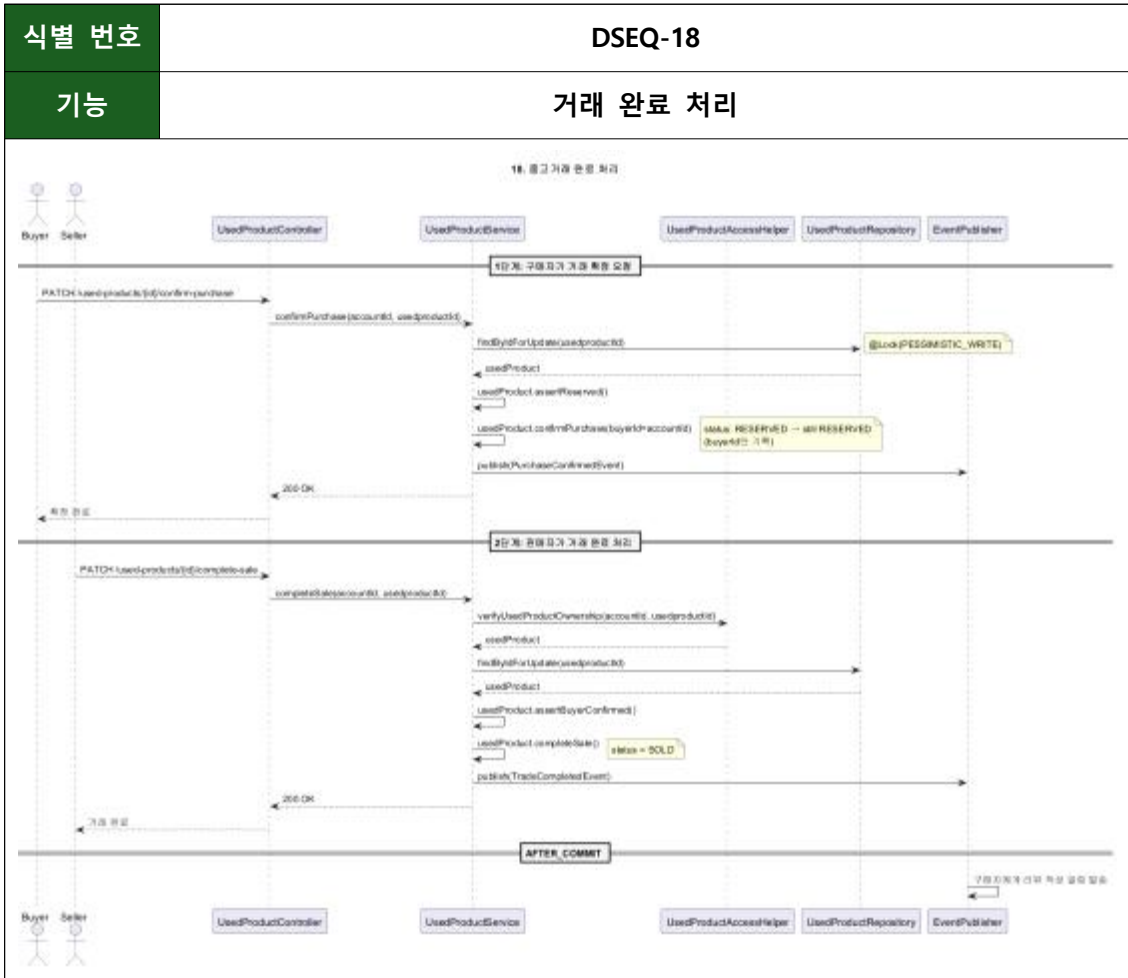
DSEQ-16

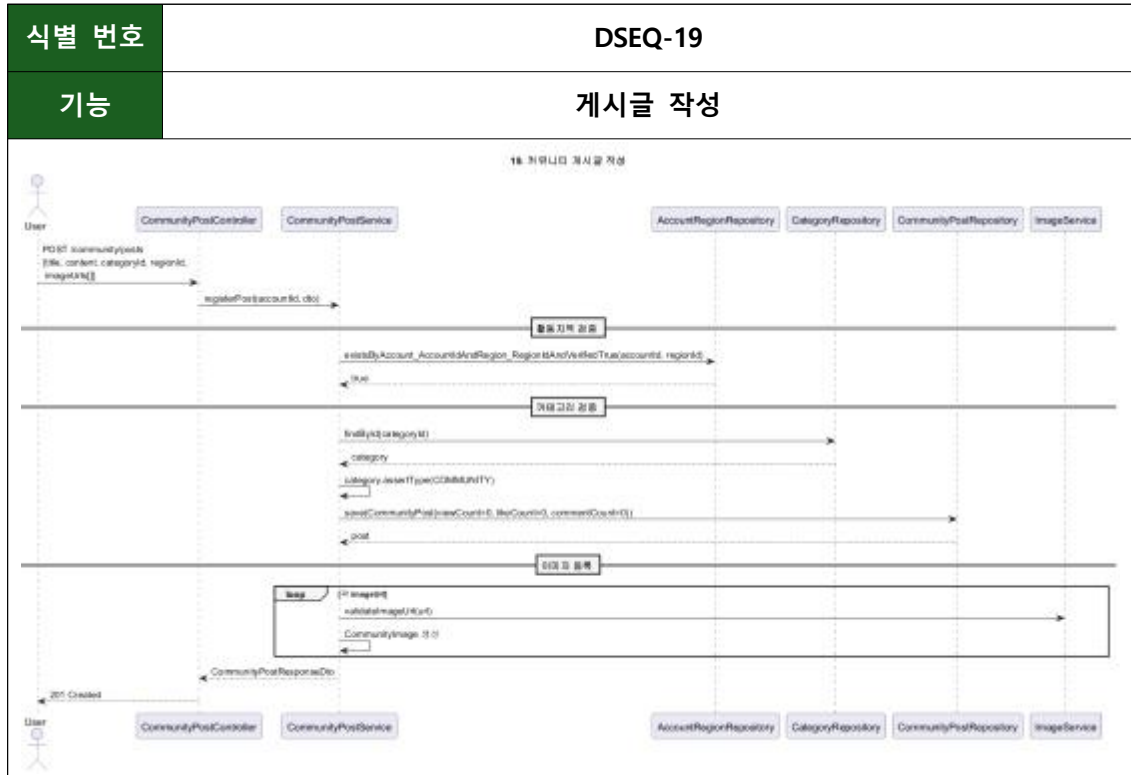
기능

매장 방문 예약 (capacity)







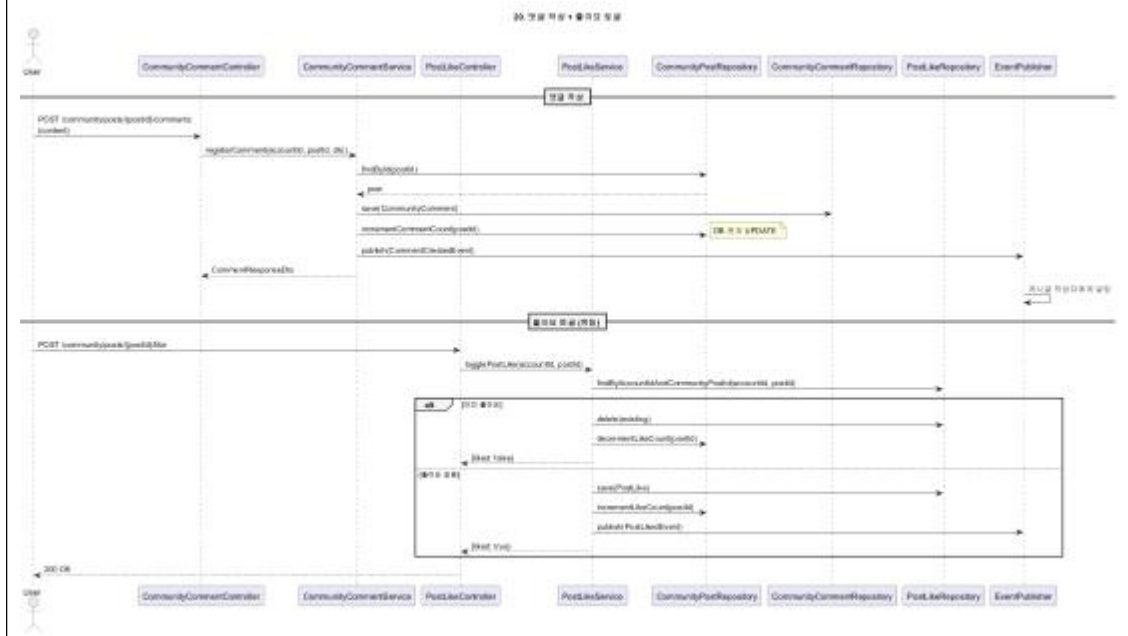


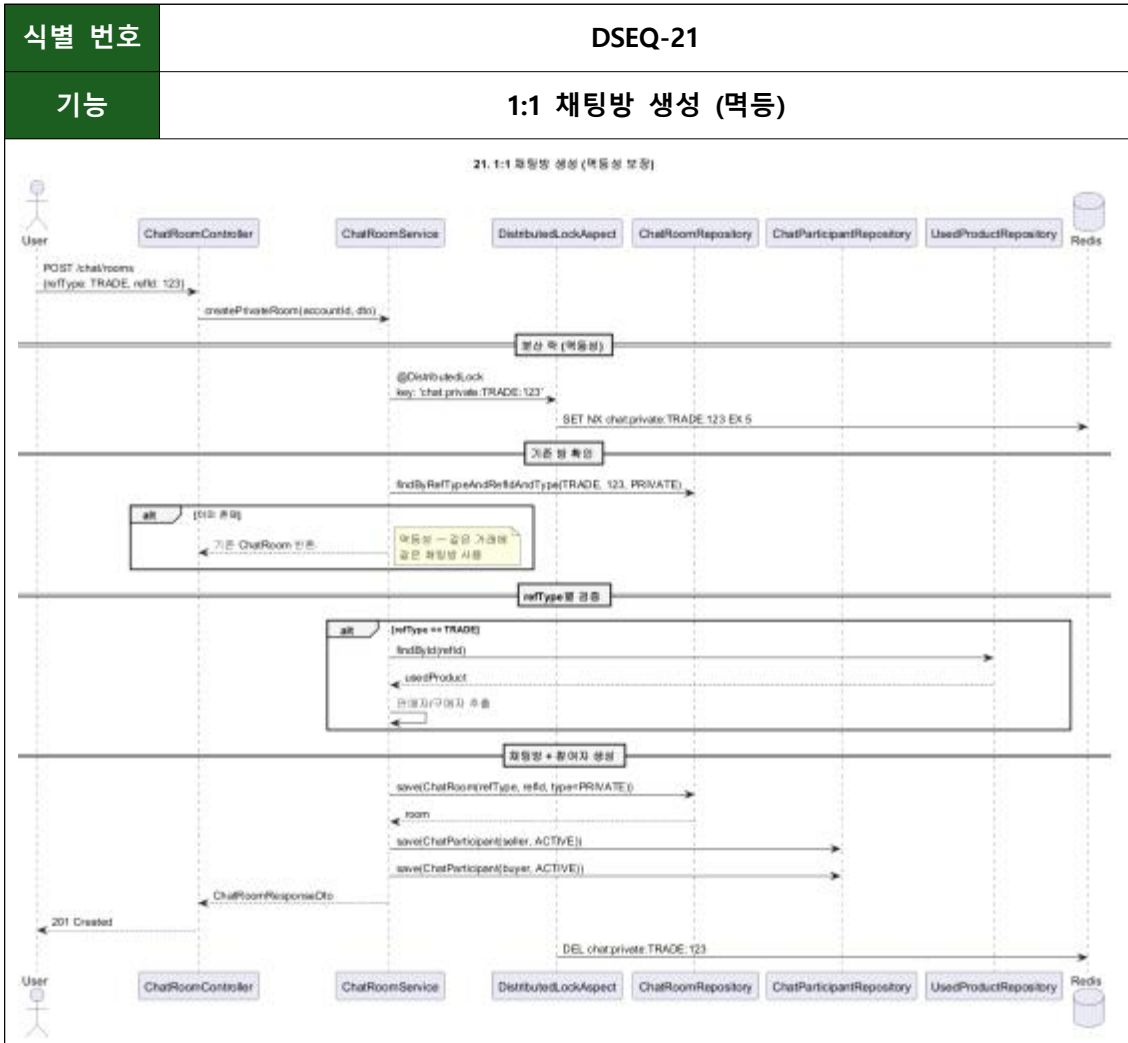
식별 번호

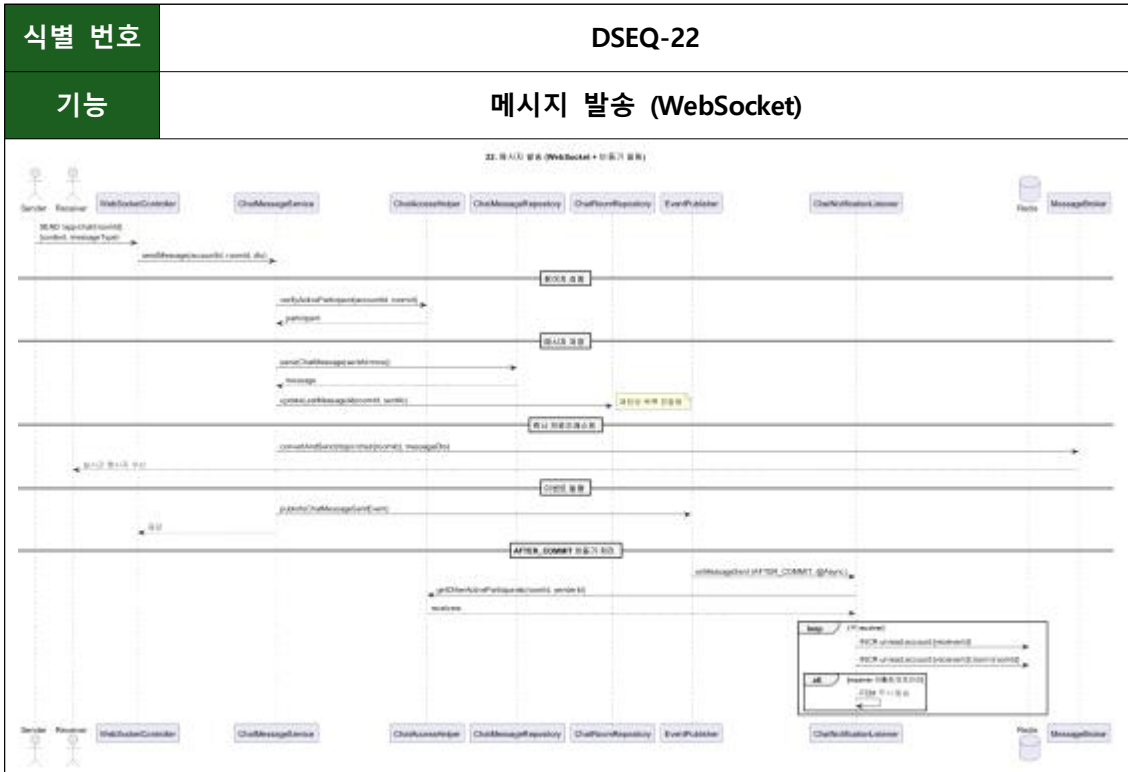
DSEQ-20

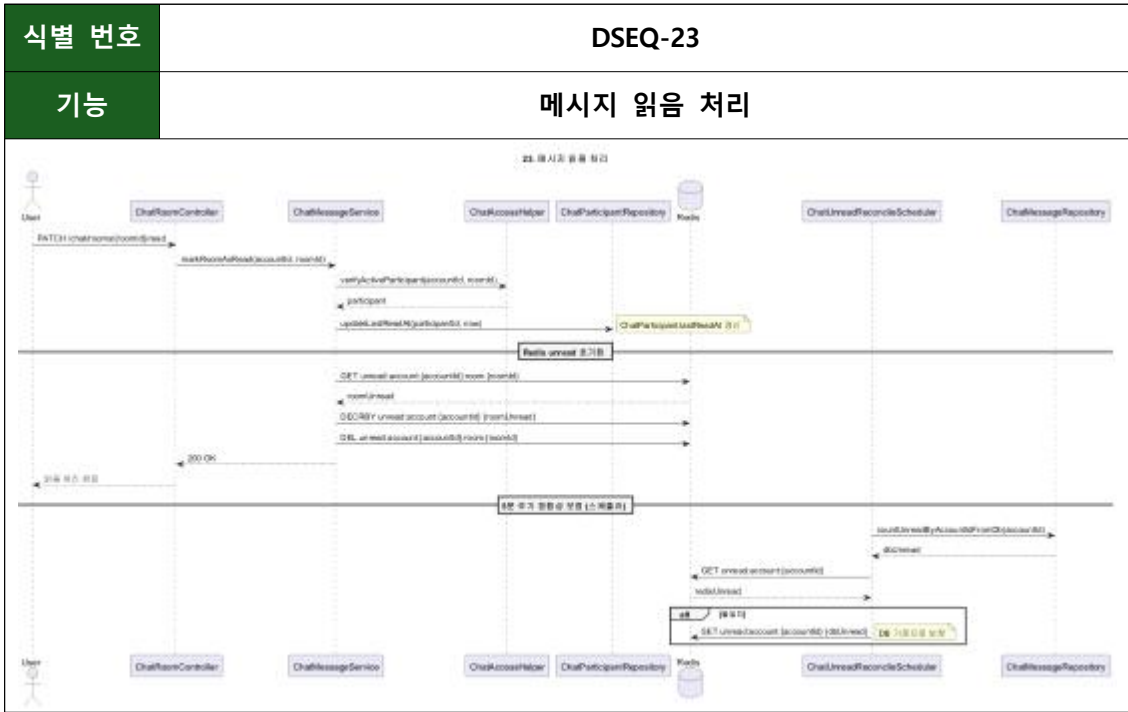
기능

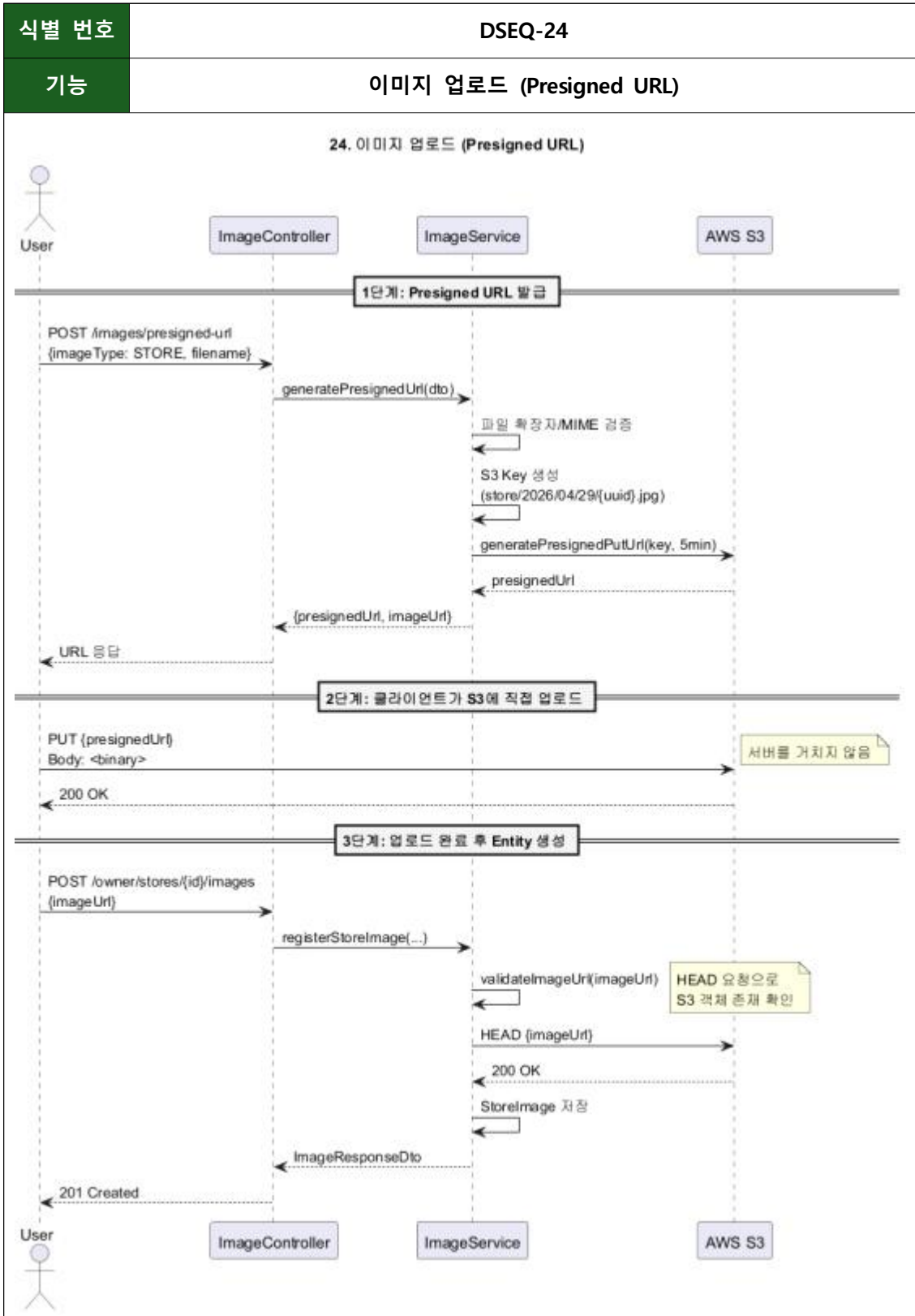
댓글 + 좋아요









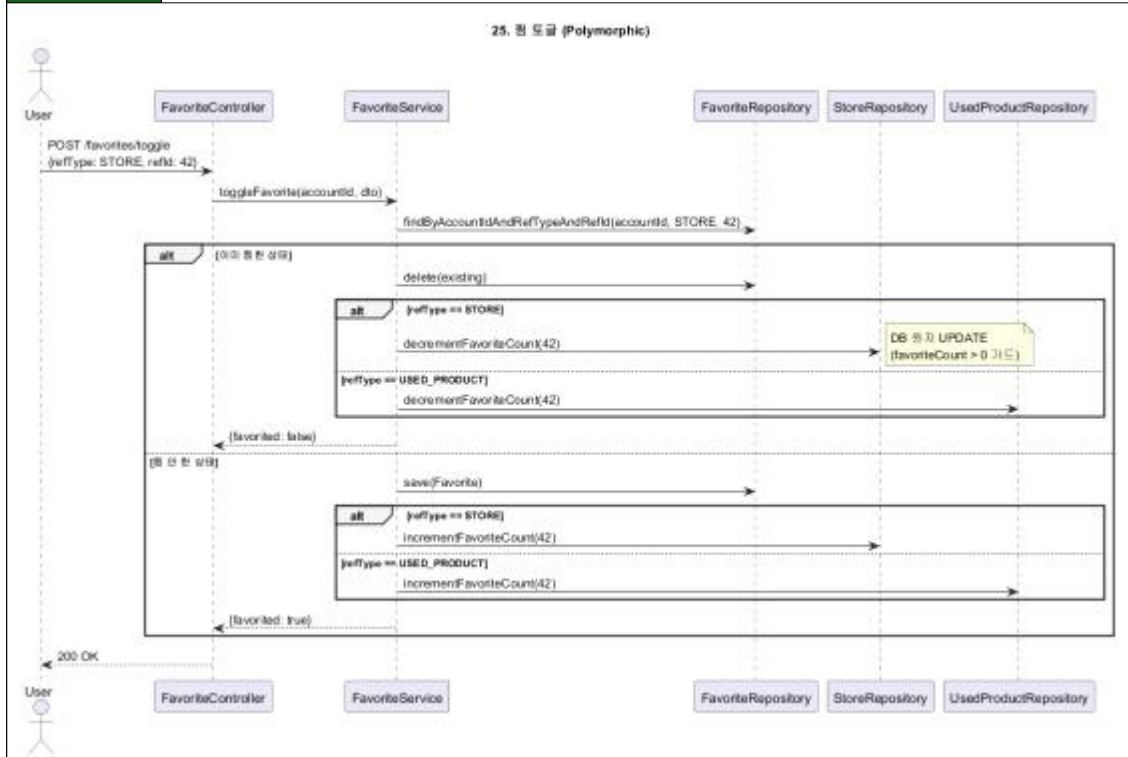


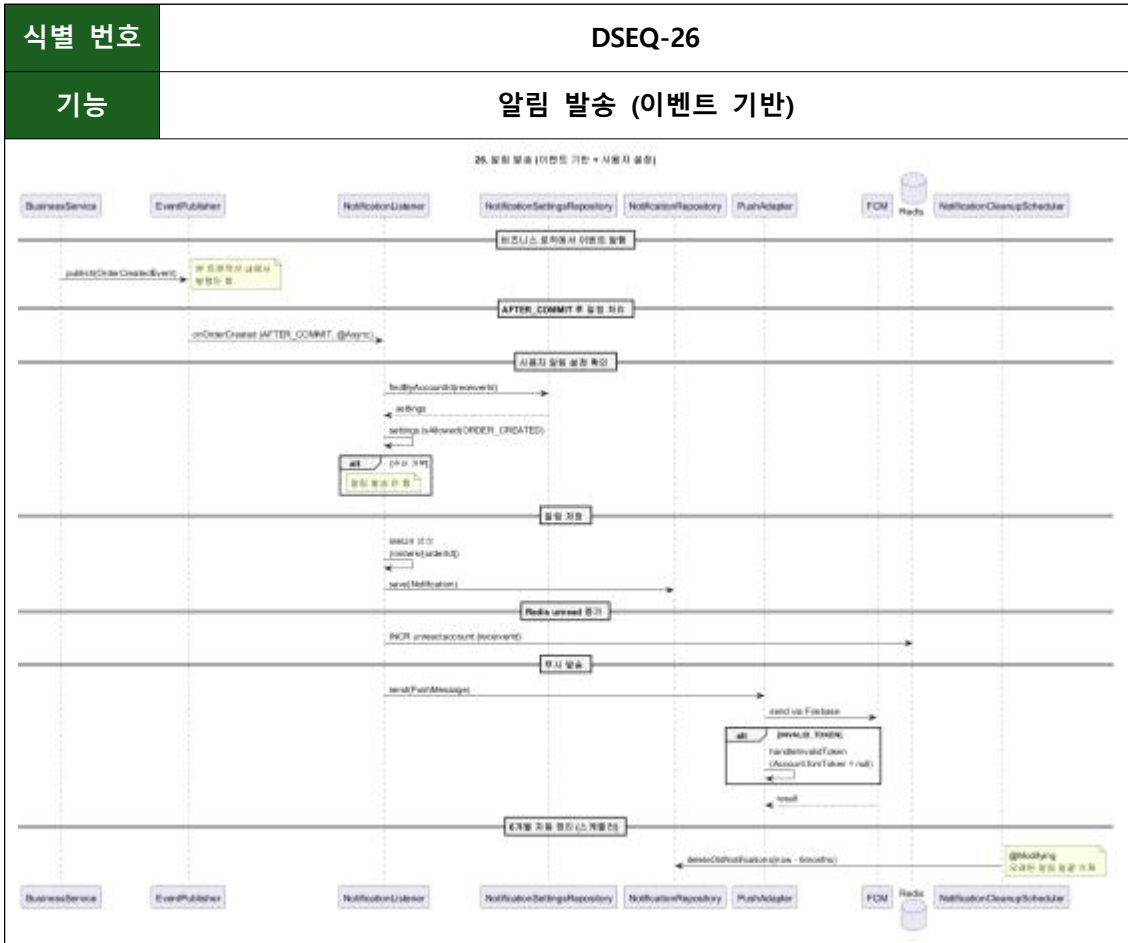
식별 번호

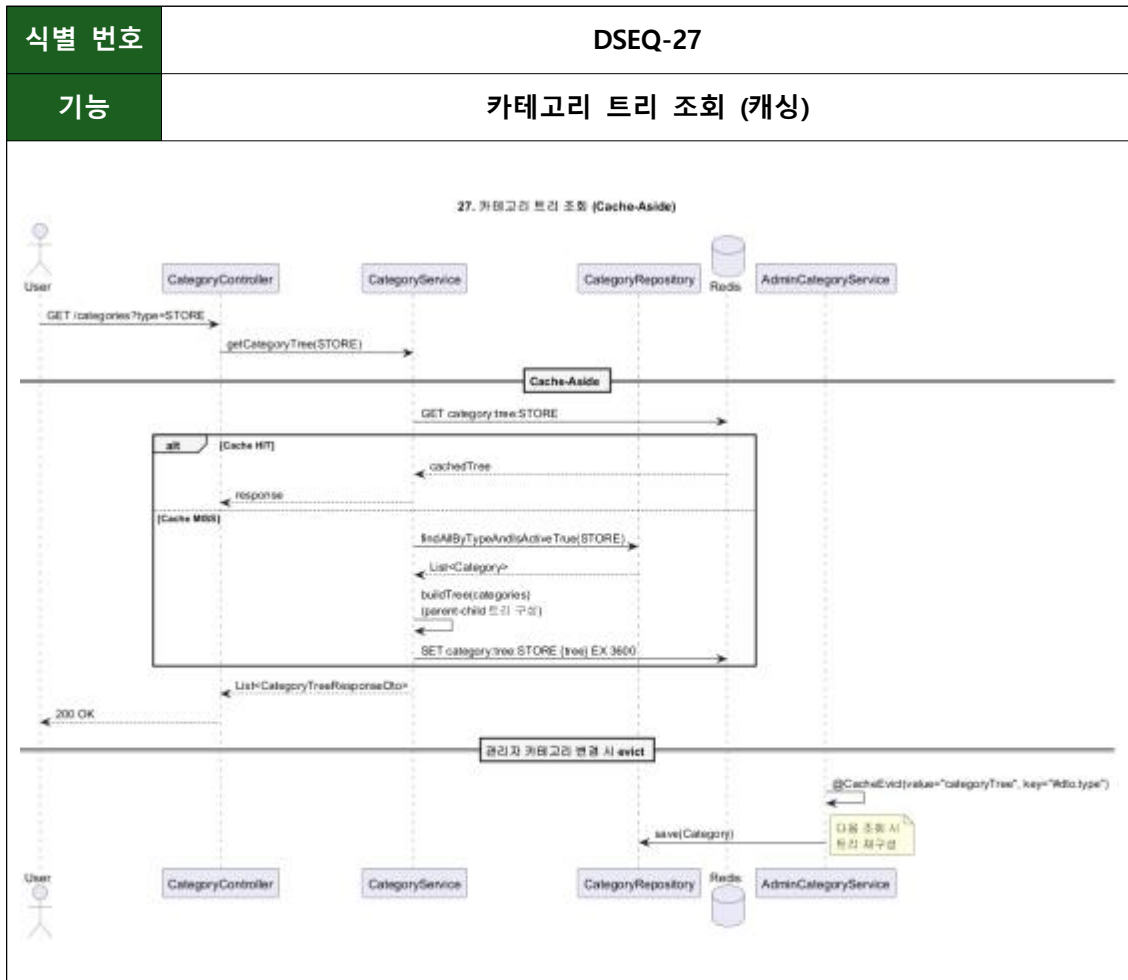
DSEQ-25

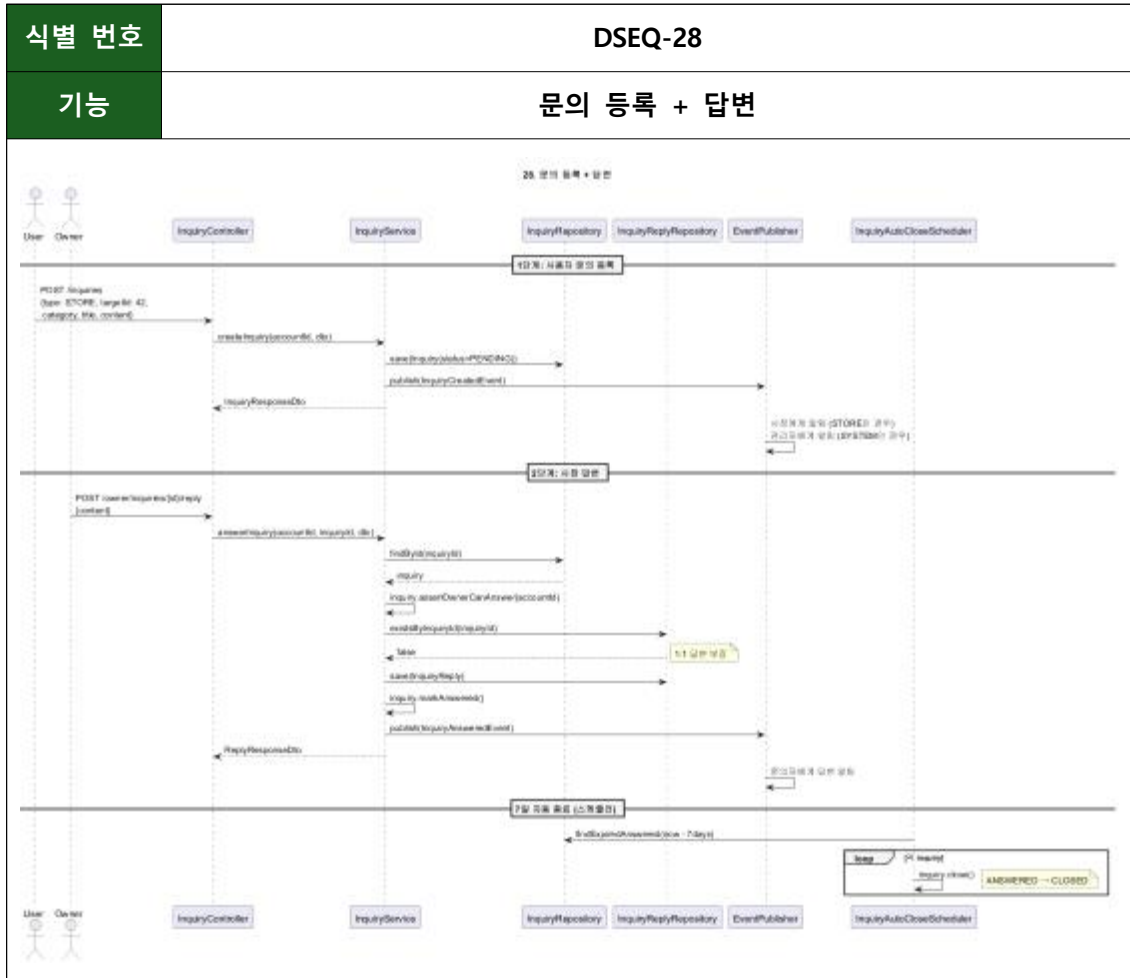
기능

찜 토글

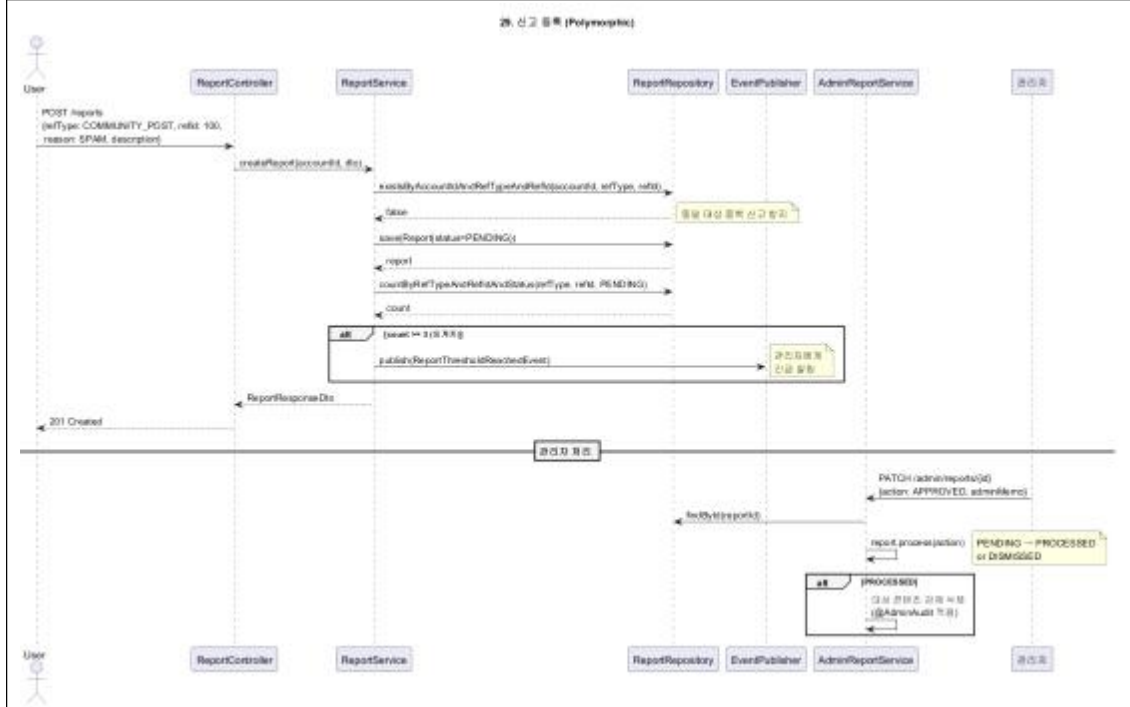








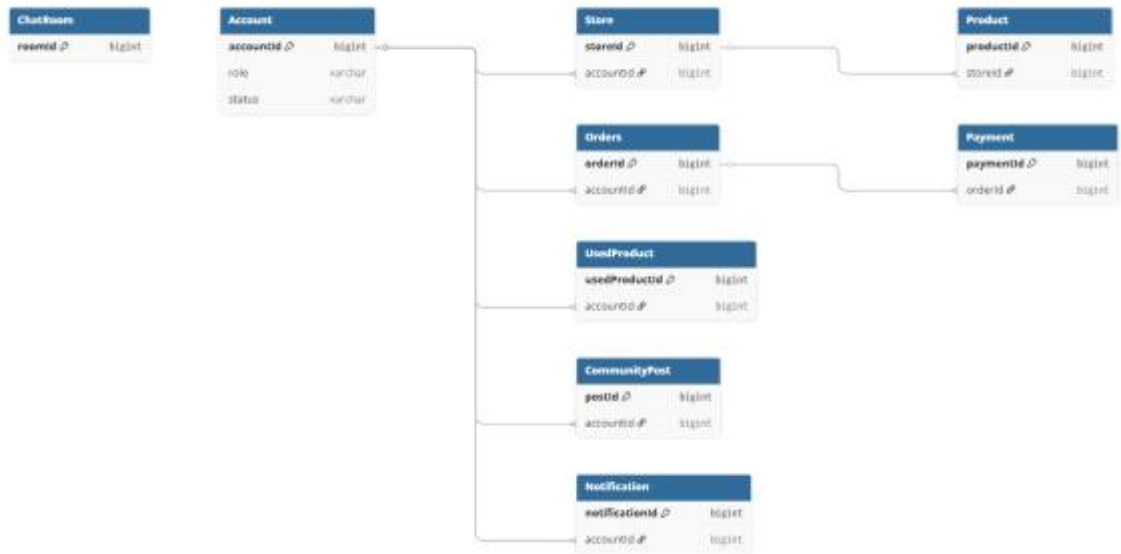
식별 번호	DSEQ-29
기능	신고 등록



[illegible]

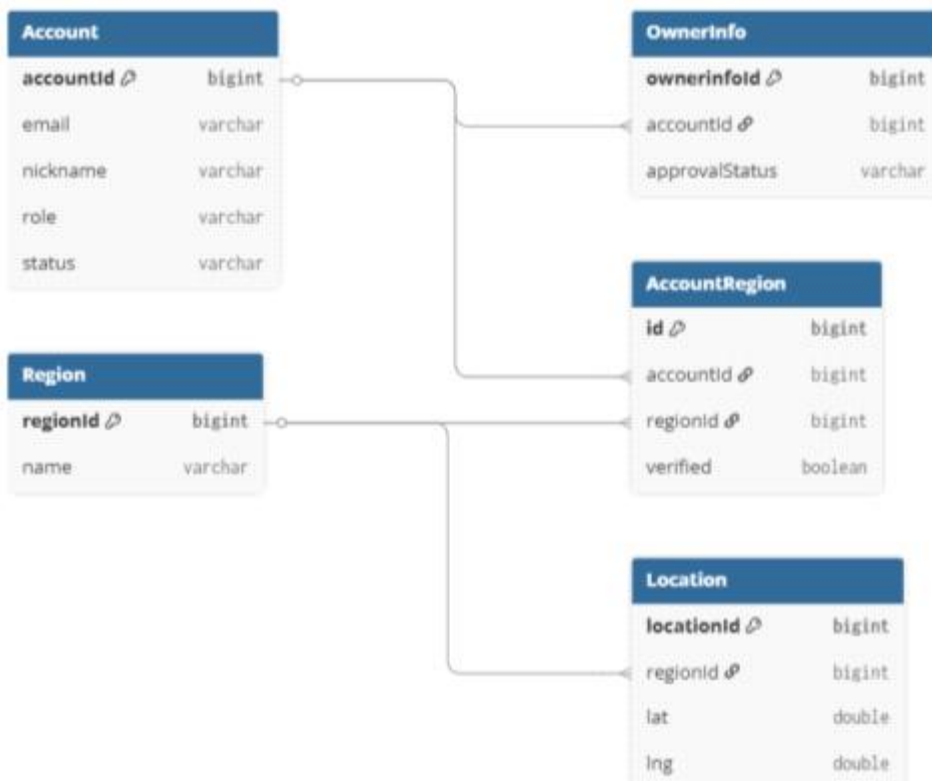
9. 데이터베이스 설계

9.1 Core ERD



9.2 도메인 ERD

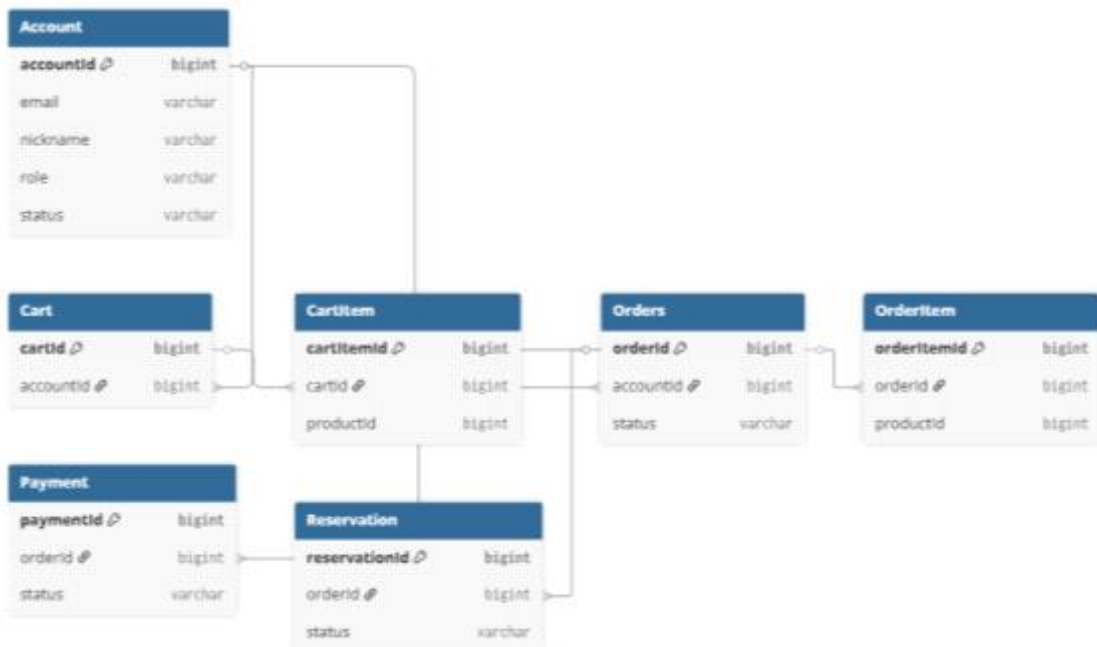
- Account / Region 도메인



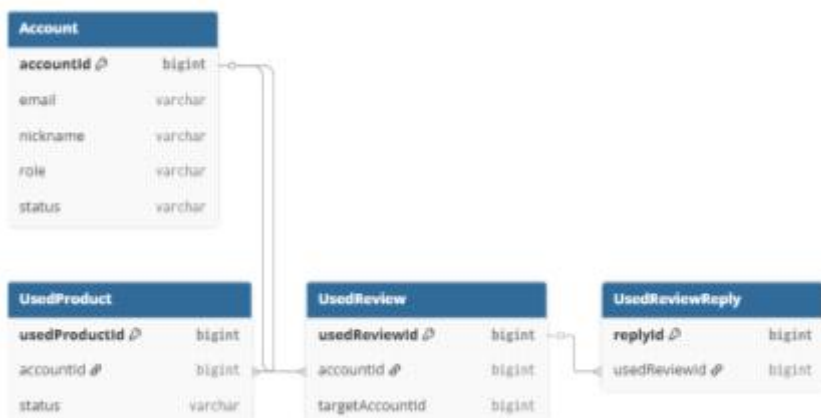
- Store / Product 도메인



- Order / Payment 도메인



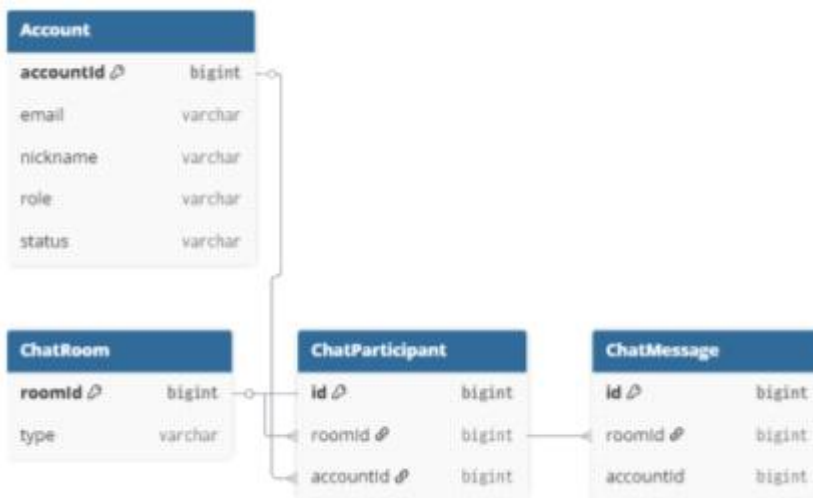
- UsedProduct 도메인



- Community 도메인



- Chat 도메인



- Admin / Notification 도메인



9.3 테이블 정의

Account (회원)			
컬럼명	타입	제약조건	설명
account_id	bigint	PK	회원 고유 식별자
primary_region_id	bigint	대표 지역	대표 활동 지역
email	varchar(255)	이메일 (유니크)	로그인 이메일
password	varchar(255)	비밀번호	BCrypt 암호화. OAuth 시null
name	varchar(100)	이름	실명
provider	varchar(20)	OAuth 제공자	OAuthProvider ENUM
provider_id	varchar(255)	OAuth 식별자	OAuth 식별자. LOCAL이면null
profile_image_url	varchar(500)	프로필 이미지	프로필 이미지URL
nickname	varchar(50)	닉네임 (유니크)	닉네임
role	varchar(20)	회원 역할	AccountRole ENUM
status	varchar(20)	회원 상태	AccountStatus ENUM
email_verified	boolean	이메일 인증 여부	이메일 인증 여부
fcm_token	varchar(255)	FCM 토큰	푸시 발송용 디바이스 토큰
created_at	datetime	생성일	BaseEntity 상속
modified_at	datetime	수정일	BaseEntity 상속
deleted_at	datetime	삭제일	탈퇴 시점(Soft Delete, 30일 유예)

OwnerInfo (사장 정보)			
컬럼명	타입	설명	설명
owner_info_id	bigint	PK, NOT NULL, AUTO_INCREMENT	사장 회원 고유 식별자
account_id	bigint	FK(account), UNIQUE, NOT NULL	연관 회원(1:1)
phone	varchar(20)	NOT NULL	사업장 연락처
business_number	varchar(50)	UNIQUE, NOT NULL	사업자 등록번호
approval_status	varchar(20)	NOT NULL	ApprovalStatus ENUM
rejection_reason	varchar(255)	nullable	거절 사유
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

AccountRegion (회원 지역)			
컬럼명	타입	설명	설명
account_region_id	bigint	PK, NOT NULL, AUTO_INCREMENT	회원 지역 고유 식별자
account_id	bigint	FK(account), NOT NULL	연관 회원
region_id	bigint	FK(region), NOT NULL	연관 지역
verified	boolean	NOT NULL, DEFAULT false	GPS 인증 완료 여부
verified_at	datetime	nullable	인증 시각
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

Region (지역)			
컬럼명	타입	설명	설명
region_id	bigint	PK, NOT NULL, AUTO_INCREMENT	지역 고유 식별자
region_code	varchar(20)	UNIQUE, NOT NULL	행정동 코드
si_do	varchar(50)	NOT NULL	시/도
gun_gu	varchar(50)	NOT NULL	시/군/구
dong	varchar(50)	NOT NULL	행정동명
radius	int	NOT NULL	서비스 반경(m)

Location (지역 좌표)			
컬럼명	타입	설명	설명
location_id	bigint	PK, NOT NULL, AUTO_INCREMENT	좌표 고유 식별자
region_id	bigint	FK(region), UNIQUE, NOT NULL	연관 지역(1:1)
latitude	double	NOT NULL	위도
longitude	double	NOT NULL	경도

AdminAuditLog (관리자 감시 로그)			
컬럼명	타입	설명	설명
audit_log_id	bigint	PK, NOT NULL, AUTO_INCREMENT	관리자 로그 고유 식별자
account_id	bigint	FK(account), NOT NULL	작업한 관리자
action_type	varchar(50)	NOT NULL	AuditActionType ENUM
target_type	varchar(50)	NOT NULL	대상 도메인
target_id	bigint	nullable	대상ID (전역 작업은null)
parameters	text	nullable	메서드 파라미터JSON 직렬화
result	varchar(20)	NOT NULL	AuditResult ENUM
failure_reason	varchar(1000)	nullable	실패 사유(예외 메시지)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

Store (상점)			
컬럼명	타입	설명	설명
store_id	bigint	PK, NOT NULL, AUTO_INCREMENT	상점 고유 식별자
account_id	bigint	FK(account), UNIQUE, NOT NULL	상점 소유자(사장1:1 상점 제약)
region_id	bigint	FK(region), NOT NULL	상점 소속 지역
category_id	bigint	FK(category), NOT NULL	상점 카테고리(type=STORE)
latitude	double	NOT NULL	상점 위도
longitude	double	NOT NULL	상점 경도
name	varchar(100)	NOT NULL	상점명
address	varchar(255)	NOT NULL	주소
phone	varchar(20)	NOT NULL	상점 연락처
description	text	nullable	상점 소개
business_hours	varchar(255)	NOT NULL	영업시간
rating	double	NOT NULL, DEFAULT 0.0	평균 평점(캐싱)
favorite_count	int	NOT NULL, DEFAULT 0	상점 좋아요 수
review_count	int	NOT NULL, DEFAULT 0	리뷰 수(캐싱)
status	varchar(20)	NOT NULL	StoreStatus ENUM
version	bigint	NOT NULL	낙관적 락(@Version) - 평점 갱신 동시성
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

StoreNotuce (상점 공지)			
컬럼명	타입	설명	설명
notice_id	bigint	PK, NOT NULL, AUTO_INCREMENT	상점 공지 고유 식별자
store_id	bigint	FK(store), NOT NULL	연관 상점
notice_type	varchar(20)	NOT NULL	NoticeType ENUM
content	text	NOT NULL	공지 상세 내용
is_active	boolean	NOT NULL, DEFAULT true	공지 노출 여부
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

StoreImage (상점 이미지)			
컬럼명	타입	설명	설명
store_image_id	bigint	PK, NOT NULL, AUTO_INCREMENT	이미지 고유 식별자(ImageBase 상속)
store_id	bigint	FK(store), NOT NULL	연관 상점
image_url	varchar(500)	NOT NULL	S3 이미지URL (ImageBase 상속)
display_order	int	NOT NULL, DEFAULT 0	표시 순서(ImageBase 상속)
is_thumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부(ImageBase 상속)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

ProductCategory (상점 카테고리)			
컬럼명	타입	설명	설명
product_category_id	bigint	PK, NOT NULL, AUTO_INCREMENT	상품 카테고리 고유 식별자
store_id	bigint	FK(store), NOT NULL	소속 상점
name	varchar(50)	NOT NULL	상품 카테고리명
display_order	int	NOT NULL, DEFAULT 0	표시 순서
is_active	boolean	NOT NULL, DEFAULT true	노출 여부(Soft Delete)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

Product (상품)			
컬럼명	타입	설명	설명
product_id	bigint	PK, NOT NULL, AUTO_INCREMENT	상품 고유 식별자
store_id	bigint	FK(store), NOT NULL	소속 상점
product_category_id	bigint	FK(product_category), NOT NULL	상품 카테고리
name	varchar(100)	NOT NULL	상품명
description	text	NOT NULL	상품 설명
price	decimal(10,2)	NOT NULL, ≥0	가격
stock	int	nullable, ≥0	재고 수량(null=무제한)
view_count	int	NOT NULL, DEFAULT 0	조회수(Redis 집계→ 5분 주기)
status	varchar(20)	NOT NULL, DEFAULT 'ACTIVE'	ProductStatus ENUM
version	bigint	NOT NULL	낙관적 락(@Version) - 재고 차감
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

ProductImage (상품 이미지)			
컬럼명	타입	설명	설명
product_image_id	bigint	PK, NOT NULL, AUTO_INCREMENT	이미지 고유 식별자(ImageBase 상속)
product_id	bigint	FK(product), NOT NULL	연관 상품
image_url	varchar(500)	NOT NULL	S3 이미지URL
display_order	int	NOT NULL, DEFAULT 0	표시 순서
is_thumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

ProductOption (상품 옵션)			
컬럼명	타입	설명	설명
product_option_id	bigint	PK, NOT NULL, AUTO_INCREMENT	상품 옵션 고유 식별자
product_id	bigint	FK(product), NOT NULL	연관 상품
group_name	varchar(50)	NOT NULL	옵션 그룹 이름(예: 용량, 매운맛)
selection_type	varchar(20)	NOT NULL, DEFAULT 'SINGLE'	OptionSelectionType ENUM
is_required	boolean	NOT NULL, DEFAULT false	옵션 선택 필수 여부
display_order	int	NOT NULL, DEFAULT 0	옵션 그룹 노출 순서
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

ProductOptionItem (상품 옵션 선택지)			
컬럼명	타입	설명	설명
product_option_item_id	bigint	PK, NOT NULL, AUTO_INCREMENT	상품 옵션 선택지 고유 식별자
product_option_id	bigint	FK(product_option), NOT NULL	연관 상품 옵션 그룹
item_name	varchar(50)	NOT NULL	옵션 선택 값(예: 300g, 매운맛)
additional_price	decimal(10,2)	NOT NULL, DEFAULT 0	선택 시 추가 금액
is_default	boolean	NOT NULL, DEFAULT false	기본 선택 여부
display_order	int	NOT NULL, DEFAULT 0	옵션 항목 노출 순서
is_available	boolean	NOT NULL, DEFAULT true	옵션 선택 가능 여부(품질/비활성)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

EventProduct (이벤트 상품)			
컬럼명	타입	설명	설명
event_product_id	bigint	PK, NOT NULL, AUTO_INCREMENT	이벤트 상품 고유 식별자
product_id	bigint	FK(product), NOT NULL	연관 상품
active_product_id	bigint	GENERATED COLUMN, UNIQUE	'ACTIVE' 일 때 product_id, 그 외 NULL
discount_rate	int	NOT NULL, 0~99	할인율(%)
stock	int	nullable, ≥0	이벤트 수량 (null=무제한)
start_date	datetime	NOT NULL	이벤트 시작일
end_date	datetime	NOT NULL	이벤트 종료일
status	varchar(20)	NOT NULL	EventStatus ENUM
version	bigint	NOT NULL	낙관적 락(@Version) - 이벤트 재고 차감
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

StoreReview (상점 리뷰)			
컬럼명	타입	설명	설명
store_review_id	bigint	PK, NOT NULL, AUTO_INCREMENT	상점 리뷰 고유 식별자
store_id	bigint	FK(store), NOT NULL	연관 상점
account_id	bigint	FK(account), NOT NULL	작성자
order_id	bigint	FK(order), UNIQUE, NOT NULL	1주문 1리뷰 보장
rating	int	NOT NULL, 1~5	별점
content	text	NOT NULL	리뷰 내용
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

StoreReviewImage (상점 리뷰 이미지)			
컬럼명	타입	설명	설명
store_review_image_id	bigint	PK, NOT NULL, AUTO_INCREMENT	이미지 고유 식별자(ImageBase 상속)
store_review_id	bigint	FK(store_review), NOT NULL	연관 상점 리뷰
image_url	varchar(500)	NOT NULL	S3 이미지URL
display_order	int	NOT NULL, DEFAULT 0	표시 순서
is_thumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

StoreReviewReply (상점 리뷰 답글)			
컬럼명	타입	설명	설명
store_reply_id	bigint	PK, NOT NULL, AUTO_INCREMENT	상점 리뷰 답글 고유 식별자
store_review_id	bigint	FK(store_review), UNIQUE, NOT NULL	연관 리뷰(1:1)
account_id	bigint	FK(account), NOT NULL	답글 작성자(사장 회원)
content	text	NOT NULL	답글 내용
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

Order (주문)			
컬럼명	타입	설명	설명
order_id	bigint	PK, NOT NULL, AUTO_INCREMENT	주문 고유 식별자
account_id	bigint	FK(account), NOT NULL	구매자
store_id	bigint	FK(store), NOT NULL	주문 상점
total_price	decimal(10,2)	NOT NULL	총 결제 금액
order_number	varchar(50)	UNIQUE, NOT NULL	사용자 표시용 주문번호
version	bigint	NOT NULL	낙관적 락(@Version)
status	varchar(20)	NOT NULL	OrderStatus ENUM
paid_at	datetime	nullable	결제 완료 시점
cancelled_at	datetime	nullable	취소 시점
cancel_reason	varchar(500)	nullable	취소 사유
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

OrderItem (주문 상품)			
컬럼명	타입	설명	설명
order_item_id	bigint	PK, NOT NULL, AUTO_INCREMENT	주문 상품 고유 식별자
order_id	bigint	FK(order), NOT NULL	연관 주문
product_id	bigint	FK(product), nullable	연관 상품
event_product_id	bigint	FK(event_product), nullable	연관 이벤트 상품
product_name	varchar(100)	NOT NULL	주문 시점 상품명 스냅샷
selected_options	text	NOT NULL	주문 시 선택된 옵션 스냅샷(JSON)
quantity	int	NOT NULL, ≥1	수량
price	decimal(10,2)	NOT NULL	주문 시점 가격 고정

Payment (결제)			
컬럼명	타입	설명	설명
payment_id	bigint	PK, NOT NULL, AUTO_INCREMENT	결제 고유 식별자
order_id	bigint	FK(order), UNIQUE, NOT NULL	연관 주문(1:1)
account_id	bigint	FK(account), NOT NULL	결제자
portone_payment_id	varchar(100)	UNIQUE, NOT NULL	PortOne 결제 고유 번호
idempotency_key	varchar(100)	UNIQUE, NOT NULL	중복 결제 방지(멥등성 키)
amount	decimal(10,2)	NOT NULL	결제 금액
status	varchar(20)	NOT NULL	PaymentStatus ENUM
pg_provider	varchar(50)	nullable	PG사 이름
payment_method	varchar(20)	NOT NULL	PaymentMethod ENUM
paid_at	datetime	nullable	결제 완료 시점
cancelled_at	datetime	nullable	취소 시점
fail_reason	varchar(500)	nullable	결제 실패 사유(PortOne 응답)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

Reservation (예약)			
컬럼명	타입	설명	설명
reservation_id	bigint	PK, NOT NULL, AUTO_INCREMENT	예약 고유 식별자
account_id	bigint	FK(account), NOT NULL	예약자
store_id	bigint	FK(store), NOT NULL	예약 상점
product_id	bigint	FK(product), nullable	예약 상품(상품 예약 시)
order_id	bigint	FK(order), nullable	연관 주문
reserved_at	datetime	NOT NULL	예약 일시
status	varchar(20)	NOT NULL	ReservationStatus ENUM
reservation_type	varchar(20)	NOT NULL	ReservationType ENUM
total_price	decimal(10,2)	nullable	PRODUCT 타입일 때만 값 존재
capacity	int	nullable	예약 최대 가능 인원(VISIT 타입에서만)
version	bigint	NOT NULL	낙관적 락(@Version)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

Cart (장바구니)			
컬럼명	타입	설명	설명
cart_id	bigint	PK, NOT NULL, AUTO_INCREMENT	장바구니 고유 식별자
account_id	bigint	FK(account), UNIQUE, NOT NULL	회원
store_id	bigint	FK(store), NOT NULL	동일 상점 제약 관리
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

CartItem (장바구니 상품)			
컬럼명	타입	설명	설명
cart_item_id	bigint	PK, NOT NULL, AUTO_INCREMENT	장바구니 상품 고유 식별자
cart_id	bigint	FK(cart), NOT NULL	연관 장바구니
product_id	bigint	FK(product), nullable	연관 상품
event_product_id	bigint	FK(event_product), nullable	연관 이벤트 상품
selected_option_item_ids	text	nullable	선택한 옵션ID 목록(JSON)
options_total_price	decimal(10,2)	NOT NULL, DEFAULT 0	옵션 추가 가격 총합
selected_options_hash	varchar(64)	NOT NULL	옵션 조합 구분용 해시값
quantity	int	NOT NULL, ≥1	수량
price	decimal(10,2)	NOT NULL	추가 당시 가격 고정

UsedProduct (중고 상품)			
컬럼명	타입	설명	설명
used_product_id	bigint	PK, NOT NULL, AUTO_INCREMENT	중고 상품 고유 식별자
account_id	bigint	FK(account), NOT NULL	판매자
buyer_id	bigint	FK(account), nullable	구매자(예약/완료 시 설정)
region_id	bigint	FK(region), NOT NULL	게시 활동 지역
category_id	bigint	FK(category), NOT NULL	카테고리(type=USED 검증은Service)
title	varchar(100)	NOT NULL	제목
description	text	NOT NULL	상품 설명
price	decimal(10,2)	NOT NULL, ≥0	희망 가격
trade_location	varchar(255)	nullable	거래 희망 장소
view_count	int	NOT NULL, DEFAULT 0	조회수(Redis 캐싱)
status	varchar(20)	NOT NULL	UsedProductStatus ENUM
favorite_count	int	NOT NULL, DEFAULT 0	중고거래 좋아요 수
is_buyer_confirmed	boolean	NOT NULL, DEFAULT false	구매자 거래 완료 확인 여부
version	bigint	NOT NULL	낙관적 락(@Version)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

UsedProductImage (중고 상품 이미지)			
컬럼명	타입	설명	설명
used_product_image_id	bigint	PK, NOT NULL, AUTO_INCREMENT	이미지 고유 식별자(ImageBase 상속)
used_product_id	bigint	FK(used_product), NOT NULL	연관 중고 상품
image_url	varchar(500)	NOT NULL	S3 이미지URL
display_order	int	NOT NULL, DEFAULT 0	표시 순서
is_thumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

UsedProductReview (중고 거래 리뷰)			
컬럼명	타입	설명	설명
used_product_review_id	bigint	PK, NOT NULL, AUTO_INCREMENT	중고 거래 리뷰 고유 식별자
used_product_id	bigint	FK(used_product), UNIQUE, NOT NULL	연관 거래(중복 방지)
account_id	bigint	FK(account), NOT NULL	작성자
rating	int	NOT NULL, 1~5	별점
content	text	NOT NULL	리뷰 내용
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

CommunityPost (커뮤니티 게시글)			
컬럼명	타입	설명	설명
community_post_id	bigint	PK, NOT NULL, AUTO_INCREMENT	커뮤니티 고유 식별자
account_id	bigint	FK(account), NOT NULL	작성자
category_id	bigint	FK(category), NOT NULL	카테고리(type=COMMUNITY)
region_id	bigint	FK(region), NOT NULL	게시 활동 지역
title	varchar(100)	NOT NULL	제목
content	text	NOT NULL	본문
view_count	int	NOT NULL, DEFAULT 0	조회수(Redis 캐싱)
like_count	int	NOT NULL, DEFAULT 0	좋아요 수(Redis 캐싱)
comment_count	int	NOT NULL, DEFAULT 0	댓글 수(Redis 캐싱)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

CommunityImage (게시글 이미지)			
컬럼명	타입	설명	설명
community_image_id	bigint	PK, NOT NULL, AUTO_INCREMENT	이미지 고유 식별자(ImageBase 상속)
community_post_id	bigint	FK(community_post), NOT NULL	연관 게시글
image_url	varchar(500)	NOT NULL	S3 이미지URL
display_order	int	NOT NULL, DEFAULT 0	표시 순서
is_thumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

CommunityComment (댓글)			
컬럼명	타입	설명	설명
community_comment_id	bigint	PK, NOT NULL, AUTO_INCREMENT	댓글 고유 식별자
community_post_id	bigint	FK(community_post), NOT NULL	연관 게시글
account_id	bigint	FK(account), NOT NULL	작성자
content	text	NOT NULL	댓글 내용
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

CommunityCommentReply (대댓글)			
컬럼명	타입	설명	설명
community_comment_reply_id	bigint	PK, NOT NULL, AUTO_INCREMENT	대댓글 고유 식별자
community_comment_id	bigint	FK(community_comment), NOT NULL	연관 댓글
account_id	bigint	FK(account), NOT NULL	작성자
content	text	NOT NULL	대댓글 내용
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

PostLike (게시글 좋아요)			
컬럼명	타입	설명	설명
post_like_id	bigint	PK, NOT NULL, AUTO_INCREMENT	게시글 좋아요 고유 식별자
account_id	bigint	FK(account), NOT NULL	좋아요 누른 사용자
community_post_id	bigint	FK(community_post), NOT NULL	좋아요 대상 게시글

ChatRoom (채팅방)			
컬럼명	타입	설명	설명
chat_room_id	bigint	PK, NOT NULL, AUTO_INCREMENT	채팅방 고유 식별자
created_by	bigint	FK(account), NOT NULL	채팅방 생성자
type	varchar(20)	NOT NULL	ChatRoomType ENUM
ref_type	varchar(20)	nullable	ChatRoomRefType ENUM
ref_id	bigint	nullable	Polymorphic 참조ID (FK 아님)
name	varchar(100)	nullable	채팅방 이름(GROUP 타입)
is_active	boolean	NOT NULL, DEFAULT true	채팅방 활성 상태
last_message_at	datetime	nullable	마지막 메시지 시각(목록 정렬용)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

ChatParticipant (채팅 참여자)			
컬럼명	타입	설명	설명
chat_participant_id	bigint	PK, NOT NULL, AUTO_INCREMENT	채팅 참여자 고유 식별자
chat_room_id	bigint	FK(chat_room), NOT NULL	연관 채팅방
account_id	bigint	FK(account), NOT NULL	참여 사용자
last_read_time	datetime	nullable	마지막 읽은 시각(안 읽은 수 계산)
status	varchar(20)	NOT NULL, DEFAULT 'ACTIVE'	ParticipantStatus ENUM
joined_at	datetime	NOT NULL	참여 시각
left_at	datetime	nullable	퇴장 시각
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

ChatMessage (채팅 메시지)			
컬럼명	타입	설명	설명
chat_message_id	bigint	PK, NOT NULL, AUTO_INCREMENT	채팅 메시지 고유 식별자
chat_room_id	bigint	FK(chat_room), NOT NULL	연관 채팅방
account_id	bigint	FK(account), NOT NULL	발신자
content	text	nullable	메시지 내용(TEXT/SYSTEM 타입)
image_url	varchar(500)	nullable	이미지URL (IMAGE 타입)
message_type	varchar(20)	NOT NULL	MessageType ENUM
is_deleted	boolean	NOT NULL, DEFAULT false	발신자 본인 삭제 여부(Soft Delete)
sent_at	datetime	NOT NULL, INDEX	발신 시각(인덱스 대상)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

Category (전역 카테고리)			
컬럼명	타입	설명	설명
category_id	bigint	PK, NOT NULL, AUTO_INCREMENT	전역 카테고리 고유 식별자
type	varchar(20)	NOT NULL	CategoryType ENUM
parent_id	bigint	FK(category), nullable	상위 카테고리(대>중>소, 최상위null)
name	varchar(50)	NOT NULL	카테고리명
display_order	int	NOT NULL, DEFAULT 0	같은 부모 내 표시 순서
depth	int	NOT NULL, DEFAULT 1	1=대분류, 2=중분류, 3=소분류(최대3)
is_active	boolean	NOT NULL, DEFAULT true	노출 여부(Soft Delete 기본값)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

Favorite (찜)			
컬럼명	타입	설명	설명
favorite_id	bigint	PK, NOT NULL, AUTO_INCREMENT	찜 고유 식별자
account_id	bigint	FK(account), NOT NULL	찜한 사용자
ref_type	varchar(20)	NOT NULL	FavoriteRefType ENUM
ref_id	bigint	NOT NULL	Polymorphic 참조ID (FK 아님)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

Notification (알림)			
컬럼명	타입	설명	설명
notification_id	bigint	PK, NOT NULL, AUTO_INCREMENT	알림 고유 식별자
account_id	bigint	FK(account), NOT NULL	알림 수신자
type	varchar(50)	NOT NULL	NotificationType ENUM (25종)
title	varchar(100)	NOT NULL	알림 제목(푸시 표시용)
content	varchar(500)	NOT NULL	알림 본문
ref_type	varchar(30)	nullable	NotificationRefType ENUM
ref_id	bigint	nullable	Polymorphic 참조ID (FK 아님)
link_url	varchar(500)	nullable	클라이언트 딥링크URL
is_read	boolean	NOT NULL, DEFAULT false	읽음 여부
read_at	datetime	nullable	읽은 시각
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

NotificationSettings (알림 설정)			
컬럼명	타입	설명	설명
notification_settings_id	bigint	PK, NOT NULL, AUTO_INCREMENT	알림 설정 고유 식별자
account_id	bigint	FK(account), UNIQUE, NOT NULL	사용자
order_enabled	boolean	NOT NULL, DEFAULT true	주문/결제 알림(필수)
reservation_enabled	boolean	NOT NULL, DEFAULT true	예약 알림(필수)
chat_enabled	boolean	NOT NULL, DEFAULT true	채팅 알림
community_enabled	boolean	NOT NULL, DEFAULT true	커뮤니티 알림(댓글/좋아요)
store_review_enabled	boolean	NOT NULL, DEFAULT true	상점/리뷰 알림
used_product_enabled	boolean	NOT NULL, DEFAULT true	중고거래 알림
system_enabled	boolean	NOT NULL, DEFAULT true	시스템 공지(필수)
marketing_enabled	boolean	NOT NULL, DEFAULT false	마케팅/이벤트 알림(명시적 동의)
marketing_agreed_at	datetime	nullable	마케팅 수신 동의 시각(법적 증빙)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

Inquiry (문의)			
컬럼명	타입	설명	설명
inquiry_id	bigint	PK, NOT NULL, AUTO_INCREMENT	문의 고유 식별자
account_id	bigint	FK(account), NOT NULL	문의 작성자
inquiry_type	varchar(20)	NOT NULL	InquiryType ENUM
target_id	bigint	nullable	대상ID (STORE면store_id, SYSTEM이면null)
category	varchar(30)	NOT NULL	InquiryCategory ENUM
title	varchar(100)	NOT NULL	문의 제목
content	text	NOT NULL	문의 본문
status	varchar(20)	NOT NULL, DEFAULT 'PENDING'	InquiryStatus ENUM
answered_at	datetime	nullable	답변 시각(ANSWERED 전이 시)
closed_at	datetime	nullable	종료 시각(CLOSED 전이 시)
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

InquiryReply (문의 답변)			
컬럼명	타입	설명	설명
inquiry_reply_id	bigint	PK, NOT NULL, AUTO_INCREMENT	문의 답변 고유 식별자
inquiry_id	bigint	FK(inquiry), UNIQUE, NOT NULL	문의(1:1)
account_id	bigint	FK(account), NOT NULL	답변자(사장 또는 관리자)
content	text	NOT NULL	답변 본문
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

InquiryImage (문의 이미지)			
컬럼명	타입	설명	설명
inquiry_image_id	bigint	PK, NOT NULL, AUTO_INCREMENT	이미지 고유 식별자(ImageBase 상속)
inquiry_id	bigint	FK(inquiry), NOT NULL	연관 문의
image_url	varchar(500)	NOT NULL	S3 이미지URL
display_order	int	NOT NULL, DEFAULT 0	표시 순서
is_thumbnail	boolean	NOT NULL, DEFAULT false	대표 이미지 여부
created_at	datetime	NOT NULL	BaseEntity 상속
modified_at	datetime	NOT NULL	BaseEntity 상속

10. UI 설계

UI 설계는 일반 사용자용 모바일 앱(React Native)과 사장 회원·관리자용 웹(React)으로 구분하여 작성되었으며, 본 SDD의 API 설계 및 유스케이스 명세와 연계하여 활용한다.

본 시스템의 UI 설계는 Figma를 활용하여 별도 작성되었으며, 아래 링크를 통해 확인할 수 있다.

- Figma 디자인 링크

<https://www.figma.com/design/OCb1Og1xHyja7HlantFlr6/EEUM?node-id=0-1&t=xsgW4Zm1hyCB0p3C-1>

- Figma 사용자 UI

<https://www.figma.com/make/ET9I5hbN2eYBNA3IiS2KL/%EC%82%AC%EC%9A%A9%EC%9E%90-UI?t=EitkOeDdBmtvP0rj-1&preview-route=%2Fhome>

- Figma 관리자 UI

<https://www.figma.com/make/1ji7jbYheAtnSLr1i9IndM/%EA%B4%80%EB%A6%AC%EC%9E%90-%EB%8C%80%EC%8B%9C%EB%B3%B4%EB%93%9C?t=dB0mEG76sWX7Wlw0-6>

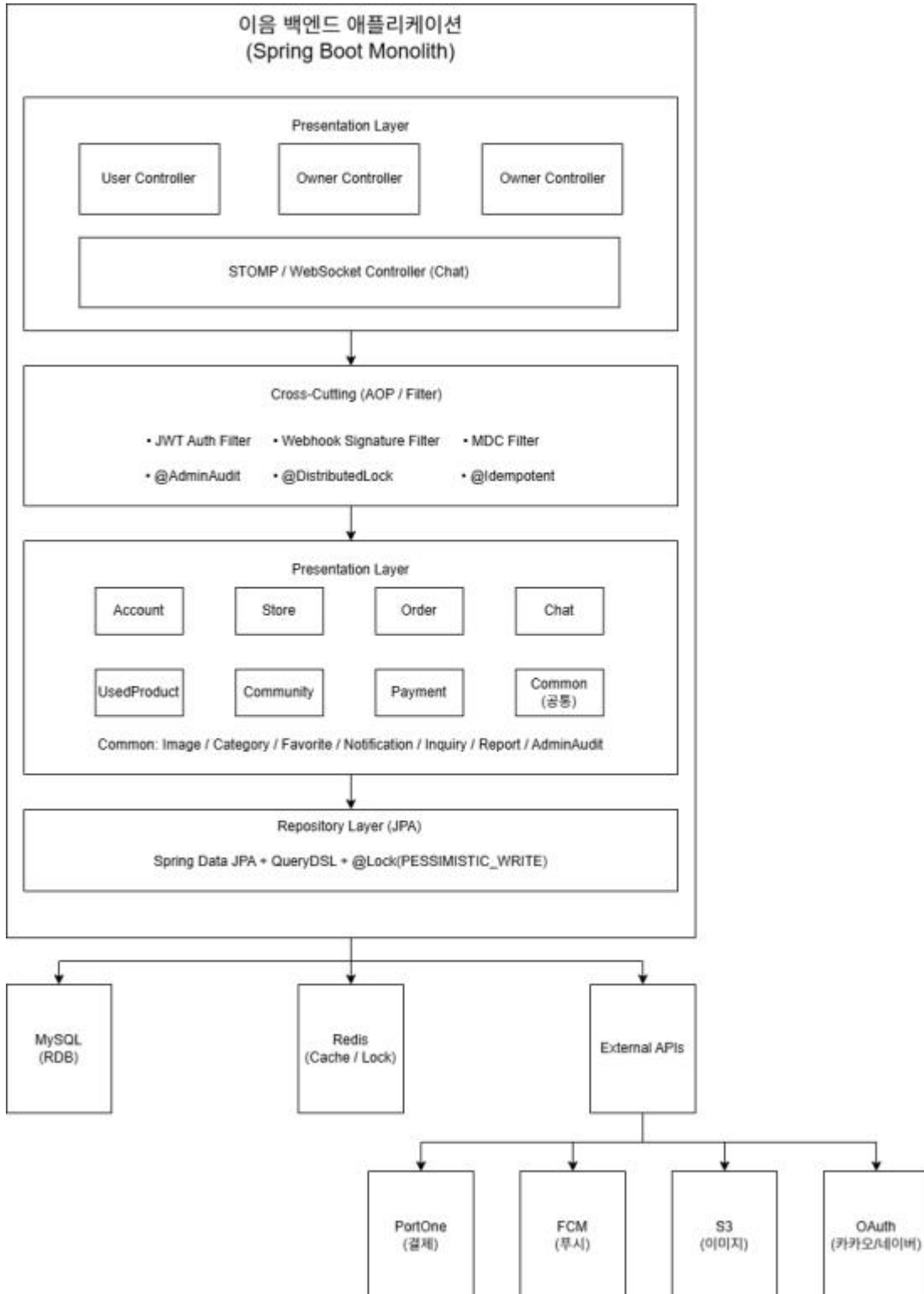
- Figma 사장회원 UI

<https://www.figma.com/make/xxalJltn6Fn5MRAxxZYBXn/%EC%82%AC%EC%9E%A5%EB%8B%98%EC%9A%A9-%EB%8C%80%EC%8B%9C%EB%B3%B4%EB%93%9C?t=dB0mEG76sWX7Wlw0-6>

11. 설계 컴포넌트 다이어그램

이름 시스템의 논리적 컴포넌트와 의존 관계를 표현한다. Spring Boot 기반 단일 애플리케이션이지만 계층 + 도메인으로 구성된 구조이다.

11.1 컴포넌트 구성도



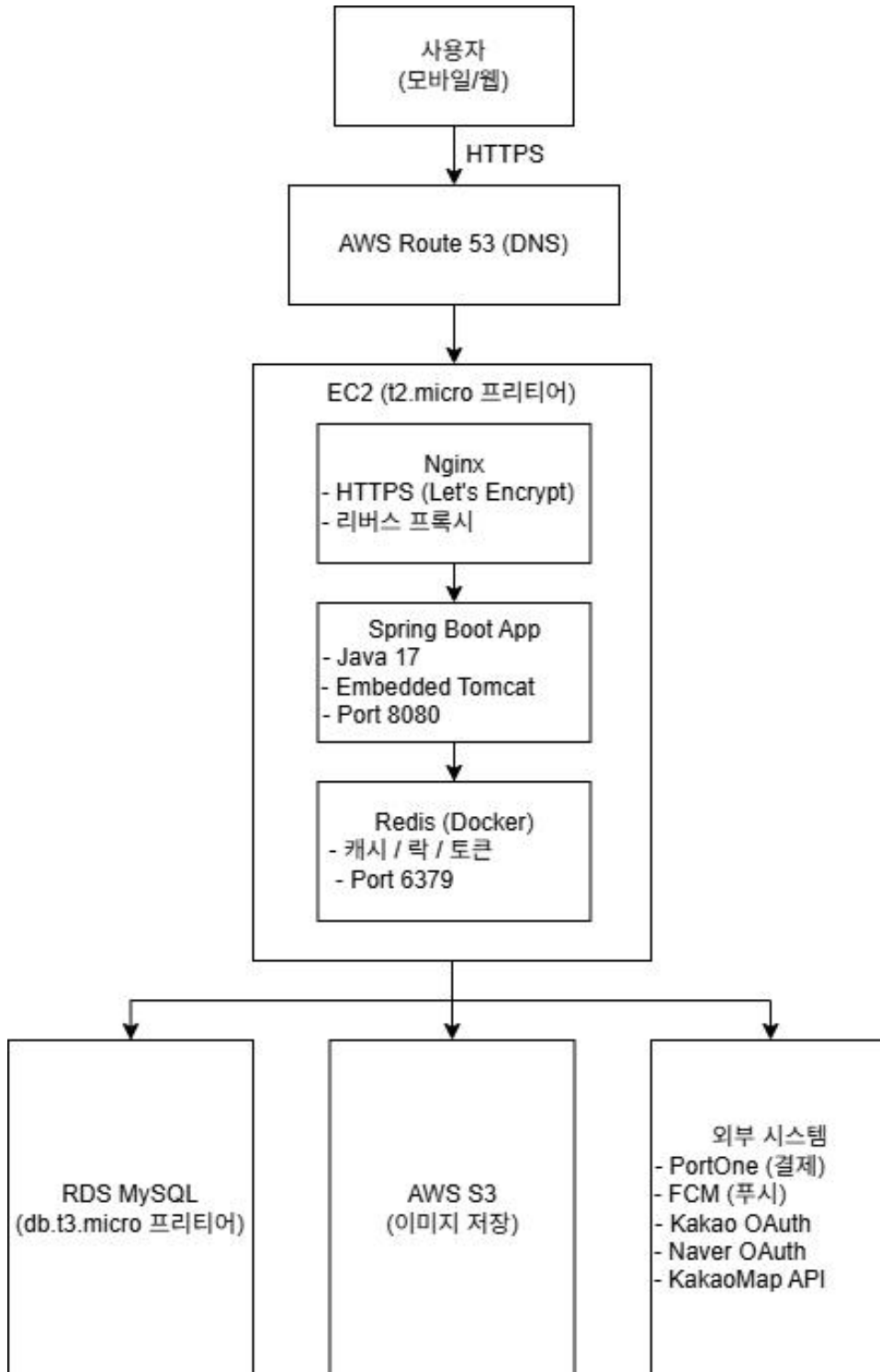
11.2 컴포넌트 의존 관계

컴포넌트	역할	주요 의존 대상
Presentation Layer	클라이언트 요청 수신 및 응답 반환 (REST,WebSocket)	Service Layer
Cross-Cutting (AOP)	인증, 감사, 분산 락, 멍등성 처리 등 공통 기능 수행	Redis, AdminAuditService
Service Layer	비즈니스 로직 수행 및 트랜잭션 경계 설정	Repository Layer, External APIs
Repository Layer	데이터베이스 접근 및 쿼리 수행	MySQL
MySQL	서비스 데이터 영속 저장	-
Redis	캐시, 분산 락, 토큰 저장, 읽지 않은 알림 수 관리	-
PortOne	외부 결제 시스템 연동 및 Webhook 처리	-
FCM	사용자 푸시 알림 전송	-
AWS S3	이미지 파일 저장 및 URL 제공	-
OAuth (Kakao / Naver)	소셜 로그인 인증 및 사용자 정보 제공	-

12. 설계 디플로먼트 다이어그램

이름 시스템의 물리적 배포 구조를 표현한다. AWS 프리티어 기반의 단일 인스턴스 구성이다.

12.1 배포 구성도



12.2 배포 노드 명세

이름 시스템은 AWS 프리티어 환경을 기반으로 단일 인스턴스 구조로 구성되며, 핵심 컴포넌트를 EC2 인스턴스에 통합하여 운영한다.

노드	종류	역할	사양
Route 53	DNS	도메인 → EC2 라우팅	(도메인 비용 발생)
EC2	애플리케이션 서버	Nginx + Spring Boot + Redis 통합 운영	t2.micro (1 vCPU, 1GB RAM)
Nginx	리버스 프록시	HTTPS 종료, 정적 파일 서빙	EC2 내부
Spring Boot	애플리케이션	비즈니스 로직 실행	Java 17, Embedded Tomcat
Redis	인메모리 저장소	캐시 / 분산 락 / 토큰 저장	Docker 컨테이너
RDS MySQL	RDB	영속 데이터 저장	db.t3.micro (20GB)
AWS S3	객체 스토리지	이미지 저장	5GB 무료
PortOne	외부 결제	결제 처리 + Webhook	외부 서비스
FCM	푸시 서비스	모바일/웹 알림 발송	무료
OAuth (Kakao/Naver)	인증	소셜 로그인	무료
KakaoMap API	지도	지도 표시 및 좌표 변환	무료 한도

- 설계 특징
 - 단일 EC2 인스턴스 기반 경량 아키텍처
 - 외부 서비스 적극 활용 (결제, 지도, 인증)
 - 이미지 및 정적 리소스는 S3로 분리

12.3 통신 프로토콜

시스템 구성 요소 간 통신은 다음과 같이 정의한다.

구간	프로토콜	포트	비고
사용자 ↔ Nginx	HTTPS (TLS)	443	Let's Encrypt 인증서 적용
Nginx ↔ Spring Boot	HTTP	8080	EC2 내부 통신
Spring Boot ↔ Redis	TCP	6379	localhost
Spring Boot ↔ RDS	TCP	3306	VPC 내부
Spring Boot ↔ S3	HTTPS	443	IAM Role 기반
Spring Boot ↔ 외부 API	HTTPS	443	API 호출 및 Webhook 수신
채팅 (WebSocket)	WSS	443	STOMP over WebSocket

- 설계 특징
 - 외부 통신은 모두 HTTPS 기반
 - 내부 통신은 성능을 고려하여 HTTP/TCP 사용
 - WebSocket을 통한 실시간 채팅 지원

12.4 프리티어 운영 전략

AWS 프리티어 환경 제약을 고려하여 비용 효율적인 구조로 설계하였다.

- 운영 전략
 - 단일 EC2 통합 운영
 - Nginx + Spring Boot + Redis를 하나의 인스턴스에서 실행
 - Redis Docker 사용
 - ElastiCache 미사용 → Docker 컨테이너로 직접 운영
 - RDS 분리 운영
 - 데이터 안정성을 위해 DB는 별도 관리
 - S3 활용
 - 이미지 저장을 S3로 분리 (Presigned URL 방식)
 - EC2 디스크 부담 최소화
 - CloudFront 미사용
 - 초기 트래픽이 적어 CDN 불필요
- 설계 의도
 - 비용 최소화
 - 단순 구조 유지
 - 빠른 개발 및 배포 가능

12.5 한계 및 향후 확장

현재 프리티어 환경은 비용 효율적이지만, 트래픽 증가 시 확장이 필요하다.

- 확장 전략

항목	현재(프리티어)	확장 시
EC2	t2.micro 1대	t3.medium 이상 + Auto Scaling
Redis	Docker (단일)	ElastiCache (Multi-AZ)
RDS	단일 인스턴스	Master + Read Replica
Load Balancer	Nginx 단독	ALB 도입
이미지 배포	S3 직접	CloudFront CDN 적용

- 확장 방향

- 수평 확장 (Scale-Out)
- 고가용성 구조 (Multi-AZ)
- 캐시 및 DB 분산 처리
- CDN을 통한 정적 리소스 최적화

13. 요구사항 추적표